

The Book of Remind

by Dianne Skoll

Copyright 2026 Dianne Skoll

Version 1.3 - 2026-05-05

Dedication

For Wayne and Rebecca

Copyright

This book is Copyright 2026 by Dianne Skoll. Except as noted in the license below, all rights are reserved.

License

If you are a Human Being, you may:

1. Read this book and make verbatim copies for your personal use.
2. Distribute verbatim copies of this book *at no charge* to other Human Beings (who then become subject to this License), with the following restriction: You may *not* place a copy of this book on a publicly-accessible Web server or FTP server or similar unless you take effective measures to ensure that it is only downloaded by humans and is not accessible to AI scrapers. Password-protecting access to the book would be considered an effective measure providing the password does not appear in plain-text on the server.

No part of this book may be used or reproduced in any manner for the purpose of training artificial intelligence technologies or systems.

Dianne Skoll expressly reserves this work from the text and data mining exception in the European Copyright Directive.

Acknowledgments

Although I wrote this book and Remind mostly by myself, the book and Remind have been immeasurably improved by the community of Remind users, especially those on the Remind-Fans mailing list.

I'd like to thank Tim Chase for reviewing an early draft of this book and for providing valuable feedback. He's written an extensive blog post about Remind at <https://blog.thechases.com/posts/remind/>, and has suggested many useful features that have made it into the software.

Thanks to Jochen Sprickerhof for reviewing this book, maintaining the Debian package of Remind and for writing the tools mentioned in Chapter 18 that allow synchronization of Remind calendars with calendars on other devices. He's also contributed many small fixes to Remind and the Remind man pages.

Thanks to Neil Hanlon for maintaining the EPEL Remind package.

Thanks to Ian! D. Allen for suggesting many improvements to Remind and to its documentation, and for contributing to the project financially.

The following people have also made contributions to Remind, submitted bug reports, donated to my Liberapay account, or have written software that works with Remind:

Ron Aaron, Valerio Aimale, Davide Alberani, Justin Alcorn, Mark Atwood, Greg Badros, Robert Black, Jim Budler, Jin Chen, Rafa Couto, Liviu Daia, Ian Darwin, Björn Davíðsson, Michael DeBusk, Larry De Coste, Laurent Duperval, Paul M. Foster, Daniel Graham, Gail Gurman, Darrel Hankerson, Russ Herman, Tina Hoeltig, Patric Hof, Robert Joop, Jonathan Kamens, Willem Kasdorp, Richard Kelly, Joop Kieft, Mathieu Laparie, Nimrod Levy, Tom Limoncelli, Mogens Lynnerup, Christopher J. Madsen, Shelagh Manton, Marek Marczykowski, John McGowan, Judah Milgram, Michael Neuhauser, Ed Oskiewicz, Marco Paganini, Paul Pelzl, Francois Pinard, Trygve Randen, Dave Rickel, Michael Salmon, David W. Sanderson, Randal L. Schwartz, Amos Shapir, Mikko Silvonen, Georg Simon, George M. Sipe, Jerzy Sobczyk, Detlef Steuer, David W. Tamkin, Wolfgang Thronicke, Frank Vance, Erik-Jan Vens, Norman Walsh, Tug Williams, Dave Wolfe, Hong Wu, Arthur G. Yaffe, and Frank Yellin,

I may well have missed some names; if so, please accept my apologies and also let me know so I can add your name to this list.

Donations

This book, like Remind, is free. However, should you wish to contribute financially to the Remind project, donations are gratefully accepted at Liberapay. The donation link is <https://dianne.skoll.ca/donate.php?rbook>

Contents

1	Introduction	1
1.1	What is Remind?	1
1.2	A Brief History of Remind	2
1.3	A Quick Example	3
1.4	The Organization of This Book	4
2	File Structure and Basic Reminders	7
2.1	The Structure of a Remind Script	7
2.1.1	Commands	7
2.1.2	Comments	8
2.1.3	Blank Lines	8
2.1.4	Continued Lines	8
2.1.5	Character Set and Encoding	9
2.1.6	Case Sensitivity	9
2.2	Running Remind	9
2.2.1	Some Remind Command-Line Options	10
2.3	The REM Command: An Introduction	11
2.4	The Remind Trigger-Calculation Algorithm	14
2.5	Advance Warning	14
2.6	The Substitution Filter	15
2.6.1	Some Substitution Sequences	16
2.6.2	Limiting the Text in Calendar Mode	16
2.7	The Banner	16
2.8	Counting Backwards	17
2.9	Some Syntactic Sugar	18

2.10	Checking What Will Happen on a Specific Date	19
2.11	The “Current” Date and Time	19
2.12	Running Remind Multiple Times in a Row	20
3	Holidays and Other Exceptions	21
3.1	Local OMITs	23
3.2	Saving and Restoring the Omit Context	24
3.3	Dumping the OMIT Context	24
3.4	Alternate Forms of Back and Delta	24
4	More Advanced Reminders	27
4.1	Arbitrary Recurrences	27
4.2	Specific Expiry Dates	28
4.3	Specific Starting Dates	28
4.4	Movable Holidays	28
4.4.1	Safe Movable Holidays	30
5	Timed Reminders	33
5.1	The AT Clause	33
5.1.1	Queuing Reminders	34
5.1.2	Fine Control over Queuing	34
5.2	Some Time-Related Substitution Sequences	34
5.3	Advance Warning	35
5.4	Specifying Durations	35
5.5	Syntactic Sugar for a Specific Date and Time	36
5.6	Priority	36
5.7	Sorted Output in Agenda Mode	36
6	Expressions	39
6.1	Variables	39
6.2	System Variables	39
6.3	Data Types	40
6.4	Constants	41
6.5	True and False Values	42
6.6	Operators	43

6.7	Order of Evaluation	44
6.8	Description of Operators	44
6.8.1	Unary Negation Operators	44
6.8.2	Multiplication, Division and Modulus	45
6.8.3	Addition	45
6.8.4	Subtraction	46
6.8.5	Comparison	47
6.8.6	Equality and Inequality Operators	47
6.8.7	Logical AND	47
6.8.8	Logical OR	48
6.9	Built-In Functions	48
6.9.1	Some Built-In Functions	49
6.10	User-Defined Functions	52
6.10.1	Redefining a Function	53
6.11	Recursive Functions	53
6.12	Commands for Manipulating Variables and Functions	54
6.12.1	Unsetting a Variable	54
6.12.2	Saving and Restoring Variables	54
6.12.3	Unsetting a Function	54
6.12.4	Saving and Restoring Functions	54
6.13	Expressions and Reminders	55
7	The SATISFY Keyword	57
7.1	The Operation of SATISFY	58
7.2	SATISFY Iteration Limit	59
7.3	Useful Functions and System Variables	60
7.4	A SATISFYing Gallery	61
7.4.1	The Fifth Weekday of a Month	61
7.4.2	Observed vs. Actual Holidays	61
7.4.3	US Presidential Election Day	62
7.4.4	Credit Card Due Date	62
7.4.5	Recycling Date	63
7.4.6	End-of-Quarter	64

8	Some Callback Functions	65
8.1	OMITFUNC	65
8.2	The WARN Keyword	66
8.3	The SCHED Keyword	67
8.4	Prefix and Suffix Callback Functions	67
8.5	Sortbanner	68
9	File Inclusion and Flow Control	71
9.1	File Inclusion	71
9.1.1	INCLUDE	71
9.1.2	DO	71
9.1.3	SYSINCLUDE	72
9.2	Conditional Execution	72
9.2.1	The IF Command	73
9.2.2	The IFTRIG Command	74
9.3	Reading Scripts from Directories	74
9.4	Command Inclusion	75
10	TODOs	77
10.1	The TODO Keyword	77
10.2	TODOs and Calendar Mode	78
11	Astronomical Events	79
11.1	Location	79
11.2	Sunrise and Sunset	80
11.3	Moon-Related Functions	81
11.3.1	Moon Phases	81
11.3.2	Moonrise and Moonset	82
11.3.3	Blue Moons	83
11.4	Solstices and Equinoxes	83
11.5	Easter	83
12	Producing Calendars	85
12.1	Calendars in the Terminal	85
12.1.1	If Your Terminal Calendar Displays are Messed Up	86

12.2	Additional Calendar-Related Command-Line Options	86
12.2.1	First Day of the Week	86
12.2.2	Size of the Calendar	86
12.2.3	Time Format	87
12.3	Machine-Readable Calendar Output	87
12.4	PDF Calendars	88
12.4.1	Generating PostScript and SVG Instead of PDF	89
12.5	HTML Calendars	89
12.6	Restricting Reminders to Calendar Mode or Agenda Mode	90
12.7	Special Data for Calendars	90
12.7.1	SHADE	91
12.7.2	MOON	92
12.7.3	WEEK	93
12.7.4	COLOR	93
12.7.5	Coloring Reminders by Default	98
12.8	Back-End-Specific SPECIALS	99
12.8.1	Rem2PDF SPECIAL: PANGO	99
12.8.2	Rem2HTML SPECIALS	100
13	The Hebrew Calendar	101
13.1	A Description of the Hebrew Calendar	101
13.2	Gregorian-to-Hebrew Conversion	102
13.3	Hebrew-to-Gregorian Conversion	103
13.3.1	Conversion of a Full Date	104
13.3.2	Conversion of Partial Dates	104
13.4	A Silly Example	105
14	The TkRemind Graphical Front-End	107
14.1	Main Window	109
14.1.1	Keyboard Shortcuts	109
14.2	Queued Reminders	110
14.3	Printing The Calendar	110
14.4	Options	111
14.4.1	Basic Options	112

14.4.2	Running a Command When Popping Up a Queued Reminder	113
14.4.3	Miscellaneous Options	113
14.4.4	Appearance	114
14.5	Adding Reminders	115
14.5.1	Basic Recurrences	116
14.5.2	Options	116
14.5.3	The Reminder Body	117
14.6	Editing Reminders	117
14.7	URLs, Location and Description	118
14.8	The Queue	118
14.9	Errors	118
14.10	Automatically Adjusting to Changes	118
14.11	Variables Set by TkRemind	118
14.12	Getting Help	119
15	Tags and Ancillary Information	121
15.1	TAG	121
15.2	INFO	121
15.2.1	INFO-related Substitution	122
15.2.2	URLs in the Terminal	122
16	Time Zones	125
16.1	Reminders in a Specific Time Zone	125
16.1.1	How TZ works	126
16.2	Time Zone Conversion Functions	127
16.3	Validating Time Zone Names	128
17	Remind and Shell Commands	129
17.1	Daemon Mode	129
17.2	The shell function	130
17.3	Safety	131
18	Interoperability	133
18.1	iCalendar and CalDAV	133
18.2	Importing Calendars into Remind	134

18.3	Exporting Calendars From Remind	134
18.4	Running Radicale	135
18.5	Radicale Remind Storage	135
19	Localization	137
19.1	The TRANSLATE Command	137
19.2	Functions and Filter Sequences	138
19.3	Localizing the Substitution Filter	139
19.3.1	Simplified Callbacks for Today/Tomorrow/Something Else	140
19.3.2	Custom Substitution Callbacks	140
19.4	Overriding ord()	141
19.5	How To Localize Remind	141
19.5.1	LANGID	141
19.5.2	The Banner	142
19.5.3	System Variables	142
19.5.4	Messages	142
19.6	More on TRANSLATE	142
19.7	Back-Ends	143
20	Debugging Remind Scripts	145
20.1	Debug Flags	145
20.1.1	Sample of t Debugging Output	146
20.1.2	Sample of x Debugging Output	146
20.2	Other Debugging Commands	148
21	Going Further	149
21.1	Example Project	149
21.2	Wrapping Up	150
A	Getting and Installing Remind	151
A.1	Prerequisites	151
A.2	Obtaining Remind	151
A.2.1	Downloading the Source Code	151
A.2.2	Prerequisites for Building Remind	151
A.3	Building Remind	153

A.4	Installing Remind	153
A.5	Uninstalling Remind	153
B	Calendar Systems	155
B.1	Reconciling Days, Months and Years	155
B.2	Year Numbering	157
B.3	Calendar Systems supported by Remind	157
C	Bibliography	159

Chapter 1

Introduction

Welcome! If you are reading this book, then you are probably interested in learning how to use [the Remind calendar program](#).

Remind does ship with fairly extensive documentation, but it's in the form of a UNIX man¹ page that expands to about 80 printed pages. While man pages make great reference material (and indeed, I encourage you to read the Remind man page from beginning to end) they are not particularly good as tutorials. This book aims to supplement the man page by teaching you how to use Remind step-by-step.

By the way: In several places in this book, I will write: "... see the **remind(1)**² man page." What this means is that you should open a terminal on your computer and type: `man remind`. Then read the resulting text.

1.1 What is Remind?

Remind is a sophisticated calendar and alarm program designed to run on Linux and other UNIX-like systems. Remind is primarily a *command-line* program. Although Remind does ship with a graphical front end (described in Chapter [14](#)), you need to be comfortable with editing script files and using the UNIX command line in a terminal to get the most out of Remind.

To use Remind, you use your favorite text editor³ to create a plain-text file called a *Remind script*. Be sure to create a *plain-text* file using a text editor for writing scripts or programs; using a word-processor will not produce a plain text file suitable for consumption by Remind.

A Remind script contains a sequence of commands that Remind reads and interprets. The output from interpreting the commands depends on how Remind is invoked; it could be a set of reminders for today, it could be a calendar drawn in your terminal, or it could be a machine-readable calendar suitable for

¹UNIX manual pages are called "man pages" because the command to read them is `man`.

²Referring to man pages as "**remind(1)**" is a UNIX tradition. It means that the man page is in Section 1 of the manual, which is devoted to user commands. You don't actually type the (1) part when you want to read the man page.

³Your favorite text editor is Emacs, right?

post-processing by another program.

In this book, I will distinguish between UNIX commands that you enter at the shell prompt, and Remind commands that you put in a Remind script, as follows: UNIX commands will be written in lower-case, like this: `remind` or `rem2ps`, while Remind commands will be written in upper-case, like this: `REM` or `OMIT`.

Remind’s scripting language is an example of a *domain-specific language* or DSL. It’s designed to let you express very concisely all kinds of complicated date specifications. As you’ll see later on, not only can Remind do simple things like remind you of weekly appointments or of people’s birthdays, but it can also do more complex things like reminding you of the second full moon in a calendar month, or telling you when sunrise and sunset are.

Remind never really started out with any roadmap or design goals, as will be evident in the next section. However, gradually a primary design goal became clear, and it is this:

Remind must be able to represent events with arbitrarily-complicated recurrence rules, so you only need to write the script *once* and Remind will *always* get the date of the events correct.

As an example, my garbage-collection day⁴ is every second Wednesday, but it is deferred by one day if the previous Monday or Tuesday or the Wednesday itself is a holiday. My city sends out calendars to show when collection day is. It also has a web site where you can look it up. And there’s even a mobile device app for this, because heaven knows the world has a severe lack of mobile device apps.

“This is nonsense!” I thought. I want a way to tell my computer when garbage-collection day is and then have it get it right *every single time*.

And the goal was achieved.

1.2 A Brief History of Remind

Back in my student days, in the Mesozoic era of computing, I had an MS-DOS PC and that sufficed for my computing needs. But in 1989, I had my first exposure to this strange new operating system called UNIX. I quickly grew to appreciate the power of UNIX: If the UNIX command line were the equivalent of a suave and articulate scholar, the MS-DOS command line would be like a grunting stone-age hominid flailing about trying to create this thing called “language”.

So I liked UNIX. And UNIX came with a command called **calendar**. This command would read a little text file and print a list of upcoming reminders. It was extremely simplistic, but I liked it. When my UNIX job ended, I was back in MS-DOS purgatory and I really missed **calendar**. So I decided to write a replacement, and in late 1989, Remind was born.

⁴What Canadians call “garbage-collection” might be called “trash-collection” if you’re American or “rubbish-collection” if you’re British.

At the time, I was a novice C programmer and had very little clue what I was doing. I also had no experience with calendars or date calculations. The [iCalendar](#) standard was almost a decade in the future, so I just hacked away at Remind until it did what I wanted.

In 1990, I changed cities and took a job that once again gave me access to UNIX. At the time, there was a thing called [Usenet](#) which was effectively a giant distributed bulletin board system, where anyone could post and read whatever they wanted. To bring some semblance of order to this chaos, Usenet was divided into topics called *newsgroups* and one newsgroup was called **comp.sources.misc**. People would post the source code of interesting programs they'd written, and anyone could compile and use the programs. So I put the source to Remind up on **comp.sources.misc** and a few people started using it.

Remind development really picked up in 1992. I had started working on my Master's Thesis, so of course I procrastinated by doing what I really liked... hacking on Remind. Apart from a break from 2005-2007, I've been working on Remind in my spare time ever since. And it has developed into a pretty... *full-featured*... program.

I hope you persist and don't fear the command-line. Wrangling Remind into doing a complicated date calculation is one of the most satisfying things you can do on a computer; I hope you agree with me and enjoy the little dopamine hit. I've enjoyed many dopamine hits from writing Remind.

1.3 A Quick Example

"OK!" I hear you say, "but show us an example of Remind working!"

If you haven't already installed Remind on your system, see Appendix [A on page 151](#) for instructions on how to do so. Go ahead... I'll wait while you do that.

OK, now that Remind is installed, let's create a little text file called **reminders.txt**. It should have the following content:

```
REM 1 January MSG New Year's Day
REM Wednesday MSG Piano lesson
REM 4 January MSG Bill's Birthday
```

Now, let's run Remind on that file. In this example, I am running Remind on Tuesday, 19th August 2025. The first line below is the shell prompt followed by what I typed in, and the rest is the output from Remind.

```
$ remind reminders.txt
No reminders.
```

Isn't that exciting?? But suppose instead I had run Remind on Wednesday, 1st January 2025. What do we get then?

```
$ remind reminders.txt
```

```
Reminders for Wednesday, 1st January, 2025 (today):
```

```
New Year's Day
```

```
Piano lesson
```

That's a bit more like it! (Though in reality, my piano teacher wouldn't have been teaching on New Year's Day. We'll see later on how to tell Remind about this fact.)

In this book, I will often write “run the following script...” or words to that effect. What I really mean is:

1. Create a text file containing the commands in the example.
2. Run `remind filename` from the shell prompt, possibly with some extra remind command-line arguments or options.

Conventionally, Remind scripts are either called `.reminders` or file names that end in `.rem`, but Remind doesn't care what you name your script files.

By the way... in the examples above, I showed a little dollar sign, `$`, on the command-line. Don't actually type that in. It just signifies that you need to type the bold part in as a shell command and then press Enter. In the few cases where you need to run a command as *root*, I will indicate that with a `#` shell prompt instead of the usual `$`

1.4 The Organization of This Book

This book has a *lot* of chapters. Chapters 2 through 7 are required reading. You should also read chapters 9, 12, and 14. The other chapters can be deferred until you need to use a feature they describe.

- Chapter 2 introduces you to the basics of writing a Remind script. It describes the overall syntax of a script and introduces you to basic types of reminders, such as one-off reminders and reminders with very simple recurrences.
- Chapter 3 explains how Remind handles holidays and exceptions that modify otherwise-regular recurrences.
- Chapter 4 talks about more advanced recurrences as well as how Remind can handle movable holidays (for example, holidays that are specified as something like “The first Monday in September”).
- Chapter 5 talks about reminders that also have a time-of-day associated with them (such as an appointment or a meeting) and how Remind can queue them and issue them at the right time.

- Chapter 6 talks about variables and expressions, important parts of Remind that are used to create very sophisticated recurring reminders.
- Chapter 7 talks about the SATISFY keyword which works together with expressions to express very complicated kinds of reminders. For example, suppose your credit card bill is due 21 days after the 15th of each month, unless that falls on a holiday or weekend, in which case the due date is delayed until after the holiday or weekend. You can express such a reminder with SATISFY.
- Chapter 8 describes a few more features that you can use for complex recurrences or scheduling, as well as so-called *callback functions* for modifying the appearance of a reminder.
- Chapter 9 describes how Remind scripts can be split up across several files that are included by a main file. It also describes the IF command that allows conditional flow control through a Remind script.
- Chapter 10 describes Remind's facilities for TODO-style reminders. These are different from normal reminders because Remind keeps reminding you about them until you explicitly mark them as complete.
- Chapter 11 describes Remind's features for handling astronomical events such as moon phases, sunrise and sunset, solstices and equinoxes. It also describes Remind's built-in support for calculating the date of Easter (both Western and Orthodox).
- Chapter 12 describes how to use Remind (and in some cases, some helper programs) to produce calendars in the terminal, as PDF output and as HTML output.
- Chapter 13 describes Remind's support for the Hebrew calendar.
- Chapter 14 describes TkRemind, a graphical front-end for Remind. This lets you make use of Remind without needing to learn the Remind scripting language (although I still recommend that you learn it).
- Chapter 15 describes facilities for adding ancillary information to a reminder, such as a location, a URL, or a long description.
- Chapter 16 describes Remind's facilities for expressing reminders in a specific time zone that might not be the same as your computer's default time zone.
- Chapter 17 describes Remind's facilities for running commands at specified dates and times.
- Chapter 18 describes tools to synchronize Remind calendars with other calendar services, such as those used on mobile devices.
- Chapter 19 describes how you can localize Remind (that is, translate it into a non-English language).
- Chapter 20 describes some features Remind has that let you debug problems with your Remind scripts.

Chapter 2

File Structure and Basic Reminders

This chapter will give you the basics of what a Remind script looks like and introduce you to writing basic reminders.

2.1 The Structure of a Remind Script

A Remind script is a plain text file consisting of a sequence of lines. There are three types of lines in a Remind script:

1. *Commands* - These are lines that actually do something.
2. *Comments* - These are notes for yourself that are ignored by Remind.
3. *Blank Lines* - These lines are ignored.

Remind reads a script from beginning to end. However, if Remind encounters a line in the script containing exactly the following:

__EOF__

(that is, two underscores followed by EOF and two more underscores and then a newline) then it stops reading the script at that point and ignores everything after the **__EOF__** marker.

2.1.1 Commands

You've already seen one command so far: The REM command. This command, in fact, is the most powerful and complicated command in Remind's repertoire.

Remind has many other commands besides REM and you'll be introduced to them in due course. This chapter will be devoted almost entirely to the REM command.

2.1.2 Comments

Comments are little notes you can leave for yourself in the reminder file. Remind ignores them. Any line that begins with¹ “#” or “;” is a comment and is ignored by Remind. Here are examples of comments:

```
# This line starts with a #. Remind will ignore it.  
; This one starts with a semicolon. It's also ignored.
```

You might wonder why a calendar program needs comments. Trust me, when you do something very tricky and are immensely pleased with yourself, you owe an explanation to some poor sap who reads the Remind script and has to understand your trickiness. Especially when the poor sap is likely to be you in six months’ time.

2.1.3 Blank Lines

Blank lines are ignored. I tried to think of something witty to write here, but I drew a blank.

2.1.4 Continued Lines

It may happen that you have a very long line in your Remind script—one that is uncomfortably long. You can continue a logical line across multiple physical lines in your Reminder file by ending all but the last physical line with a backslash. Here is an example:

```
REM 10 December MSG OK, this is a pretty long line. \  
It might be best to split it across multiple lines. \  
Looks like three physical lines suffice.
```

Make sure there’s no whitespace after the backslash.

When Remind processes continued lines, it removes the trailing backslash but *not* the newline, and smooshes the result together. So it’s best to continue a line in a place where a space naturally appears and not in the middle of a word. Otherwise that embedded newline will cause chaos. Note, however, that in the *body* of a reminder (the body is the part that comes after MSG), Remind will *remove* any embedded newlines. That is why in the previous example, I left a space before the backslash. Otherwise, the sentences would have been squashed together with no spaces after the periods. This removal of the newline only happens in the body of a reminder and not anywhere else.

Remind does not impose any specific limit on the length of a logical (or physical) line. If you want a reminder on March 14th (π Day) consisting of the first ten million digits of π , have at it. Make your line as long as you want.

Of course, there is an ultimate limit, that being the amount of memory in your computer. So perhaps it’s best to avoid being reminded of the first trillion digits of π , unless you have a very large computer indeed.

¹Actually, any line whose first non-whitespace character is “#” or “;”

2.1.5 Character Set and Encoding

Remind is not particularly fussy about the character set or encoding you use in your Remind script, except that it must be a superset of ASCII. All Remind commands are plain ASCII, but you can use whatever you like in the body of a reminder.

However, almost all modern UNIX and Linux systems use the Unicode character set and the UTF-8 encoding. Many of the Remind scripts that ship with Remind assume a UTF-8 encoding, so in practice you need to use the Unicode character set with UTF-8 encoding. If you're lucky enough to speak a language like English that doesn't need funny characters like ô or Ł or ø, then you can get away with ASCII.

2.1.6 Case Sensitivity

Remind is case-insensitive almost everywhere. I'll point out the few places that are case-sensitive when we get there, but in your scripts, Remind doesn't care if you spell it REM or Rem or rem. Similarly, you can use JUNE or June or june... whatever strikes your fancy.

2.2 Running Remind

Remind has four major modes of operation, although you will almost always use one of the first two modes listed below:

1. **Agenda Mode** is the default mode of Remind. In Agenda Mode, Remind runs your script once and prints any reminders that happen to be triggered today.
2. **Calendar Mode** is used to create calendars. In Calendar Mode, Remind runs your script multiple times (typically once for each day of a month) and creates a calendar with reminders filled into the appropriate calendar boxes.

This will create a nice ASCII (or UTF-8) calendar in your terminal. We'll see in Chapter 12 how you can use different options, along with some helper programs, to create PDF, PostScript and HTML calendars.

3. **Purge Mode** is used to remove reminders from your script if they have expired. Purge Mode will not be covered in this book, because it's not essential to use it and it is very well documented in the **remind(1)** man page.
4. **Server Mode** is a special mode used by the TkRemind graphical front-end for Remind. Again, I won't cover it in this book because it is very well documented in the **tkremind(1)** man page. There is a slight variant called **Daemon Mode** that is covered in Chapter 17.

2.2.1 Some Remind Command-Line Options

Remind has *many* command-line options; I won't document them all in this book. Instead, see the **remind(1)** man page. The following list shows some ways to run Remind. They all assume that *pathname.rem* contains the following, and that Remind is being run on January 2nd, 2026.

```
# Example reminder script for command-line invocations that follow
REM Wednesday MSG Swimming
REM 15 January 2026 MSG Meeting
REM 20 January MSG Jane's Birthday
```

And here are some ways to run Remind:

- **\$ remind *pathname.rem*** – this is the normal way to run Remind in Agenda Mode.
- **\$ remind -c *pathname.rem*** – this runs Remind in Calendar Mode and produces a nice ASCII calendar in your terminal. The calendar will look something like this:

January 2026						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
				1	2 *****	3
4	5	6	7 Swimming	8	9	10
11	12	13	14 Swimming	15 Meeting	16	17
18	19	20 Jane's Birthday	21 Swimming	22	23	24
25	26	27	28 Swimming	29	30	31

If you are running a terminal capable of displaying UTF-8 characters, you can use this variant instead:

\$ remind -cu *pathname.rem* which produces solid lines instead of fake lines composed of +, - and | characters. The output will look something like this:

January 2026						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
				1	2 *****	3
4	5	6	7 Swimming	8	9	10
11	12	13	14 Swimming	15 Meeting	16	17
18	19	20 Jane's Birthday	21 Swimming	22	23	24
25	26	27	28 Swimming	29	30	31

- **\$ remind -s *pathname.rem*** – runs Remind in Calendar Mode, but produces “simple output” suitable for reading by both a human and a machine. It can be a quick way to check the output of your reminders in Calendar Mode. The output will look something like this:²

```
2026/01/07 * * * * Swimming
2026/01/14 * * * * Swimming
2026/01/15 * * * * Meeting
2026/01/20 * * * * Jane's Birthday
2026/01/21 * * * * Swimming
2026/01/28 * * * * Swimming
```

(The asterisks have to do with an interchange format for sending calendar data to other programs; you can ignore them for now. The details are described in the **rem2ps(1)** man page.)

- **\$ remind -ppp *pathname.rem*** – runs Remind in Calendar Mode, but produces JSON output designed for machine consumption. Remind ships with various helper programs that ingest this JSON and produce PDF and HTML calendars. Because the JSON output is not meant for human consumption, I won’t show it here. But you can easily run Remind to generate it yourself if you want to see what it looks like.
- **\$ remind -n *pathname.rem*** – runs Remind in a variant of Agenda Mode called *Next Mode*. In this mode, the *next* trigger date of each REM command is printed in a format similar to the Simple Calendar format. The output for our example will look like this:

```
2026/01/07 Swimming
2026/01/15 Meeting
2026/01/20 Jane's Birthday
```

These are the *next* dates that each REM command will trigger as calculated on January 2nd, 2026.

If you supply the single character - instead of a pathname, Remind reads commands from standard input. This lets you type commands interactively. If Remind was compiled against the readline library, then in interactive mode it will have bash-style command-line editing.

2.3 The REM Command: An Introduction

You’ve already seen examples of the REM command. In its simplest form, a REM command looks like this:

```
REM trigger-spec MSG body
```

²Yes, I know the dates are printed using the format YYYY/MM/DD instead of YYYY-MM-DD. This is an accident of history and hasn’t been changed so as not to break backward compatibility,

A *trigger-spec* tells Remind when to *trigger* the reminder. A basic trigger-spec contains the following parts:

- A *year*. This is a number from 1990 to 5990. A trigger-spec may contain zero or one *year* parts. Remind cannot handle years before 1990; that is because internally, it represents dates as a count of days since 1 January 1990. The date 1990-01-01 is Remind's *beginning of time*.
- A *month*. This is an English month name such as February or June, or an abbreviation of three letters or more such as Apr or Sept. A trigger-spec may contain zero or one *month* parts.
- A *day*. This is a number from 1 to 31 that represents a day number within a month. A trigger-spec may contain zero or one *day* parts.
- A *weekday list*. This is a space-separated list of English weekday names like Sunday or abbreviations of at least three letters like Fri or Thur. A trigger-spec may contain zero or one *weekday lists*.

A trigger-spec may be *satisfied* on one or more dates. Any time a date matches whichever components of a trigger-spec are present, then the date satisfies the trigger-spec. If a trigger-spec is satisfied on a particular date, then the reminder will be *triggered* on that date. The closest such date on or after today is called the *trigger date*.

Here are some examples of REM commands:

```
REM MSG This will trigger every day.
REM 1 MSG This will trigger on the 1st of every month.
REM Apr MSG This will trigger every day in April, in any year.
REM 2025 MSG This will trigger every day in 2025.
REM 3 Sep MSG This will trigger on September 3rd, every year.
REM 15 2025 MSG This will trigger on the 15th of every month in 2025.
REM July 2026 MSG This will trigger every day in July 2026.
REM 10 October 2027 MSG This will trigger only on October 10th, 2027.
```

The eight commands above do not include weekday lists. The behavior of a weekday list depends on whether or not the trigger-spec has a *day* component. Here are trigger-specs without a day component:

```
REM Sat Sun MSG Triggered every Saturday and Sunday.
REM Wed June MSG Triggered every Wednesday in June, any year.
REM Thu 2026 MSG Triggered every Thursday in 2026.
REM Tue Thu Apr 2026 MSG Every Tuesday and Thursday in April 2026.
```

If the trigger-spec has *both* a day *and* a weekday component, then the behavior changes: The reminder is triggered on the first day *on or after the day* that is *also* found in the weekday list. It's probably easiest to look at examples to understand:


```
REM 1 Sat MSG First Saturday of every month.
REM 8 June Sat Sun MSG The first day after 8 June that is a Sat or Sun.
REM Mon 8 2026 MSG The second Monday of every month in 2026.
REM Thu Fri 20 Aug 2025 MSG Only August 21st, 2025.
```

Let's explain these last four:

- In the first reminder, the trigger date will be the first Saturday that occurs on or after the 1st of every month.
- In the second reminder, the trigger date will be the first day on or after 8 June that is a Saturday or a Sunday. In 2025, June 8th is a Monday, so the trigger date will be the following Saturday, June 13th 2025. Note that the reminder will *not* trigger on Sunday, June 14th 2025. As soon as *any* weekday constraint is satisfied, then that is the *only* one that is used. In 2031, June 8th is a Sunday, so in that year, the trigger date will also be June 8th, satisfying the Sun weekday constraint.
- In the third reminder, the trigger date will be the first Monday on or after the 8th of every month in 2026. This happens to be the second Monday of every month in 2026; convince yourself of that!
- In the fourth reminder, 20 Aug 2025 is a Wednesday. The first day on or after that date that is in the weekday list is Thursday, 21 Aug 2025 and that is the only trigger date.

Generally speaking, Remind is pretty relaxed about the order of trigger components. The REM commands within each of the following groups are equivalent to one another. Additionally, if all three day, month, and year components are present, you can simply write YYYY-MM-DD:

```
REM January 1 MSG New Year's Day
REM 1 January MSG New Year's Day
```

```
REM 25 March 2025 MSG Concert
REM 2025 March 25 MSG Concert
REM March 25 2025 MSG Concert
REM 2025-03-25 MSG Concert
```

```
REM Mon 5 Feb MSG Appointment
REM 5 Mon Feb MSG Appointment
REM Feb 5 Mon MSG Appointment
```

2.4 The Remind Trigger-Calculation Algorithm

This is the most important section in this book. Make sure you understand the Remind trigger-calculation algorithm. If you do, you will have much success writing Remind scripts. If you do not, you will be eternally mystified.

When Remind executes a REM command, here's what it does internally.

1. Set a variable X to today's date.
2. Look at the trigger-spec and determine whether or not X satisfies the trigger-spec.
3. If X does indeed satisfy the trigger-spec, then X is returned as the trigger date.
4. Otherwise, add one day to X and go back to step 2.

Of course, Remind does not *actually* work that way. Consider the following REM command:

```
REM 14 September 2003 MSG Wow.
```

If we ran Remind on 20 Aug 2025, the algorithm above would never end because Remind would keep checking subsequent days in a fruitless attempt to find something that satisfies the trigger-spec.

So the actual algorithm is modified in two ways: (1) It terminates if it can convince itself that *no* future date will ever satisfy the trigger spec, and (2) it incorporates many optimizations that eliminate the need to actually check every single day starting from today and stretching out into the future. However:

Remind *behaves* as if it uses the algorithm described previously.

2.5 Advance Warning

Let's say you've set up a bunch of reminders to remind you about friends' birthdays. For example, suppose you have this:

```
REM 16 July MSG Jane's birthday.
```

You dutifully run Remind every day when you log in to your computer, and on 15 July, the output of Remind is:

```
No reminders.
```

But then on 16 July, you're confronted with:

```
Jane's birthday.
```

Oh no! You would have liked to have had some heads-up to go and buy Jane a card. As it is, you have to scramble to get the card in time. It sure would have been nice if Remind had a way to give you a little bit of advance notice for an upcoming event...

Well, there is a way. You can add a *delta* to the trigger spec. A delta is a number prefixed by two plus signs³—for example, ++10.

This causes Remind to trigger a reminder *not only* on the actual computed trigger date, *but also* on the delta days prior to the actual trigger date. So if we modified the example to look like this:

```
REM 16 July ++10 MSG Jane's birthday.
```

Remind would start warning you of Jane's birthday on 6 July, and would issue the reminder every day through 16 July.

But there's a problem. If you run Remind on, say, 8 July, it will print:

```
Jane's birthday.
```

But Jane's birthday isn't on 8 July. It's on 16 July. How can we get Remind to give us more information about exactly when a reminder's trigger date is?

The answer is the *substitution filter*.

2.6 The Substitution Filter

Before Remind prints the body of a reminder, it runs it through a text filter called the *substitution filter*. This filter replaces certain sequences of characters that begin with a percent sign % with different information. The substitution filter is covered in gory detail in the **remind(1)** man page, so I'll just mention some of the more popular sequences.

Let's change our REM command to look like this:

```
REM 16 July ++10 MSG Jane's birthday is %b.
```

If we run Remind on 6 July, we get this:

```
Jane's birthday is in 10 days' time.
```

If we run it on 15 July, we get this:

```
Jane's birthday is tomorrow.
```

and if we run it on 16 July:

```
Jane's birthday is today.
```

³There's another form of delta that has *one* plus sign that will be explained in Section [3.4 on page 24](#).

2.6.1 Some Substitution Sequences

As I wrote earlier, the substitution filter recognizes many sequences. I'll list some of the most popular here. For the exhaustive list, see the **remind(1)** man page.

%a Replaced with “on *weekday, day month, year*”. For example, a reminder whose trigger date is 2025-09-14 would yield “on Sunday, 14 September, 2025”

%b Replaced with “in *n* days’ time”, “tomorrow” or “today”, depending on the relationship between the trigger date and today’s date.

%c Replaced with “on *weekday*”.

%_ Replaced with a newline. Normally, Remind *removes* newlines in the body of the reminder (this is the fate of the newlines embedded in continued lines.) **%_** lets you explicitly put a newline in a reminder.

%% Replaced with a literal percent sign, **%**.

2.6.2 Limiting the Text in Calendar Mode

The special substitution sequence **%"** is used to limit what appears in Calendar Mode. For example, suppose you have the following reminder:

```
REM 20 January +8 MSG Bob's birthday is %b
```

In Agenda Mode on 20 January, that will result in: Bob's birthday is today. However, in Calendar Mode, the text “is today” also uselessly appears in the calendar box, when all you really need is “Bob’s birthday”.

You can fix it as follows:

```
REM 20 January +8 MSG %"Bob's birthday%" is %b
```

In Agenda Mode, the **%"** sequences are *completely ignored*. In Calendar Mode, however, if they appear, then *only* the text between the two **%"** sequences is put into the calendar.

2.7 The Banner

You may have noticed that in Agenda Mode, if there are reminders, Remind prints a banner similar to this:

```
Reminders for Monday, 2nd February, 2026 (today):
```

The banner may be changed with the `BANNER` command, which looks like this:

```
BANNER header-text
```

The *header-text* can be anything you want; before issuing the banner, Remind passes *header-text* through the substitution filter, with the effective date as the trigger date. The default `BANNER` text is equivalent to:

```
BANNER Reminders for %w, %d%s %m, %y%O:
```

Consult the **remind(1)** man page for information about these substitution sequences. If you don't want any banner at all, you can put this at the top of your reminder script:

```
BANNER %
```

2.8 Counting Backwards

So far, we've seen how to write reminders for the first, second, third and fourth Monday (for example) of a month. But how do we write a reminder for the *last* Monday of a month? Or the last day? Months have different lengths; some have four Mondays and some have five.

The solution is to count backwards using a *back*, which is a number preceded by two minus signs,⁴ like this: `--n`

If you want a reminder on the last day of every month, for example, write it like this:

```
REM 1 --1 MSG Last day of the month
```

The way Remind calculates the trigger date is as follows: It calculates what the trigger date would be without the back, and then it subtracts the number of days indicated by the back. Since a reminder of `REM 1` is simply the first day of every month, subtracting 1 gives the last day of the previous month. This reminder therefore triggers on the last day of every month.

How about if you want the last Friday of every month? That's simply the first Friday of the following month and then go back by 7 days, like this:

```
REM Friday 1 --7 MSG Last Friday of the month
```

Similarly, this gives you the last Tuesday in February:

```
REM Tue 1 March --7 MSG Last Tuesday of February
```

Note that in the previous example, even though we want the last Tuesday of February, we specify the month as March because we are going backwards from there.

⁴As you may have guessed, there's another form of *back* that uses only one minus sign and that will be explained in Section [3.4 on page 24](#).

2.9 Some Syntactic Sugar

Remind has syntactic sugar for some common types of reminders. For example, although you can write the following reminders (convince yourself that they’re correct):

```
REM Mon 1 MSG First Monday of every month
REM Tue 8 May MSG Second Tuesday in May
REM Fri 15 MSG Third Friday of every month
REM Sat 22 June 2026 MSG Fourth Saturday in June 2026
```

Remind lets you write them as follows:

```
REM First Monday MSG First Monday of every month
REM Second Tuesday in May MSG Second Tuesday in May
REM Third Friday MSG Third Friday of every month
REM Fourth Saturday in June 2026 MSG Fourth Saturday in June 2026
```

The word “in” is ignored, but allowed in order to make the reminders more readable.

You might ask why there’s no syntactic sugar for:

```
REM Fifth Thursday MSG Fifth Thursday of every month
```

The reason is that while there are four of every weekday in every month, not all weekdays appear five times in a month and so more complicated mechanisms are needed for the “fifth Thursday” reminder, which we will discover in [Section 7.4.1 on page 61](#).

Another piece of syntactic sugar simplifies writing reminders for “the last *something* of a month”. The reminders within each of the following groups are equivalent to others within the group. Note that the character ~ in the second reminder of each pair is a tilde, not a minus-sign.

```
REM 1 --1 MSG The last day of every month
REM ~~1 MSG The last day of every month
```

```
REM Fri 1 --7 MSG The last Friday of every month
REM Fri ~~7 MSG The last Friday of every month
```

```
REM 1 Mar --1 MSG The last day of February
REM Feb ~~1 MSG The last day of February
```

```
REM Fri 1 Jan 2027 --7 MSG The last Friday in 2026
REM Fri Dec 2026 ~~7 MSG The last Friday in 2026
```

Note that in the last examples, the ~~1 automatically corrects the month and year for the one you *want* rather than the one you’re moving back from.

You cannot use a *day* component together with a *~~n* component because the latter has an implied day component. In fact, *~~n* is *exactly* the same as *1 - -n* except for the potential adjustment of month and year mentioned previously.

One last piece of syntactic sugar is based on the previous syntactic sugar⁵. The following sets of reminders are equivalent:

```
REM ~~1 MSG The last day of every month
REM Lastday MSG The last day of every month

REM June ~~1 MSG The last day of every June
REM June Lastday MSG The last day of every June

REM Monday May ~~7 MSG The last Monday in May
REM Last Monday May MSG The last Monday in May
REM Last Monday in May MSG The last Monday in May
```

2.10 Checking What Will Happen on a Specific Date

Normally, when you run a Remind script from the command line like this:

```
$ remind filename
```

then today's date is taken from your computer's internal clock and trigger dates are calculated on the basis that today is... well... *today*. But what if you want to see how Remind will behave on some other date? You can simply supply a date on the command line after the *filename*, in one of two ways. Consider these examples:

```
$ remind filename 14 June 2033
$ remind filename 2033-06-14
```

Both of those commands run the Remind script *filename*, but they pretend that today is 14 June, 2033. This lets you see how the script will behave on the given date.

2.11 The “Current” Date and Time

Remind's idea of the current date and time are not as simple as the clock time as reported by the operating system. Remind's ideas of the current date and time are called the *effective date* and *effective time*, and here's how they are defined.

⁵So perhaps it is syntactic icing sugar?

- In Agenda Mode:
 - If you run Remind without supplying a date or time on the command line, then the effective date and time are the clock date and time returned by the operating system, in the local time zone. (That is, exactly what you see when you type the command date).
 - If you supply a date, but not a time, on the command line, then the effective date is set to the date you supply on the command line, and the effective time is set to the clock time as returned by the operating system.
 - If you supply a date and a time on the command line, then the effective date is set to the date you supply on the command line, and the effective time is set to the time you supply on the command-line.
- In Calendar Mode, the effective date is set to the date of each calendar box as Remind runs through the days of the calendar, and the effective time is set to midnight (00:00).

I will also sometimes refer to *system date* and *system time*. These always refer to the clock date and time returned by the operating system, in the local time zone. And if I refer to *today* or *now*, those refer to the *effective date* and *effective time*, respectively.

2.12 Running Remind Multiple Times in a Row

You can run Remind multiple times in a row by adding a command-line parameter of the form **n*, where *n* is a positive integer, to the end of the command-line. You should protect it from shell expansion by enclosing it in single quotes. This causes Remind to run *n* times in Agenda Mode, with the effective date incremented by one day for each subsequent run. Here is an example:

```
# Run Remind 5 times, starting on 2026-04-05 and running through 2026-04-09
$ remind file.rem 2026-04-05 '*5'
```

You don't need to supply a date on the command line; if you don't, then the **n* repetitions start from the actual system date.

Chapter 3

Holidays and Other Exceptions

Quite often, the regular recurrence of an event is disrupted by holidays or other exceptional days. When this happens, the normal recurrence must be suspended or modified.

Remind handles this with the concept of *Omitted Dates*. It keeps a global list of such dates and this global list is called the *omit context*. When Remind starts up, the omit context is empty. To add dates to the omit context, use the OMIT command. This command can take one of three forms:

- OMIT *weekday* ...
- OMIT *day month*
- OMIT *day month year*

The first form is called a *weekday omit*. The second is called a *partial omit* and the third is called a *full omit*. Here are examples of each:

```
# Weekends are normally holidays
OMIT Sat Sun
```

```
# New Year's Day is a holiday
OMIT 1 January
```

```
# This is a special one-time holiday
OMIT 6 January 2027
```

Any of the forms can also include a MSG clause, in which case it is *also* treated as a REM command. The single OMIT command on the first line in the following example is equivalent to the two separate commands that follow it:

```
OMIT 1 January MSG New Year's Day
```

```
OMIT 1 January  
REM 1 January MSG New Year's Day
```

Each OMIT command adds an omitted date to the omit context.

Let's see how the omit context works in practice. 1 January 2026 is a Thursday, so what happens on 1 January 2026 when we run this script?

```
OMIT 1 January  
REM Thursday MSG Meeting
```

If you run that script on 1 January 2026, the result is:

```
Reminders for Thursday, 1st January, 2026:
```

```
Meeting
```

Wait... what? 1 January is omitted! What's going on?

By default, Remind *ignores* the omit context when calculating the trigger date of the reminder. If you don't want that, the REM command has three keywords that give you three different options:

1. SKIP simply skips the reminder. In this example, the meeting on January 1st won't happen, and the trigger date will be the following Thursday, January 8th—the date of the next normal recurrence.
2. BEFORE moves the reminder to *before* any block of omitted dates. In this example, BEFORE will move the trigger date to December 31st, 2025.
3. AFTER moves the reminder to *after* any block of omitted dates. In this example, AFTER will move the trigger date to January 2nd, 2026.

Here are examples, with comments explaining what will happen if the script is run on January 1st, 2026:

```
OMIT 1 January
```

```
# Triggered January 1st, 2026 as usual
REM Thursday MSG Meeting
```

```
# Not triggered January 1st, but is triggered January 8th
REM Thursday SKIP MSG Meeting
```

```
# Triggered December 31st, 2025
# assuming Remind is run on that date...
REM Thursday BEFORE MSG Meeting
```

```
# Triggered January 2nd, 2026
REM Thursday AFTER MSG Meeting
```

A date of the form *day month year* is said to be *omitted* if one or more of the following is true:

1. The weekday of the day is an omitted weekday.
2. The *day* and *month* match a partial omit.
3. The entire date matches a full omit.

3.1 Local OMITs

It is very common to have a number of reminders that are affected if they fall on weekends (for example, paydays) while others are not. Because this is so common, rather than globally omitting weekends, you can have a *local OMIT* clause right within a REM command. Here are some examples:

```
REM 15 AFTER MSG 15th of every month; delayed after holidays
```

```
REM 15 OMIT Sat Sun AFTER MSG 15th of every month; \
                                delayed after holidays/weekend
```

In the second REM command, Remind considers Saturday and Sunday to be omitted even if there are no global weekday Saturday or Sunday OMIT entries.

A local OMIT within a REM command can only be followed by a list of weekday names. You can't locally omit specific dates or day-month pairs.

3.2 Saving and Restoring the Omit Context

Sometimes, certain omitted days such as holidays apply to one block of reminders but not to others. Remind has three commands for manipulating the omit context:

- `PUSH-OMIT-CONTEXT` takes a snapshot of the current omit context (the weekday, partial and full omits) and saves it on a stack.
- `CLEAR-OMIT-CONTEXT` clears out the omit context, starting you off with a blank slate.
- `POP-OMIT-CONTEXT` restores the omit context from the most recent matching `PUSH-OMIT-CONTEXT`. Any changes to the omit context after the most recent `PUSH-OMIT-CONTEXT` are completely reverted by `POP-OMIT-CONTEXT`

For example, suppose August 5th is a holiday for the purposes of your garbage collection date, but not for the purposes of a work meeting. You could do something like this:

```
REM Wednesday SKIP MSG Work meeting

PUSH-OMIT-CONTEXT
OMIT 5 August
REM Wednesday AFTER MSG Garbage pickup
POP-OMIT-CONTEXT

# At this point, 5 August is no longer in the omit context
```

3.3 Dumping the OMIT Context

For debugging purposes, you can dump the current OMIT context as follows:

```
OMIT DUMP
```

3.4 Alternate Forms of Back and Delta

Back in Chapter 2 we mentioned that the delta `++n` and back `--n` have alternate forms with single plus or minus signs. Here's how they differ:

- A delta of `++n` starts triggering the reminder exactly n days before the trigger date. However, a delta of `+n` starts triggering the reminder n days before the trigger date *not counting omitted dates*.

- A back of `--n` moves the trigger date back by exactly n days. However, a back of `-n` moves the trigger date back by n days *not counting omitted dates*.
- An alternate back of `~n` triggers on the n th last day of the month, *not counting omitted dates*.

Here are some examples illustrating the differences:

```
REM 1 --1 OMIT Sat Sun MSG The last day of every month
REM 1 -1 OMIT Sat Sun MSG The last working day of every month

OMIT 25 Dec MSG Christmas
REM 26 Dec ++2 MSG Boxing Day %b (start triggering on 24 Dec)
REM 26 Dec +2 MSG Boxing Day %b (start triggering on 23 Dec)

REM November 2025 ~~1 MSG The last day of Nov 2025 (Nov 30)
REM November 2025 ~1 OMIT Sat Sun MSG The last weekday of Nov 2025 (Nov 28)
```

And one last sweet bit of syntactic sugar: `LASTWORKDAY` is equivalent to `~1` just as `LASTDAY` is equivalent to `~~1`.

Chapter 4

More Advanced Reminders

So far, we've covered the basic uses of the REM command and covered how holidays are handled with OMIT. In this chapter, we'll look at some of the more advanced ways to use REM.

4.1 Arbitrary Recurrences

It's very easy to write a reminder that recurs weekly, monthly or annually in Remind. But some events recur on a different schedule. For example, my garbage and plastic recycling collection day is every second Wednesday.¹ And on the other Wednesdays in between, paper recycling is collected.

You can express arbitrary recurrences with a start date and a *repeat*. A repeat is simply an asterisk followed by an integer *n*. The reminder is triggered on the start date and then every *n* dates thereafter. Here are my garbage and recycling days:

```
REM 2012-11-07 *14 MSG Garbage and plastic recycling
REM 31 October 2012 *14 MSG Paper recycling
```

Let's examine these examples. The first example triggers on 7 November 2012 and then every 14 days thereafter, in perpetuity. The second one triggers on 31 October 2012 and then every 14 days thereafter.

Note that you can write the starting date as either YYYY-MM-DD or spell out the day, month and year separately.

If you include a weekday, then it is used *only* to determine the start date and does *not* adjust the recurrence. For example, consider the following reminder:

```
REM Fri 15 Sep 2025 *10 MSG Test
```

The first Friday on or after 15 Sep 2025 is 19 Sep 2025. The reminder will trigger on 19 Sep 2025 and then every 10 days, *regardless* of whether it is a Friday or not.

¹Actually, it's a lot more complicated than that, but for now let's just pretend it's every second Wednesday.

4.2 Specific Expiry Dates

Quite often, you will have a recurring reminder that starts on a specific date and ends on another date rather than repeating indefinitely. You can use the `UNTIL` keyword to specify an expiry date for a reminder that normally wouldn't have one.

For example, suppose you have a class every Tuesday, starting on 23 September 2025 with the final class on 9 December 2025. You can express this as follows:

```
REM 2025-09-23 *7 UNTIL 2025-12-09 MSG Class
```

This example triggers every 7 days, starting on 2025-09-23 and finishing on (and including) 2025-12-09. After the `UNTIL` keyword, Remind expects a fully-specified date like 2025-12-09 or 9 December 2025.

4.3 Specific Starting Dates

Sometimes, you will have a series of reminders that start on a specific date and should not be triggered before that date. Suppose that you have a meeting on the 15th of every month, starting on 2025-06-15 with the last meeting taking place on 2025-11-15. You can express this as follows:

```
REM 15 FROM 2025-06-15 UNTIL 2025-11-15 MSG meeting
```

The `FROM 2025-06-15` means the reminder will never be triggered prior to that date. Note that we *cannot* use a repeat (`*n`) form here because the gap between the 15th of two subsequent months is not constant. If we do have a constant repetition, we have options on how to write the reminder. For example, the following are equivalent:

```
REM 2025-09-23 *7 UNTIL 2025-12-09 MSG Class
REM Tuesday FROM 2025-09-23 UNTIL 2025-12-09 MSG Class
```

Feel free to use whichever you find easier to read and understand.

Just like `UNTIL`, `FROM` expects a fully-specified date like 2025-12-09 or 9 December 2025.

4.4 Movable Holidays

We've seen in Chapter 3 how the `OMIT` command lets you mark certain dates as "omitted", and how Remind can adjust reminders that fall on omitted dates.

But some holidays are movable. For example, Labour Day in Canada is the first Monday in September, and the `OMIT` command has no syntax to express this.

Instead, we can use a normal REM command with the ADDOMIT keyword.²

Here is a naive (but *incorrect*) way to use ADDOMIT; I'll explain why it's wrong and how to fix it in the next section:

```
REM Mon 1 Sep ADDOMIT MSG Labour Day
```

The ADDOMIT keyword tells Remind that if it computes a trigger date for the reminder, then it must add that computed trigger date to the global OMIT context. For example, suppose that today is 15 August 2025 and we run Remind on the following input file:

```
REM Mon 1 Sep ADDOMIT MSG Labour Day
OMIT DUMP
```

Then the output³ is:

```
Global Full OMITs (1 of maximum allowed 1000):
    2025-09-01
Global Partial OMITs (0 of maximum allowed 366):
    None.
Global Weekday OMITs:
    None.
No reminders.
```

As you can see, the computed date of Labour Day (2025-09-01) is added to the global OMIT context.

BUT THIS IS A TRAP. Because of how the Remind algorithm works, specifying movable holidays in this way is incorrect. Let's look at this example file:

```
REM Mon 1 Sep ADDOMIT MSG Labour Day
REM 1 AFTER MSG Meeting
```

If we run this reminder file on 2025-09-01, the output is as follows:

```
Reminders for Monday, 1st September, 2025:
```

```
Labour Day
```

²Earlier versions of Remind used a different mechanism to handle movable holidays, but you should not use them any more. Stick with ADDOMIT.

³Versions of Remind starting from 06.01.03 will issue a warning about the ADDOMIT line, so that's a bit of a spoiler that something is probably wrong.

We get the Labour Day reminder. And the meeting reminder does *not* trigger because the AFTER keyword tells Remind to move it until after any holidays. The computed trigger date is thus 2025-09-02.

So let's run this reminder file on 2025-09-02. What will the output be? The answer is:

```
No reminders.
```

What?? Why wasn't the delayed meeting triggered?

The answer is that on 2025-09-02, the trigger date for the Labour Day reminder, as calculated by the Remind algorithm, is 2026-09-07, the first Monday in September *that is on or after today*. So the date 2026-09-07 is entered into the global OMIT context, and the meeting reminder has no idea that 2025-09-01 is in fact supposed to be a holiday.

So how do we fix this? How do we make movable holidays safe?

4.4.1 Safe Movable Holidays

If we think back to the trigger-calculation algorithm in Section 2.4 on page 14, we can see that the trouble occurs because Remind *always* seeks a date *on or after today* that satisfies the trigger. What if we could temporarily pretend that *today* was a bit earlier? Well, we can! And this is the key to fixing the problem.

The SCANFROM clause of the REM command lets us *pretend* that *today* is actually a few days earlier just for the purposes of calculating a trigger date. A SCANFROM clause looks like this: SCANFROM *-n* where *n* is an integer. This is called a *relative SCANFROM*⁴ Let's look at a concrete example:

```
REM Mon 1 Sep SCANFROM -28 ADDOMIT MSG Labour Day
REM 1 AFTER MSG Meeting
```

We get very different results when we run this file. On 2025-09-01, we get:

```
Reminders for Monday, 1st September, 2025:
```

```
Labour Day
```

and on 2025-09-02, we get:

```
Reminders for Tuesday, 2nd September, 2025:
```

```
Meeting
```

Huzzah! Success!

⁴There is also an absolute SCANFROM, but it is almost never used. It was used in older versions of Remind that lacked a relative SCANFROM, but is now considered obsolete.

If we look at the REM command with ADDOMIT:

```
REM Mon 1 Sep SCANFROM -28 ADDOMIT MSG Labour Day
```

The trigger date on 2025-09-01 is 2025-09-01 (because that happens to be a Monday.) On 2025-09-02, the trigger date is *still* 2025-09-01 rather than the expected 2026-09-07 because SCANFROM -28 means that the Remind algorithm starts scanning from 2025-09-02 minus 28 days, or 2025-08-25. In fact, the trigger date *remains* 2025-09-01 until Remind is run on 2025-09-30, 29 days after 2025-09-01. That is because in each case, the Remind algorithm considers “today” to be the *actual* run date *minus* 28 days.

SCANFROM thus ensures that the addition to the omit context stays stable for any reminders within 28 days of the omitted date. Since typically, there are no long blocks of omitted days, it is very likely that any reminder further than 28 days from the omitted day won’t be affected by it.

For stable movable omitted dates, observe the following rules of thumb:

- For *annually* recurring holidays, use SCANFROM -28
- For *monthly* recurring holidays or exceptions, use SCANFROM -7

In both cases, the SCANFROM amount should be sufficiently large to make sure that omitted days have the correct effect on subsequent REM commands.

Chapter 5

Timed Reminders

So far, all of the reminders we've seen have had granularity of one day. In other words, they tell us what *days* events happen, but not what *time*. It's pretty useless to know that you have a staff meeting every Tuesday if you don't also know what time the meeting is.

Remind (naturally) allows you to specify the time of events. You do this with the AT clause.

5.1 The AT Clause

In its simplest form, the AT clause is simply the keyword AT followed by a time of the form *HH:MM* where *HH:MM* is a time-of-day in 24-hour clock format. Note that Remind *always* interprets dates and times in the time zone that your computer is set to—unless you tell it otherwise, as we'll see in a later chapter.

For example, if you have a staff meeting at 3:00PM every Tuesday, you could write a reminder for it like this:

```
REM Tuesday AT 15:00 MSG Staff Meeting
```

If you do not like to use the 24-hour clock format, Remind also lets you write AM/PM style times, so this would work too:

```
REM Tuesday AT 3:00PM MSG Staff Meeting
```

You can also use `.` instead of `:` to separate the hour and minute parts of a time, so `15:00` can also be written `15.00`.

5.1.1 Queuing Reminders

Remind handles timed reminders differently from non-timed ones in the following ways:

- In Calendar Mode, Remind prefixes the reminder with its start time.
- In Agenda Mode, if a timed reminder is scheduled for a time *later* in the day than the effective time (Section 2.11 on page 19), then after Remind prints the day’s agenda, it runs in the background. At the scheduled time, it prints out the reminder.

In the days before most people used a graphical windowing system, printing out reminders in the background in Agenda Mode was pretty cool. Nowadays, however, it’s mostly useless. We’ll see later that Remind has ways to pop up reminders on a graphical system using helper programs. But for now, if you don’t want Remind to run in the background and issue timed reminders, simply invoke it with the `-q` command-line option.

5.1.2 Fine Control over Queuing

If you have some reminders that you *never* want queued, you can use the `NOQUEUE` keyword to prevent them from being queued.

Later on, we’ll see how to get Remind to automatically calculate the time of sunrise and sunset, but for now, let’s suppose you’ve determined that the time of sunset on 30 June 2026 where you live is 20:55 and you want this in your calendar, but you don’t want Remind printing a reminder in the background at sunset. Simply use this:

```
REM 2026-06-30 AT 20:55 NOQUEUE MSG %"Sunset%" %! %2
```

5.2 Some Time-Related Substitution Sequences

The previous example use the substitution sequences `%!` and `%2`. This sequence is used for timed reminders; here are some of the more common time-related substitution sequences:

%1 Replaced with “now”, “*m* minutes from now”, “*m* minutes ago”, “*h* hours from now”, “*h* hours ago”, “*h* hours and *m* minutes from now” or “*h* hours and *m* minutes ago” depending on the relationship between the AT-time and the effective time (Section 2.11 on page 19).

%2 Replaced with “at *hh:mmam*” or “at *hh:mmpm*”

%3 Replaced with “at *hh:mm*” where *hh:mm* is the AT-time in 24-hour format.

%! Replaced with “is” if the trigger time is now or in the future, or “was” if it is in the past.

5.3 Advance Warning

We've seen in Section 2.5 on page 14 how Remind can trigger reminders several days before the actual event date. Remind has an analogous facility for issuing queued reminders several minutes before the actual event time. Consider these two examples:

```
REM Friday AT 11:00 MSG Piano Lesson is %2.  
REM Friday AT 11:00 +30 *10 MSG Piano Lesson is %2.
```

On Fridays, the first reminder will be queued in the background and then at exactly 11:00, it will be triggered. The second reminder will also be queued, but it will be triggered 30 minutes before the event time (so, at 10:30) and every 10 minutes thereafter until 11:00.

The $+n$ part of an AT class is called a *time delta* and the $*m$ part is called a *time repeat*.

Remind handles queued reminders as follows:

- If there is a time delta of n , then the reminder is issued n minutes before the actual event time.
- If there is a time repeat of m , then the reminder is re-issued every m minutes until the actual event time.
- If there is a time delta of n and a time repeat of m , the reminder is *always* triggered on the actual event time, even if n is not a multiple of m

5.4 Specifying Durations

A timed reminder often has an end time as well as a start time. For example, you may have a 45-minute singing lesson every Friday at 9:15AM. You can specify that in either of the following ways:

```
REM Friday AT 9:15 DURATION 45 MSG Singing lesson  
REM Friday AT 9:15 DURATION 0:45 MSG Singing lesson
```

The first form specifies the duration as an integer number of minutes, while the second form specifies it as an hour:minute duration. So for example, DURATION 90 and DURATION 1:30 are the same.

5.5 Syntactic Sugar for a Specific Date and Time

If you have a reminder on a specific date and time, you can use a shortcut syntax to combine the date and time into one token. For example, the following reminders are all equivalent:

```
REM 8 January 2027 AT 15:30 MSG Susan's Wedding
REM 2027-01-08@15:30 MSG Susan's Wedding
REM 2027-01-08T15:30 MSG Susan's Wedding
```

The first one spells everything out; the second uses Remind-specific syntax for combining a date and time, and the third uses the official [ISO 8601](#) date/time syntax.

5.6 Priority

A REM statement can be given a priority with the `PRIORITY` keyword, which looks like this:

```
REM trigger PRIORITY p MSG body
```

The priority *p* can be any integer from 0 to 9999. If a REM command has no `PRIORITY` keyword, then the default priority is 5000.¹

Remind, by default, does not make use of the priority, but it can be used to break ties if you sort reminders in Agenda Mode.

5.7 Sorted Output in Agenda Mode

By default, in Agenda Mode, Remind issues reminders in the order they're encountered in the reminder script. For example, consider this script:

```
REM Wed +5 AT 16:30 MSG Piano Lesson %! %b
REM 10 Jan 2026 +3 MSG Meeting %! %b.
REM 9 Jan +3 MSG Bob's birthday %! %b.
```

If we run Remind in Agenda Mode on 2026-01-09, the output is as follows (blank lines have been removed.)

```
Piano Lesson is in 5 days' time
Meeting is tomorrow.
Bob's birthday is today.
```

¹Actually, the default priority can be changed using something called a *system variable*; see the **remind**(1) man page and look for `$DefaultPrio`

As you see, the reminders come out in the same order they appeared in the script. However, if you supply the `-g` command-line option, then the reminders are sorted by date and time. The output is:

```
Bob's birthday is today.  
Meeting is tomorrow.  
Piano Lesson is in 5 days' time
```

The `-g` option has a number of sub-options to control the sorting; the full syntax of the option is:

```
-g[a|d[a|d[a|d[a|d]]]]
```

In other words, `-g` may be followed by up to four additional characters; each additional character must be an `a` or a `d`.

The characters have the following meanings:

- If the first character is `a`, then reminders are sorted in ascending order by trigger date. If it is `d`, then the reminders are sorted in descending order by trigger date.
- If the second character is present, then reminders with the same trigger date are further sorted by trigger time, in ascending or descending order depending on whether the second character is `a` or `d`.
- If the third character is present, then reminders with the same trigger date and time are further sorted by priority, in ascending or descending order depending on whether the third character is `a` or `d`.
- If the fourth character is present and is `a`, then untimed reminders are sorted after timed reminders. If it is `d`, then untimed reminders are sorted before timed reminders.

If any character after `-g` is omitted, it is treated as `a`. Therefore, just plain `-g` will sort reminders in ascending order by trigger date; then ascending order by trigger time; then ascending order by priority; and finally timed reminders will be sorted before untimed reminders.

Chapter 6

Expressions

So far, you've seen that Remind has some pretty powerful ways to specify events. I'm going to switch gears in this chapter and talk about *expressions*, a part of Remind that at first blush seems to have absolutely nothing to do with specifying events. But don't worry; we'll tie it all together before the end of this chapter.

6.1 Variables

A variable in Remind (as with many other programming languages) is simply a named slot that can hold a *value*. In Remind, variables are *global* and can have names consisting of letters, numbers and underscores, though the first character must be a letter or an underscore.

You set the value of a variable using the SET command. Here are some examples:

```
SET adv_warning 7
SET message "I'm sorry Dave, I'm afraid I can't do that"
```

The first SET command sets the variable `adv_warning` to the integer value 7. The second SET command sets the variable `message` to the string value `"I'm sorry Dave, I'm afraid I can't do that"`.

6.2 System Variables

Remind has another type of variable called a *system variable*. These variables' names always begin with `$`. Unlike normal variables that can be brought into existence simply with a SET command, system variables always exist and only a specific list of system variables exists; you can't create new ones.

System variables have a multitude of uses, but generally they are used to query something about the internals of Remind, or to alter the behavior of Remind.

For example, the system variable `$DontQueue` returns 1 if `Remind` was invoked with the `-q` command-line option, or 0 if not. It is read-only: You can use it in an expression, but you cannot `SET` it.

An example of a writable system variable is `$AddBlankLines`. Normally, `Remind` adds a blank line between reminders in Agenda Mode, but if you `SET $AddBlankLines` to 0, then these blank lines are not added.

Writable system variables have restrictions on what values they can be `SET` to. `Remind` will issue an error if you try to set a system variable to a value that it does not support.

This book will document some system variables; for an exhaustive list, see the **`remind(1)`** man page.

6.3 Data Types

A variable has no intrinsic data type; it simply takes on the type of whatever value it was last set to. `Remind` itself has five data types:

1. **INT** - an integer. `Remind`'s **INT** type uses the underlying C *int* type. If your machine uses 32-bit twos-complement arithmetic for the C *int* type (which most modern UNIX systems do), then the `Remind` **INT** type can range from -2 147 483 648 to 2 147 483 647.
2. **STRING** - a string of bytes or characters. By default, strings can be a maximum of 65535 bytes long, though this limit can be changed at run-time by setting the system variable `$MaxStringLen`. In `Remind`, a **STRING** can be viewed in two ways:
 - (a) As a sequence of *bytes*. From this viewpoint, a string is just a sequence of 8-bit numbers from 1 to 255, terminated by a byte of 0.
 - (b) As a sequence of *characters*. From this viewpoint, a string is a sequence of UNICODE characters where some characters might require multiple 8-bit bytes (so-called *multibyte strings*.) Even from this viewpoint, a string is still terminated by a byte of 0, which cannot appear in the middle of a string.

`Remind` has functions for processing strings from both points of view. If you are using a UTF-8 locale (which you really, really should be doing) then the character interpretation of strings assumes they are validly UTF-8-encoded.

If you know for sure that all of your strings are ASCII, then the byte-oriented functions can be used safely and are faster than the character-oriented functions. If you are using non-ASCII characters, then it's better to use the character-oriented functions.

3. **DATE** - a date (with granularity one day) on or after 1990-01-01. Internally, `Remind` stores **DATE** types as the number of days since 1990-01-01.
4. **TIME** - a time of day (with granularity one minute). A **TIME** can range from 00:00 (midnight) to 23:59. Internally, `Remind` stores **TIME** types as the number of minutes since midnight. Any

arithmetic on a TIME type is done modulo the number of minutes in a day (which happens to be 1440.)

5. DATETIME - a date and time with granularity one minute. Internally, Remind stores DATETIME types as the number of minutes since 1990-01-01@00:00.

6.4 Constants

A constant is a value that is fixed—for example, the number 7 or the date 2026-09-04. Each of the five Remind data types has rules for how you write a constant of that type:

1. An INT constant is written as a sequence of decimal digits, optionally preceded by a minus sign. You can also write an INT constant in hexadecimal by preceding it with 0x. Here are some examples of INT constants:

```
7
2029
-45
0xFE01
```

2. A STRING constant is written as a sequence of zero or more characters enclosed in double-quotes. Here are some examples of STRING constants:

```
""
"Hello, world!"
"This here: \" is a quote"
"Here is a newline.\n"
```

The first example is the *empty string* consisting of no characters. The last two examples show some *backslash escapes*. Remind supports the following backslash escapes:

- \a - the bell character (ASCII character 7.)
- \b - the backspace character (ASCII character 8.)
- \f - the form-feed character (ASCII character 12.)
- \n - the newline character (ASCII character 10.)
- \r - the carriage-return character (ASCII character 13.)
- \t - the horizontal tab character (ASCII character 9.)
- \v - the vertical tab character (ASCII character 11.)
- \\ - a literal backslash.
- \" - a double-quote character.
- \xab - here, *a* and *b* are hexadecimal digits and the corresponding byte is inserted into the string. The sequence \x00 is not permitted.

3. A DATE constant is written in the form YYYY-MM-DD or YYYY/MM/DD, *enclosed in single-quotes*. Here are some date constants:

```
'2025-02-28'
'1999/01/01'
'2040-12-31'
```

4. A TIME constant is written in the form HH:MM or HH.MM. Normally, HH ranges from 00 to 23 (meaning times are written in 24-hour format). However, you can append “am” or “pm” to write times in 12-hour format. Here are some time constants:

```
14:45
9.13
2:45pm (which is equivalent to 14:45)
```

5. A DATETIME constant is written as a date followed by either @ or T followed by a time, all enclosed in single quotes. Here are some DATETIME constants:

```
'2025-12-31@14:45'
'1990/01/01T00.00'
'2055-02-22@09:30'
```

6.5 True and False Values

Every data type has a *zero value* which is treated as **false** in contexts where a logical true/false value is expected. Every value *other than* the zero values is treated as **true**. The zero values for the various types are:

- INT 0
- STRING "" (the empty string)
- DATE '1990-01-01'
- TIME 00:00
- DATETIME '1990-01-01@00:00'

6.6 Operators

An *operator* performs a calculation on one or more values, resulting in a new value. For example, consider the following two SET commands:

```
SET a 5 * 6
SET b a + 2
```

In the first SET command, the operator `*` is a multiplication operator; it operates on its *operands*, namely 5 and 6 to yield the product, 30. So after this SET command is executed, the variable `a` is set to the value 30.

In the second SET, command, the operator `+` is an addition operator. It adds its two operands, namely `a` (which has just been assigned the value 30) and 2 to yield 32, and that is the value assigned to `b`.

This second example shows how to write an expression containing a variable; wherever a value is expected, you simply write the variable's name. If you use a variable that has not yet been assigned a value, an error results. For example, this script:

```
SET a 45 * nonexistent_variable
```

yields the following error:

```
Undefined variable: `nonexistent_variable'
```

Here are all of the operators Remind supports, listed in order of highest to lowest precedence. The first two (`!` and `-`) are *unary operators* that appear just before their only operand. The remaining operators are *binary operators* that appear between their two operands.

- `!` and `-` (logical negation and arithmetic negation)
- `*`, `/` and `%` (multiplication, division and modulus)
- `+` and `-` (addition and subtraction)
- `<`, `<=`, `>` and `>=` (comparisons)
- `==` and `!=` (equality and inequality tests)
- `&&` (logical AND)
- `||` (logical OR)

6.7 Order of Evaluation

Remind uses the following rules when evaluating an expression:

1. Operators with higher precedence are evaluated before operators with lower precedence.
2. Operators with the same precedence are *always* evaluated from left to right.
3. Parentheses may be used to change the order of evaluation.

```
SET a 4 + 5 * 6
SET b (4 + 5) * 6
SET c 4 + 5 + "Hello"
```

In the first example, $5 * 6$ is evaluated first, yielding 30, and then $4 + 30$ is evaluated, yielding a final value of 34 which is stored in a.

In the second example, parenthesis change the order of evaluation. $4 + 5$ is evaluated first, yielding 9. Then $9 * 6$ is evaluated yielding a final value of 54 for b.

In the third example, $4 + 5$ is evaluated first, yielding 9. Then $9 + \text{"Hello"}$ is evaluated. In this context, the $+$ operator acts as a concatenation operator, so the final result stored in c is the STRING "9Hello". The third example shows that the $+$ operator is not necessarily associative, so the left-to-right evaluation rule becomes important.

6.8 Description of Operators

6.8.1 Unary Negation Operators

The logical operator `!` precedes its operand. The operand can be any type. If the operand is one of the zero values (Section 6.5 on page 42) then `!` evaluates to the INT 1. Otherwise, it evaluates to the INT 0. Here are some examples:

```
SET a !3                (a is set to 0)
SET b ! ""              (b is set to 1)
```

The arithmetic operator `-` precedes its operand, which must be of type INT. It returns the negative of its operand. Here are examples:

```
SET a - 44              (a is set to -44)
SET b -a                (b is set to 44)
```


6.8.2 Multiplication, Division and Modulus

The `*` operator takes two operands. If both operands are of type INT, then it returns the product of its operands. If one operand *n* is of type INT (and is non-negative) and the other operand *s* is of type STRING, then it returns a STRING consisting of *n* copies of *s*. Here are some examples:

```
SET a 9 * -3           (a is set to -27)
SET b 3 * "bar"        (b is set to "barbarbar")
SET c b * 2            (c is set to "barbarbarbarbarbar")
```

The `/` operator takes two operands, both of which must be of type INT. It returns the integer part of the quotient of the first operand divided by the second. Some examples:

```
SET a 9/2              (a is set to 4)
SET b (-8)/2           (b is set to -4)
```

The `%` operator takes two operands, both of which must be of type INT. It returns the remainder of the first operand divided by the second. Some examples:

```
SET a 9%2              (a is set to 1)
SET b (-8)%2           (b is set to 0)
```

6.8.3 Addition

The `+` operator takes two operands of various types. Here are all the combinations that are supported:

- If both operands are of type INT, the result is also of type INT and is the sum of the operands.
- If one operand *d* is of type DATE and the other operand *n* is of type INT, the result is of type DATE and is the date found by adding *n* days to *d*.
- If one operand *t* is of type TIME and the other operand *n* is of type INT, the result is of type TIME and is the time found by adding *n* minutes to *t*, modulo 24 hours. That is, the result will *always* range from 00:00 through 23:59.
- If one operand *dt* is of type DATETIME and the other operand *n* is of type INT, the result is of type DATETIME and is the time found by adding *n* minutes to *dt*.
- If at least one operand is of type STRING, then both operands are converted to type STRING. The result is a string that is the concatenation of the two strings.

Here are some examples:

```
SET a 9 + 21                (a is set to 30)
SET a '2024-02-01' + 365    (a is set to '2025-01-31')
SET a 14:33 + 900           (a is set to 05:33)
SET a '2026-07-03@05:45' + 199 (a is set to '2026-07-03@09:04')
SET a "Hello, " + "world."  (a is set to "Hello, world.")
```

6.8.4 Subtraction

The `-` operator takes two operands of various types. Here are all the combinations that are supported:

- If both operands are of type `INT`, the result is also of type `INT` and is the result of subtracting the second operand from the first.
- If both operands are of type `DATE`, the result is of type `INT` and is the number of days from the first operand to the second. The result is negative if the first operand is later than the second.
- If both operands are of type `TIME`, the result is of type `INT` and is the number of minutes from the first operand to the second. The result is negative if the first operand is later than the second.
- If both operands are of type `DATETIME`, the result is of type `INT` and is the number of minutes from the first operand to the second. The result is negative if the first operand is later than the second.
- If the first operand d is of type `DATE` and the second n is of type `INT`, the result is a `DATE` found by subtracting n days from d .
- If the first operand t is of type `TIME` and the second n is of type `INT`, the result is a `TIME` found by subtracting n minutes from t , modulo the number of minutes in a day.
- If the first operand dt is of type `DATETIME` and the second n is of type `INT`, the result is a `DATETIME` found by subtracting n minutes from dt .

Here are some examples:

```
SET a 9 - 7                (a is set to 2)
SET a '2025-01-01' - '2024-01-01' (a is set to 366)
SET a 04:33 - 02:33        (a is set to 120)
SET a '2025-01-31@00:00' - \
    '2025-01-01@00:00'      (a is set to 43200)
SET a '2025-01-01' - 33     (a is set to '2024-11-29')
SET a 03:45 - 600           (a is set to 17:45)
set a '2026-01-31@12:00' - 6000 (a is set to '2026-01-27@08:00')
```

6.8.5 Comparison

The operators `<`, `<=`, `>` and `>=` take two operands. The operands may be of any type, but must both be of the same type.

The operators `<`, `<=`, `>` and `>=` return 1 if and only if the first operand is strictly less than the second, less than or equal to the second, strictly greater than the second, or greater than or equal to the second, respectively. Otherwise, they return 0.

String comparisons are done byte-by-byte. All other comparisons are done numerically, with later DATES, TIMES and DATETIMES being considered greater than earlier ones.

Here are some examples:

```
SET a 9 < 7                (a is set to 0)
SET a 14:33 <= 14:33       (a is set to 1)
SET a '2025-01-01' > '2024-12-31' (a is set to 1)
SET a "fox" >= "dog"        (a is set to 1)
```

6.8.6 Equality and Inequality Operators

The operators `==` and `!=` test for equality and inequality, respectively. They can take operands of any type.

If the operands have *different* types, then `==` returns 0 and `!=` returns 1.

If the operands have the *same* types, then `==` returns 1 if they are equal to one another, or 0 if not. `!=` returns 0 if they are equal to one another, or 1 if not.

Here are some examples:

```
SET a 9 == 7                (a is set to 0)
SET a 9 != 7                (a is set to 1)
SET a "foo" == 14:33        (a is set to 0)
SET a "foo" != 14:33        (a is set to 1)
```

6.8.7 Logical AND

The `&&` operator is a *short-circuit logical AND*. It takes two operands of any type, and works as follows:

- The first operand is evaluated. If it is a *false* value (Section 6.5 on page 42) then that value is returned. The second operand is *not* evaluated in this case.
- Otherwise, the second operand is evaluated and its value is returned as the final value.

Here are some examples:

```
SET a 0 && 3           (a is set to 0)
SET a "" && 3           (a is set to "")
SET a "FOO" && 42       (a is set to 42)
SET a 95 && 00:00       (a is set to 00:00)
```

6.8.8 Logical OR

The `||` operator is a *short-circuit logical OR*. It takes two operands of any type, and works as follows:

- The first operand is evaluated. If it is a *true* value (Section 6.5 on page 42) then that value is returned. The second operand is *not* evaluated in this case.
- Otherwise, the second operand is evaluated and its value is returned as the final value.

Here are some examples:

```
SET a 0 || 3           (a is set to 3)
SET a "" || 3          (a is set to 3)
SET a "FOO" || 42      (a is set to "FOO")
SET a 95 || 00:00      (a is set to 95)
```

6.9 Built-In Functions

In addition to operators, Remind has many built-in *functions*. To call a function, you simply write the function name followed by (, a comma-separated list of arguments, and a closing).

For example, the `max` function can take one or more arguments. They may be of any type (but must all be the same type). It returns the maximum of its arguments. So, for example:

```
SET a max(1, 2, 9, 1, 3, -9)   (a is set to 9)
SET a 5 * max("x", "y")       (a is set to "yyyyy")
```

Remind has *many* built-in functions (145 as of version 06.02.04) and I'm not going to describe them all in this book. For a comprehensive description of the built-in functions, see the **remind(1)** man page. I'll describe a few functions in this chapter and a few more in other chapters as I describe other features of Remind.

6.9.1 Some Built-In Functions

Here are a few built-in functions.

Arithmetic Functions

- `abs(n)` – returns the absolute value of *n* where *n* is of type INT.
- `max(n1, n2, ...)` – returns the maximum value of its arguments. The arguments can be of any type, but must all be the same type.
- `min(n1, n2, ...)` – returns the minimum value of its arguments. The arguments can be of any type, but must all be the same type.
- `sgn(n)` – returns 1, 0 or -1 if the INT argument *n* is greater than, equal to, or less than zero, respectively.

String Functions

- `asc(str)` – returns the value of the first byte in the STRING *str* (an INT ranging from 1 to 255, or 0 if *str* is the empty string.)
- `char(n)` – returns a single-byte string consisting of the byte *n*. *n* must be an INT from 0 to 255; if *n* is 0 then the empty string is returned.
- `codepoint(str)` – returns the UNICODE code-point of the first character in the STRING *str*. It is the character-oriented counterpart of `asc`. Whereas `asc` treats its argument as a string of bytes, `codepoint` treats it as a string of characters in the current locale.
- `index(haystack, needle [, start])` – Returns the 1-based byte index where the STRING *needle* appears in the STRING *haystack*, or 0 if *needle* is not found. If the optional INT argument *start* is provided, then the search starts from that byte index.

Here are some examples:

```
SET a index("babble", "bb")      (a is set to 3)
SET a index("babble", "bob")     (a is set to 0)
SET a index("abcabcabc", "a")    (a is set to 1)
SET a index("abcabcabc", "a", 2) (a is set to 4)
```

- `mbchar(n)` – returns a string that contains the (possibly multi-byte) UNICODE character whose code-point is the INT *n*. It is the character-oriented counterpart of `char`.

Here are some examples:

```

SET a mbchar(128578)           (a is set to "😊")
SET a char(240) + char(159) + \
    char(153) + char(130)      (a is set to "😊")

```

The second example assumes the locale is UTF-8 and it shows that the UTF-8 representation of 😊 is 4 bytes long.

- `mbindex(haystack, needle [, start])` – This is the character-oriented counterpart to `index` (which is byte-oriented).
- `strlen(str)` – returns the length *in bytes* of the string *str*.
- `mbstrlen(str)` – returns the length *in characters* of the string *str*.
- `substr(str, start [, end])` – Returns a STRING consisting of all the bytes in *str* starting from *start* (a 1-based offset) up to and including *end*. If *end* is omitted, then it defaults to the length of *str*. Both *start* and *end* must be of type INT.
- `mbsubstr(str, start [, end])` – Returns a STRING consisting of all the characters in *str* starting from *start* (a 1-based offset) up to and including *end*. If *end* is omitted, then it defaults to the length of *str*. Both *start* and *end* must be of type INT.
- `ord(n)` – returns a STRING representing the ordinal number corresponding to the INT *n*. Here are some examples:

```

SET a ord(1)           (a is set to "1st")
SET a ord(4)           (a is set to "4th")
SET a ord(13)          (a is set to "13th")
SET a ord(23)          (a is set to "23rd")

```

- `pad(src, padstr, len [, right])` – Converts the *src* argument to a STRING (if necessary). If it is shorter than *len* bytes, then it is padded on the left with copies of *padstr* (possibly including a partial copy) until it is exactly *len* bytes long. If the optional *right* argument is supplied and non-zero, then padding is done on the right instead of the left.
- `mbpad(src, padstr, len [, right])` – This is the character-oriented counterpart to `pad`.
- `plural(num [, str1 [, str2]])` – Takes from 1 to 3 arguments. If one argument is supplied, returns "s" if *num* is not 1, and "" if *num* is 1.

If two arguments are supplied, returns *str1* if *num* is 1 or *str1* + "s" if *num* is not 1. If three arguments are supplied, returns *str1* if *num* is 1 and *str2* otherwise.

Date and Time Functions

- `today()` – Returns the effective date (Section 2.11 on page 19) as a DATE object.
- `now()` – Returns the effective time (Section 2.11 on page 19) as a TIME object.
- `current()` – Returns the effective date and time as a DATETIME object.
- `realtoday()` – Returns the system date (Section 2.11 on page 19) as a DATE object.
- `realnow()` – Returns the system time (Section 2.11 on page 19) as a TIME object.
- `realcurrent()` – Returns the system date and time as a DATETIME object.
- `isomitted(x)` – Given a DATE or DATETIME *x*, returns 1 if (the date part of) *x* is in the global OMIT context, or 0 if it is not.
- `day(x)` – Given a DATE or DATETIME *x*, returns the day of the month component (as an INT).
- `wkdaynum(x)` – Given a DATE or DATETIME *x*, returns the day of the week, where 0 is Sunday, 1 is Monday, and so on until 6 which is Saturday.
- `monnum(x)` – Given a DATE or DATETIME *x*, returns the month number from 1 to 12.
- `year(x)` – Given a DATE or DATETIME *x*, returns the year (as an INT).
- `mon(x)` – Given a DATE or DATETIME *x*, returns a STRING that is the English name of the month.
- `hour(x)` – Given a TIME or DATETIME *x*, returns the hour (as an INT from 0 to 23).
- `minute(x)` – Given a TIME or DATETIME *x*, returns the minute (as an INT from 0 to 59).
- `weekno(date)` – Given a DATE *date*, returns the [ISO 8601 week number](#) of the given *date*. `weekno` can take additional arguments; consult the **remind**(1) man page for more details.

Logical Functions

- `choose(n, arg1 [, arg2 ...])` – Takes at least two arguments. If *n* is less than 1, then *arg1* is returned. Otherwise, the *n*th subsequent argument is returned. If *n* is greater than the number of subsequent arguments, then the last argument is returned.

Note that `choose` only evaluates *n* and whichever argument it actually returns; all other arguments are not evaluated at all.

- `iif(test1, arg1 [, test2, arg2, ...], default)` – takes at least three arguments and must have an odd number of arguments. The function operates as follows:
 1. `test1` is evaluated. If it is true, then `arg1` is returned.
 2. Otherwise, `test2` (if present) is evaluated and if it is true, `arg2` is returned.
 3. This continues for all of the `testN` arguments. If none of the tests evaluates to true, then the final argument `default` is returned.

Note that `iif` evaluates only those arguments that are needed to compute a final value. Any arguments that are not needed are not evaluated.

- `catch(arg1, arg2)` – evaluates `arg1` and if no error occurs, returns the resulting value. Otherwise, evaluates `arg2` and returns it.

`catch` can be useful for using a default value if a variable is not set. For example:

```
SET b catch(a, 42)
```

If `a` is defined, then `b` will be set to the value of `a`. Otherwise, `catch` suppresses the usual “Undefined variable” error and returns 42 (which is then assigned to `b`.)

6.10 User-Defined Functions

In addition to built-in functions, Remind lets you define your own user-defined functions, using the `FSET` command. The `FSET` command looks something like this:

```
FSET name(arg1, arg2, ...) expression
```

A user-defined function may take zero or more arguments, which have the same syntax as variable names. When a user-defined function is evaluated, any reference to an argument in *expression* will refer to the function argument rather than a global variable.

Here are some examples of user-defined functions:

```
FSET constant_function() 98
SET a constant_function()      (a is set to 98)

FSET sumsq(x, y) x*x + y*y
SET a sumsq(3, 4)              (a is set to 25)

FSET weirdo(x) a * x
SET a weirdo(2)                (a is set to 50)
```


In the second example, x and y refer to function arguments rather than global variables.

In the last example, the a in the expression $a * x$ refers to the *global variable* a , because there is no a in the list of function arguments.

6.10.1 Redefining a Function

Remind normally issues a warning if you redefine a user-defined function. You can suppress the warning by putting a $-$ between the FSET and the function definition. For example:

```
FSET func(x) x*2
FSET func(x) x*3           (Remind issues a warning)
FSET - func(x) x*4         (No warning is issued)
```

6.11 Recursive Functions

It is possible to write recursive functions, though rarely useful. Consider the following example:

```
FSET sum_to(n) iif(n <= 0, 0, n + sum_to(n-1))
```

Because *iif* only evaluates *one* of the second or third arguments, this recursive function will eventually terminate when called with a positive integer value n because the termination condition $n \leq 0$ will eventually be hit.

However, Remind limits the number of recursive calls to 1000 and this is a hard limit. Consider these examples:

```
FSET sum_to(n) iif(n <= 0, 0, n + sum_to(n-1))
SET a sum_to(400)           (a is set to 80200)
SET a sum_to(3000)         (error and a is not changed)
```

The final SET command yields the error “Too many recursive function calls” and does not change the value of a .

6.12 Commands for Manipulating Variables and Functions

6.12.1 Unsetting a Variable

We've seen how the SET command can be used to store a value into a variable. If you want to unset a variable so it holds no value, use the UNSET command, which takes a list of variable names. Here is an example:

```
UNSET a b c
```

After the command executes, the variables a, b and c will be unset and will hold no values.

6.12.2 Saving and Restoring Variables

If you have a section of a Remind script where you want to set some variables, but not have the changes “leak out” to other parts of the script, you can use PUSH-VARS and POP-VARS to isolate the scope of the changes. Here is an example:

```
PUSH-VARS a b c
# Code that sets and uses a, b and c
POP-VARS
```

After the POP-VARS command executes, the variables a, b and c will have the same values they did at the time PUSH-VARS was executed. If any of them was unset at the time of PUSH-VARS, then it will be unset after the corresponding POP-VARS.

6.12.3 Unsetting a Function

If you wish to undefine or unset a function, use the FUNSET command. Here is an example:

```
FUNSET func1 foo cabbage
```

After the execution of that command, the functions func1, foo and cabbage will not be defined.

6.12.4 Saving and Restoring Functions

Just as with variables, you may wish to define helper functions for a block of reminders without letting those definitions leak out into surrounding parts of the file and without permanently destroying any previous definitions the functions may have had. The commands PUSH-FUNCS and POP-FUNCS do the work:

```
PUSH-FUNCS foo cabbage
FSET foo(x) x*2
FSET cabbage(x, y) x*3 + y*5
# Use the functions here...
```

```
POP-FUNCS
# Now foo and cabbage have the same definitions they had
# just prior to PUSH-FUNCS. If they were not defined prior
# to PUSH-FUNCS, then they will no longer be defined here.
```

6.13 Expressions and Reminders

“Expressions are cool,” I hear you say, “but Remind is supposed to be a calendar program. How on Earth does any of this stuff help with scheduling?”

The answer is a Remind feature called *expression pasting*.

The way Remind parses a script is as follows: First, it reads an entire line from the script (this may consist of multiple physical lines if you use line continuation.)

Next, it parses the line character-by-character and does the normal lexical analysis to build up tokens and then grammatical parsing that you’d expect from any interpreter. However, the routine that feeds the parser characters has a special feature: Whenever it encounters text of the form `[expr]` where *expr* is a Remind expression, it evaluates *expr*, converts the result to a string, and *replaces* the entire `[expr]` text with that string. Then it feeds the parser characters from the result. So compare the following two lines:

```
REM 2026-07-01 MSG July 1st!
REM ['2026-07-08' - 7] MSG July [ord(1)]!
```

In the second example, the text `['2026-07-08' - 7]` is *replaced* by the result of the expression (which happens to be 2026-07-01) and the text `[ord(1)]` is replaced by 1st, which is the result of evaluating `ord(1)`.

The Remind parser has *no idea* that anything nefarious happened and it sees the two lines as identical.

Now, you might think, “That’s cool. But also pointless.” and given what I’ve shown you so far, you’d be right. However, have a look at this:

```
REM [easterdate(today())] +7 MSG %"Easter Sunday%" is %b.
```

The function `easterdate` computes the date of Easter Sunday that falls on or after its argument. If you run the above snippet on April 1st, 2026, the result will be:

```
Easter Sunday is in 4 days' time.
```

This is a *far* more convenient way of calculating Easter than trying to do it by hand. Remind has many built-in functions for more complex date calculations, and expression-pasting is invaluable for solving complex problems.

In the next chapter, we will look at the SATISFY keyword, that lets you solve some very tricky recurrence problems using expressions.

Chapter 7

The SATISFY Keyword

We've seen in Chapters [2](#) through [4](#) that Remind has pretty powerful techniques for computing recurring reminders. But here's something you can't do with what you've learned so far: Issue a reminder every Friday the 13th.

Your first attempt might be something like this:

```
REM Friday 13 MSG Boo!
```

But this does not work. Because `Friday 13` means to issue the reminder on the first Friday on or after the 13th of a month. So for example, it will be issued on December 19, 2025 which is a Friday, but certainly not a Friday the 13th.

This is where SATISFY can help. To use it, put a clause of the form `SATISFY [expr]` just before the MSG keyword, like this:

```
REM trigger SATISFY [expression] MSG body
```

Note that the square brackets are literals and must be there. Additionally, SATISFY is a special case that does not follow the normal expression-pasting rules; the expression after the SATISFY is *not* evaluated when the line is parsed. Instead, it is evaluated *each time* Remind runs the SATISFY algorithm described in [Section 7.1 on the next page](#).

Anyway: here is how you can issue a reminder every Friday the 13th:

```
REM 13 SATISFY [$Tw == 5] MSG Boo!
```

Wait, *what*?

7.1 The Operation of SATISFY

Here's how SATISFY works:

1. The next trigger date is calculated as usual using the other parts of the REM trigger specification following the algorithm in Section 2.4 on page 14.
2. Next, the expression following the SATISFY keyword is evaluated. If this expression returns a **true** value, then the trigger date calculated previously is the ultimate trigger date and we are done.
3. Otherwise, add 1 to the trigger date calculated previously and compute a *new* trigger date that is on or after this date. We then go back to step 2.

The final piece of the puzzle that you need to know is that the funny expression \$Tw is a *system variable*. All system variables have names starting with \$. The specific \$Tw system variable holds the weekday of the *most recently computed trigger date*, where 0 is Sunday and 6 is Saturday. So let's see what happens if you run the previous example on December 1st, 2025.

1. Remind calculates the trigger date of REM 13. It happens to be Saturday, 2025-12-13.
2. Remind evaluates the SATISFY expression, which returns false, because the weekday (\$Tw) is 6 rather than 5.
3. Remind adds 1 to the trigger date to get 2025-12-14.
4. Remind calculates the trigger date of REM 13 again, but this time *starting from* 2025-12-14. The result is Tuesday, 2026-01-13
5. Since the SATISFY expression is still false, Remind adds 1 and calculates the trigger date of REM 13 starting from 2026-01-14. The result is Friday, 2026-02-13.
6. Now the SATISFY expression is true, so the final trigger date of the entire REM command is 2026-02-13.

In principle, you can do everything you need just with SATISFY and don't need any other parts of the trigger. For example, this reminder also triggers on Friday the 13th (you need to know that \$Td is a system variable that holds the day of the month of the most recently calculated trigger date):

```
REM SATISFY [$Td == 13 && $Tw == 5] MSG Boo!
```

However, this is a lot slower because Remind has to step forward one day at a time and evaluate the expression for each day; it cannot take advantage of the optimizations built in to the normal Remind algorithm implementation. Consider the following three reminders:

```
REM SATISFY [$Td == 13 && $Tw == 5] MSG Boo!
REM Friday SATISFY [$Td == 13] MSG Boo!
REM 13 SATISFY [$Tw == 5] MSG Boo!
```

If I run the first one 100 000 times on my computer, for the 100 000 days starting on 1990-01-01 and ending on 2263-10-17¹, it takes Remind 2.30 seconds.² The second one takes 0.74 seconds and the third takes 0.58 seconds.³

The reason is that the first reminder has to evaluate the SATISFY expression for every single day as it steps through them one by one.

The second reminder steps forward by 7 days on each attempt, significantly reducing how often the SATISFY expression must be evaluated.

And the third one is even better, stepping forward a month at a time and reducing the SATISFY evaluations even further. In fact, over the 100 000 days, the first REM has to evaluate the SATISFY expression a total of 13 214 377, times, the second one 1 930 626 times, and the third one 482 026 times.

As a general rule when you're figuring out how to use SATISFY, do as much as you can with the most-specific normal REM trigger components. Then add a SATISFY expression to refine or filter the results.

7.2 SATISFY Iteration Limit

The algorithm description in Section [7.1 on the preceding page](#) is not quite accurate. Remind also imposes an *iteration limit* after which it gives up trying to satisfy the SATISFY expression. Consider the following example:

```
REM SATISFY [$Td == 100] MSG Hello!
```

This SATISFY expression can only be satisfied if the day-of-the-month of the trial trigger date is 100. Since the day of the month can be at most 31, the expression will never be true. If you run the above example, the output will be:

```
Can't compute trigger
```

The system variable \$MaxSatIter controls how many times Remind will try to satisfy the expression before giving up. By default, it is set to 10 000 and you should probably not change the default.

You can set \$MaxSatIter using the SET command. If you try something like this:

¹Yes, 100 000 days is a bit under 274 years.

²I have a pretty fast computer.

³In case you are wondering: there are 471 Friday-the-13ths from 1990-01-01 through 2263-10-17, or about 1.7 per year. The first one is 1990-04-13 and the last is 2263-03-13.

```
SET $MaxSatIter 2000000000
REM SATISFY [$Td == 100] MSG Hello!
```

then the output is a bit different:

```
`==': Maximum expression complexity exceeded
```

In this case, another built-in limit is hit that limits the number of expression-evaluation steps on a given line.

Now, it could happen that you think that you need to use a SATISFY expression that will always return false after some date. This is bad practice and should be avoided, but you can tell Remind that you know what you are doing as follows:

```
REM MAYBE-UNCOMPUTABLE 13 SATISFY [$Tw == 5 && $Ty < 2029] MSG Boo!
```

The SATISFY expression above is true if the trigger weekday is Friday *and* the trigger year is less than 2029. Obviously, it will never be satisfied after 2028. Normally, Remind would issue the “Can’t compute trigger” in that case, but the MAYBE-UNCOMPUTABLE keyword suppresses that warning and simply results in the reminder never triggering after 2028.

Of course, as I wrote, this is bad practice. The better approach is to use:

```
REM 13 UNTIL 2028-12-31 SATISFY [$Tw == 5] MSG Boo!
```

7.3 Useful Functions and System Variables

The following built-in functions and system variables are very useful within a SATISFY expression:

- `trigdate()` – returns the most-recently computed trigger date, as a DATE object. The system variable `$T` is equivalent to this function.
- `$Ty` – equivalent to `year(trigdate())`
- `$Tm` – equivalent to `monnum(trigdate())`
- `$Td` – equivalent to `day(trigdate())`
- `$Tw` – equivalent to `wkdaynum(trigdate())`
- `$U` – equivalent to `today()`
- `$Uy` – equivalent to `year(today())`
- `$Um` – equivalent to `monnum(today())`
- `$Ud` – equivalent to `day(today())`
- `$Uw` – equivalent to `wkdaynum(today())`

7.4 A SATISFYing Gallery

This section is a cookbook of some things that are easy to do with SATISFY, but hard or impossible without it.

7.4.1 The Fifth Weekday of a Month

Back in Section [2.9 on page 18](#) I mentioned that there was no straightforward way to create a reminder for the fifth Friday of a month, for example. Well, you now have the tools to do it!

First of all, the fifth Friday of a month must be on or after the 29th. However, writing something like:

```
REM Friday 29 June MSG Fifth Friday in June!
```

is not correct. Running this in June 2026, for example, results in a trigger date of Friday, 2026-07-03, which is certainly not in June. We need to add a SATISFY clause that accepts the trigger if and only if the day-of-the-month of the result is at least 29. So this one works:

```
REM Friday 29 June SATISFY [$Td >= 29] MSG Fifth Friday in June!
```

If we run the above in June 2026, the trigger date ends up being Friday, 2028-06-30. That is the first time after 2026 that June has five Fridays in it.

7.4.2 Observed vs. Actual Holidays

American Independence Day is July 4th. So writing a reminder for that is super-easy:

```
REM 4 July MSG Independence Day
```

However, for Federal government employees, the following rules apply: If July 4th is a Saturday, then they get Friday, July 3rd off. If July 4th is a Sunday, then they get Monday, July 5th off. It's possible to express this with one REM command, but it's much easier to specify the two cases separately:

```
REM 3 July SATISFY [$Tw == 5] MSG Independence Day (observed)
REM 5 July SATISFY [$Tw == 1] MSG Independence Day (observed)
```

The first reminder triggers on July 3rd, but *only* if it's a Friday, and the second triggers on July 5th, but *only* if it's a Monday.

In case you're wondering how to handle both cases in a single REM command, here's one way. Analysis is left as an exercise; convince yourself that this method is slower than the two separate REM commands.

```
REM July SATISFY [$Td == 3 && $Tw == 5 || $Td == 5 && $Tw == 1] \
  MSG Independence Day (observed)
```

7.4.3 US Presidential Election Day

US election day is defined as the Tuesday after the first Monday in November, or in simpler terms, the first Tuesday on or after November 2nd. Presidential elections happen every 4 years, and mid-term elections are also every 4 years, but offset from presidential elections by 2 years.

All of this can be expressed as follows:

```
REM Tuesday 2 November SATISFY [($Ty % 4) == 0] MSG Presidential Election Day
REM Tuesday 2 November SATISFY [($Ty % 4) == 2] MSG Midterm Election Day
```

7.4.4 Credit Card Due Date

Suppose your credit card statement is generated on the 15th of every month, and the due date is 21 days later. How can you make a reminder for the due date? At first glance, it seems impossible because with the different lengths of months, the due date could be anywhere from the 5th to the 8th of the following month.

To solve this problem, ask yourself: “If today is a due date, then what condition holds?” And the answer is: 21 days ago, it was the 15th of a month. This immediately leads to the solution:

```
REM SATISFY [day($T-21) == 15] MSG Credit Card Payment Due
```

For my credit card, things are a little more complex. If the normal due date is on a holiday or weekend, then the actual due date is postponed until after the holiday or weekend. So the code I use for my due date is as follows:

```
# Calculate 21 days after the 15th:
REM SCANFROM -7 SATISFY [day($T-21) == 15]

# Now slide to the first non-omitted day
REM [$T] OMIT SAT SUN AFTER MSG Credit Card Payment Due
```

In this example, we see a SATISFY clause being used *without* a MSG. If you do that, then Remind simply calculates the trigger date and stores it in \$T without printing a reminder.

This value of \$T is used in the *next* REM command as a basis for issuing the reminder, with the AFTER keyword being used to slide the reminder past the weekend or holiday.

Computing \$T in one REM command and then reusing it in a subsequent one is *almost always a bad idea* because it can break the Remind algorithm. Here, I was careful to use a SCANFROM clause for the first REM command to make sure I didn’t break the algorithm.⁴

⁴A SCANFROM clause is not needed for the second REM command because the AFTER keyword automatically adjusts the starting point of the Remind algorithm to ensure it gives a correct result.

For example, the due date in July 2025 is 2025-07-07, which is 22 days after June 15th, not 21. That is because July 1st is a holiday in Canada.

If we didn't have the SCANFROM clause in the first REM statement, then on 2025-07-07, it would incorrectly calculate the trigger date as 2025-08-05 since the "21 days after the 15th rule" for July would be in the *past* (2025-07-06).

7.4.5 Recycling Date

My plastic recycling pickup happens every second Wednesday. However, if that day *or* the previous Monday or Tuesday is a holiday, then pickup is deferred to Thursday. (If Thursday is also a holiday, pickup still happens on the Thursday.)

The 2-week cycle started on Wednesday, 31 October 2012. Let's break this down.

First of all, how can we recognize a regular Wednesday pickup? Well, this happens on a multiple of 14 days from 2012-10-31 if and only if none of the Monday, Tuesday or Wednesday is a holiday. Let's write a function for this:

```
# Normal pickups: Every 14 days from the start date,
# but only if none of Monday, Tuesday or Wednesday is
# a holiday.
FSET normal_recycling(x) ( (x-'2012-10-31') % 14 == 0 && \
    !isomitted(x-2) && !isomitted(x-1) && !isomitted(x) )
```

OK, now how about the delayed recycling pickup? That happens on 1 more than the multiple of 14 days, but only if at least one of the preceding Monday, Tuesday or Wednesday is a holiday. This gives us:

```
# Delayed pickups: One day after every 14 days from the start date,
# but only if at least one of Monday, Tuesday or Wednesday is
# a holiday.
FSET delayed_recycling(x) ( (x-'2012-10-31') % 14 == 1 && \
    (isomitted(x-3) || isomitted(x-2) || isomitted(x-1) ) )
```

Note that the amounts subtracted in the `isomitted()` calls are 1 more than in `normal_recycling` because now we're looking at the omitted dates from the point of view of Thursday rather than Wednesday. Finally, recycling happens either on a normal recycling day or a delayed recycling day:

```
FSET is_recycling(x) normal_recycling(x) || delayed_recycling(x)
```

And finally, the REM command is trivial:

```
REM SATISFY [is_recycling($T)] MSG Plastic Recycling
```

If we want to be very fancy, we can add a “ (Delayed)” suffix to delayed pickups like this:

```
# If the pickup day is a Thursday, then pickup is delayed
FSET is_delayed(x) iif(wkdaynum(x) == 4, " (Delayed)", "")
REM SATISFY [is_recycling($T)] MSG Plastic Recycling[is_delayed($T)]
```

In the body of a reminder, the system variable `$T` refers to the final trigger date after the SATISFY expression (if present) was satisfied. In fact, Remind does not even begin to parse the body of a reminder until the trigger date has been calculated, so any expressions in the body of a reminder will see the final, calculated trigger date when using any of the related built-in functions or system variables.

7.4.6 End-of-Quarter

Many companies divide their financial reports into quarters, with the quarters ending on 31 March, 30 June, 30 September and 31 December. While this could be implemented as four separate reminders:

```
REM 31 March      MSG End of Q1
REM 30 June       MSG End of Q2
REM 30 September  MSG End of Q3
REM 31 December   MSG End of Q4
```

Why not do it in one REM command?

```
REM ~1 SATISFY [($Tm % 3) == 0] MSG End of Q[$Tm/3]
```

Without the SATISFY clause, the reminder would trigger on the last day of every month. But the SATISFY expression is true only for month numbers that are a multiple of 3, hence the whole command triggers only in March, June, September and December. In the body of the reminder, we use expression-pasting to insert the proper quarter number.

Chapter 8

Some Callback Functions

Sometimes, rather than using SATISFY for complicated reminders, a different approach that uses a function to manipulate the OMIT context is an easier way to write reminders. The OMITFUNC keyword lets you do this. We'll also look at a couple of other keywords that give you finer-grained control over advanced warning and reminder queuing. And after that, we'll look at unrelated callback functions that let you modify the appearance of reminders.

8.1 OMITFUNC

Recall the recycling day example in Section [7.4.5 on page 63](#). It required a pretty complicated SATISFY expression to get it to work properly. But really, the dead-simple reminder:

```
REM 2012-10-31 *14 AFTER MSG Plastic Recycling
```

is *almost* correct. The only problem is that for this specific reminder, we want the Wednesday to be considered OMITted if it is omitted *or* the previous Tuesday is *or* the previous Monday is. A small adjustment fixes the problem:

```
# Return true if: Today is a Wednesday, and
# either today, yesterday or the day before yesterday is a holiday.
FSET recycle_holiday(x) wkdaynum(x) == 3 && \
    (isomitted(x) || isomitted(x-1) || isomitted(x-2))
```

```
REM 2012-10-31 *14 OMITFUNC recycle_holiday AFTER MSG Plastic Recycling
```

Here's how it works: If a REM command has an OMITFUNC clause, then *instead* of looking in the global OMIT context to see if a date is OMITted, Remind calls the function named in the OMITFUNC clause with

a single argument of type DATE. The function should return **true** if Remind should consider the date to be OMITted, or **false** if it should not.

In our example, the function returns true if the date is a Wednesday, and if *any* of the date itself, the previous day, or the day before that is OMITted. In other words: If the Monday, Tuesday or Wednesday is OMITted, the function returns true. Otherwise, it returns false.

We also ensure that the function only returns true if the trigger day is a Wednesday, because if a Wednesday and a Thursday happen to be holidays, collection still takes place on the Thursday.

The solution using OMITFUNC is faster than the SATISFY solution in Section 7.4.5 on page 63 because the built-in Remind algorithm does more of the “heavy lifting” and fewer expression evaluations are required. I also find it easier to understand, though you should pick whichever way you find easiest.

8.2 The WARN Keyword

Recall the delta feature (Section 2.5 on page 14) that (in Agenda Mode) gives you advance warning of an upcoming reminder. The WARN keyword gives you more control over exactly which days the advance warning is issued.

The WARN keyword must be followed by the name of a function. This function is passed a single argument of type INT and must return an INT. To figure out the days on which to give advance warning, Remind calls the WARN function successively with the arguments 1, 2, 3, ... The return value is interpreted as follows:

- If the return value n is positive, then the reminder is triggered exactly n days before its trigger date, just as if a delta of $++n$ had been supplied.
- If the return value n is negative, then the reminder is triggered m days (where $m = |n|$) before its trigger date not counting OMITted dates, just as if a delta of $+m$ had been supplied.
- If an error occurs during the evaluation of the function, or it returns a non-INT, or the absolute values of the return values are not monotonically decreasing, then Remind stops calling the function and subsequently only triggers the reminder on its actual trigger date.
- As a matter of style, the final return value from the WARN function should be zero, but even if this is not the case, Remind *always* triggers the reminder on its actual trigger date.

With all of that in mind, consider this example:

```
FSET wfun(x) choose(x, 5, 3, 1, 0)
REM 4 July WARN wfun MSG American Independence Day is %b.
```

As wfun is called with the successive values 1, 2, 3, ... it returns 5, 3, 1 and 0. So the reminder is triggered 5 days before, 3 days before, 1 day before and on the day of 4 July. (In other words, it is triggered on 29 June, 1 July, 3 July and 4 July.)

8.3 The SCHED Keyword

Just as WARN gives you more control over a date delta, SCHED gives you more control over a time delta for queued reminders. The SCHED keyword (as with WARN) must be followed by the name of a function, which is successively called with the arguments 1, 2, 3, ... The result of the function controls when the queued reminder is issued.

The return value must be an INT or a TIME. It is interpreted as follows:

- If the return value is a negative INT, then the reminder is issued that many minutes before the trigger time.
- If the return value is a positive INT, then the reminder is issued that many minutes after it was previously issued.
- If the return value is a TIME, then the reminder is issued at that time.

Again, an example will probably serve to clarify:

```
FSET sfun(x) choose(x, -60, 30, 15, 10, 3, 1, 1, 1, 1, 0)
REM AT 13:00 SCHED sfun MSG Meeting %2.
```

When this reminder is queued, it will first be triggered at 12:00 (which is 13:00 - 60 minutes). It will then be triggered 30 minutes later, at 12:30. Then 15 minutes after that at 12:45, then at 12:55, 12:58, 12:59, 13:00, 13:01 and 13:02. Note that a SCHED function can request that Remind continue to trigger a queued reminder even *after* the trigger time.

As with WARN, if the SCHED function yields an error, does not return a TIME or an INT, or the sequence of times is not monotonically increasing, Remind stops calling it.

8.4 Prefix and Suffix Callback Functions

If you define certain user-defined functions, then Remind calls them to prefix and/or suffix the text that normally appears in a reminder. These are called *callback functions* and the four that you can use to modify how reminders appear are:

- `msgprefix(priority)` – If this function is defined, it is called with a single INT argument that is the priority of the REM command being executed. It must return a string, and that string is prefixed to the reminder when the reminder is output.

`msgprefix` is called *only* in Agenda Mode.

- `calprefix(priority)` – If this function is defined, it is called with a single INT argument that is the priority of the REM command being executed. It must return a string, and that string is prefixed to the reminder when the reminder is output.

`calprefix` is called *only* in Calendar Mode.

- `msgsuffix(priority)` – If this function is defined, it is called with a single INT argument that is the priority of the REM command being executed. It must return a string, and that string is suffixed to the reminder when the reminder is output. Note that normally, Remind leaves a blank line after a reminder and the result of `msgsuffix` will appear on a new line. If you want it to appear on the *same* line as the reminder, make sure the return value of `msgsuffix` begins with a backspace character, or `char(8)`.

`msgsuffix` is called *only* in Agenda Mode.

- `calsuffix(priority)` – If this function is defined, it is called with a single INT argument that is the priority of the REM command being executed. It must return a string, and that string is suffixed to the reminder when the reminder is output.

`calsuffix` is called *only* in Calendar Mode.

Here's an example of how you might use these functions. I have set the system variable `$AddBlankLines` to 0 to suppress the blank lines that normally appear between reminders.

```
# $DefaultPro is the default priority of a REM statement
# and is set to 5000 by default
SET $AddBlankLines 0
FSET msgprefix(p) iif(p > $DefaultPrio, "URGENT: ", \
                        p < $DefaultPrio, "Non-urgent: ", \
                        "Normal: ")
REM PRIORITY 9000 MSG Buy cat food
REM PRIORITY 5000 MSG Check lottery ticket
REM PRIORITY 1000 MSG Polish shoes
```

The output of this script in Agenda Mode is:

```
URGENT: Buy cat food
Normal: Check lottery ticket
Non-urgent: Polish shoes
```

8.5 Sortbanner

Another callback function that you can define is `sortbanner`, which takes a single DATE argument. If you use the `-g` command-line option to sort reminders, then `sortbanner` is called just before each block of reminders on a different date. The result is passed through the substitution filter and printed.

This is most easily explained with an example. Consider this script `sorted.rem`:

```
SET $AddBlankLines 0
# Switch off the normal banner
BANNER %
REM 11 March 2026 ++1 MSG Not so important
REM 17 March 2026 ++7 MSG Way in the future
REM 10 March 2026 MSG Important Reminder
REM 11 March 2026 ++1 MSG Not so important - B

FSET sortbanner(x) iif(x == today(), \
    "***** THINGS TO DO TODAY *****", \
    "%_----- Things to do %b -----")
```

If we run this script on 10 March 2026 as follows:

```
$ remind -g sorted.rem 2026-03-10
```

Then the output is:

```
***** THINGS TO DO TODAY *****
Important Reminder

----- Things to do tomorrow -----
Not so important
Not so important - B

----- Things to do in 7 days' time -----
Way in the future
```

Note that the second `iif` return value starts with `%_`, which causes the substitution filter to emit a newline. This spaces the blocks of reminders apart from one another.

Chapter 9

File Inclusion and Flow Control

Remind lets you organize your reminders into different files using the `INCLUDE` family of commands. It also has facilities for conditionally executing portions of a script.

9.1 File Inclusion

We saw earlier that Remind is invoked with the name of a file containing the reminder script. It's often convenient to organize your reminders into several files. For example, you might want one file holding public holidays, another holding birthdays, a third holding work events, and so on.

The `INCLUDE` family of commands lets you do this.

9.1.1 `INCLUDE`

The `INCLUDE` command's syntax is as follows:

```
INCLUDE pathname
INCLUDE "pathname"
```

The first variant may be used if the `pathname` does not contain any spaces. If the `pathname` does contain spaces, then use the second double-quote-enclosed form.

9.1.2 `DO`

If you supply a relative `pathname` to the `INCLUDE` command, then the file is taken from the current working directory, *not* the directory containing the file with the `INCLUDE` command. This is arguably a mistake, but can't be changed for historical reasons. To fix this, we have the `DO` command:

```
DO pathname
DO "pathname"
```

For absolute pathnames, `DO` is equivalent to `INCLUDE`. For relative pathnames, `DO` reads the file relative to the directory containing the file with the `DO` command, rather than relative to the current working directory. `DO` is the most convenient command for organizing your files. For example, you might have a `main.rem` file that includes all of your other files:

```
DO holidays.rem
DO birthdays.rem
DO work.rem
DO personal.rem
```

9.1.3 SYSINCLUDE

The final command for file inclusion is `SYSINCLUDE`. It looks like this:

```
SYSINCLUDE pathname
SYSINCLUDE "pathname"
```

For absolute pathnames, `SYSINCLUDE` is equivalent to `INCLUDE`. Relative pathnames, however, are treated relative to an installation-defined *system include directory*. Remind ships with many useful reminder files containing holidays for various countries, Chinese New Year dates, and so on, and they all live in the system include directory. For example, to get a reminder of Chinese New Year, you can use:

```
SYSINCLUDE holidays/chinese-new-year.rem
```

The system include directory's location is stored in a system variable called `$SysInclude`. You can find out where the directory is by running this Remind script:

```
REM MSG System Include Directory is: [$SysInclude]
```

It's likely to be something like `/usr/share/remind` or `/usr/local/share/remind`. I encourage you to rummage around in this directory and look at some of the reminder files included with Remind.

9.2 Conditional Execution

Remind has a couple of commands that let you conditionally execute certain parts of a script.

9.2.1 The IF Command

The general format of the IF command is as follows:

```
IF expression
    # True-block of commands..
ELSE
    # False-block of commands..
ENDIF
```

The ELSE part can be omitted, so you can also have just this:

```
IF expression
    # True-block of commands..
ENDIF
```

IF works as follows:

1. The *expression* is evaluated.
2. If the result is a true value (Section 6.5 on page 42), then all of the commands in the *True-block of commands* are executed.
3. Otherwise, if there is an ELSE clause, then all of the commands in the *False-block of commands* are executed.
4. ENDIF is mandatory.

The indentation shown in the examples is just for clarity; it is not required although I recommend you use it. Also, IF commands may be nested up to a limit of 64 levels.¹

For an example of how you might use IF, suppose you've organized your reminders into separate files as show in Section 9.1.2 on page 71. And suppose you want an option *not* to include work reminders in some cases. You could write the main file like this:

```
DO holidays.rem
DO birthdays.rem
IF ! catch(exclude_work, 0)
    DO work.rem
ENDIF
DO personal.rem
```

Now, you can invoke Remind in the following ways:

¹If you actually write a Remind script with 64 nested levels of IF, it's time to re-evaluate your approach to life.

```
$ remind main.rem
$ remind -iexclude_work=1 main.rem
```

The first command includes all of your reminders. The second one uses a command-line option `-i` to define a variable on the command-line. The `IF` expression evaluates false if `exclude_work` is true, so the `DO work.rem` command is skipped.

9.2.2 The IFTRIG Command

The `IFTRIG` command lets you conditionally execute parts of a script based on whether or not a trigger specification would trigger. The general format is as follows:

```
IFTRIG trigger_spec
    # True-block of commands..
ELSE
    # False-block of commands..
ENDIF
```

The *trigger_spec* is any trigger specification that would be accepted by a `REM` command, *not including* a `SATISFY` clause. If the *trigger_spec* would trigger today, then the *True-block of commands* is executed. If not, then the *False-block of commands* is executed. As with `IF`, the `ELSE` part is optional.

Here is an example:

```
IFTRIG Mon 1 SEP SCANFROM -7 ++8
    # Executed on the first Monday in September
    # and the preceding 8 days
ELSE
    # Executed on all other days
ENDIF
```

While `IFTRIG` looks like a powerful and useful command, I find it is almost never needed. It's almost always better to have each `REM` command compute its own trigger date from scratch.

9.3 Reading Scripts from Directories

If the pathname you supply to `Remind` either on the command-line or in a `DO` or `INCLUDE` command is a *directory* rather than a plain file, then `Remind` looks inside the directory for files that match the shell pattern `*.rem` and reads them in sorted order (the same sorted order that `echo *.rem` would produce.)

Conventionally, the default name for a `Remind` script is `.reminders` in your home directory. I made a *directory* called `.reminders` that holds all my `*.rem` files. This makes it very easy to organize reminders; I can simply drop in or delete a `*.rem` file where that makes sense.

You might want to name your files starting with two or three decimal digits to ensure a consistent sort order. For example, here are some of the files that live in my `.reminders` directory:

```
000-preamble.rem
010-sun.rem
030-chinese-new-year.rem
062-holidays.rem
064-library-books.rem
080-misc.rem
082-domain-expiries.rem
100-tkremind.rem
101-birthdays.rem
```

9.4 Command Inclusion

The `INCLUDE` family of commands has one more member you haven't seen yet: `INCLUDECMD`. Rather than opening a file, `INCLUDECMD` executes a shell command, whose output is expected to be a Remind script.

This can let you do powerful things. For example, you could write a script that looks up events in a database and emits `REM` commands to create reminders for them. Or you could write a script that pulls down an online iCal calendar and converts it to a Remind script.

Here is an example of how you can (ab)use `INCLUDECMD`. Suppose you have four driving lessons scheduled in July, 2026. One is on the 6th at 10:00; another is on the 9th at 14:00; the third is on the 16th at 17:00 and the last one is on the 20th at 12:00. There's no obvious pattern to these reminders, so you'd normally write them as separate reminders. However, you could also write a script like this:

```
SET lessons "2026-07-06@10:00 2026-07-09@14:00 " + \
    "2026-07-16@17:00 2026-07-20@12:00"

INCLUDECMD L="[lessons]" ; total=`echo $L | wc -w`; n=0; for i in $L ; \
do n=`expr $n + 1`; \
echo "REM $i MSG Driving Lesson ($n of $total)"; done
```

Note that all of the text after `INCLUDECMD` is a *shell script* and is *not* written in Remind's scripting language. The only exception is the expression-pasting of `[lessons]` used to initialize the shell variable `L`.

If we run the above script with the `-n` option, we get:

```
2026/07/06 10:00am Driving Lesson (1 of 4)
2026/07/09 2:00pm Driving Lesson (2 of 4)
2026/07/16 5:00pm Driving Lesson (3 of 4)
2026/07/20 12:00pm Driving Lesson (4 of 4)
```

If your driving lesson schedule changes, you merely need to edit the `SET lessons` line and everything else will adjust. Whether you consider this trick a thing of beauty or a horror show is up to you.

Chapter 10

TODOs

Remind is really designed to remind you about events and is not a TODO manager or a task manager. However, it has limited features that do support TODOs.

10.1 The TODO Keyword

A TODO is a normal REM command, but it includes the TODO keyword. The syntax looks like this:

```
REM TODO trigger [COMPLETE-THROUGH yyyy-mm-dd] [MAX-OVERDUE n] MSG body
```

The TODO keyword modifies the behavior of REM as follows:

- Rather than using today's date as the starting point of the Remind algorithm (Section [2.4 on page 14](#)), Remind uses the COMPLETE-THROUGH date plus one day. If there is no COMPLETE-THROUGH, then Remind starts calculating the trigger date from the *beginning of time*, 1990-01-01.
- If the calculated trigger date is in the past, the reminder is nevertheless triggered. However, if there is a MAX-OVERDUE *n* clause, then the reminder is only triggered for *n* days past its due date.

Examples should clarify this. Suppose you pay rent on the first day of every month, and you are fully paid-up through 2026-09-01. Then you might use this reminder:

```
REM TODO 1 +7 COMPLETE-THROUGH 2026-09-01 MSG %"Rent%" %! due %b.
```

Let's see what happens if we run Remind in Agenda Mode on the previous script on various dates.

```
# 2026-09-01  
No reminders.
```

```
# 2026-09-27  
Rent is due in 4 days' time.
```

```
# 2026-10-01  
Rent is due today.
```

```
# 2026-10-03  
Rent was due 2 days ago.
```

```
# 2026-10-15  
Rent was due 14 days ago.
```

Note a few points:

- The reminder is *not* triggered on 2026-09-01. In the absence of TODO and COMPLETE-THROUGH, it would have been. However, the presence of TODO and the fact that it is marked complete through 2026-09-01 means that the calculated trigger date is 2026-10-01, so the reminder is not triggered on 2026-09-01.
- On 2026-09-27 and 2026-10-01, the reminder is triggered as usual.
- On 2026-10-03 and 2026-10-15, the reminder is still triggered *even though the trigger date is in the past*. Essentially, Remind nags you to complete a TODO until you update the COMPLETE-THROUGH date to indicate that you have done it. You can limit the number of days that Remind nags you with a MAX-OVERDUE clause. For example, if you use MAX-OVERDUE 15, then Remind will stop nagging you about a TODO that is more than 15 days late, on the assumption that it's pointless and you have abandoned the task.

10.2 TODOs and Calendar Mode

TODO-style reminders are treated specially only in Agenda Mode. In Calendar Mode, they appear in the calendar exactly as any normal event would.

Chapter 11

Astronomical Events

Remind has built-in functions for calculating the dates and times of various events like solstices, equinoxes, Moon phases, sunrise and sunset. This chapter describes the astronomical functions.

11.1 Location

In order to correctly compute the times of astronomical events, Remind needs to know your location (latitude and longitude.) There are three system variables that specify your location. They are as follows:

- `$Latitude` – set this to a `STRING` that is a floating-point number representing your latitude in degrees, from -90 to 90. Positive latitudes are north of the Equator and negative latitudes are south of the Equator.

Because Remind does not have a floating-point data type, you have to specify latitude as a `STRING`, but it must have the syntax of a floating-point number. For example, the default latitude that Remind ships with is the equivalent of:

```
SET $Latitude "45.42055556"
```

- `$Longitude` – set this to a `STRING` that is a floating-point number representing your longitude in degrees, from -180 to 180. Positive longitudes are east of the Greenwich Meridian, and negative longitudes are west of the Meridian.

As with latitude, you must specify longitude as a `STRING`, but it must have the syntax of a floating-point number. For example, the default longitude that Remind ships with is the equivalent of:

```
SET $Longitude "-75.68944444"
```

The default latitude and longitude happen to be the location of the City Hall in Ottawa, Ontario, Canada.

- `$Location` – this is optional, but is designed to hold the name of the locale where you are located. The default setting that Remind ships with is the equivalent of:

```
SET $Location "Ottawa"
```

11.2 Sunrise and Sunset

The following built-in functions all take an optional `DATE` or `DATETIME` argument. They return a `TIME`. Note that if a `DATETIME` argument is supplied, only the date portion is used; the time portion is ignored. If no argument is supplied, then a default of `today()` is used.

- `sunrise([x])` – returns a `TIME` object which is the local time of sunrise. Sunrise is defined as the moment when the upper rim of the Sun appears on the horizon in the morning.
Some places may not have a sunrise every day; if the Sun never sets on a specific day, then the INT `0` is returned. If it never rises on a specific day, then the INT `1440` is returned.
- `sunset([x])` – returns a `TIME` object which is the local time of sunset. Sunset is defined as the moment when the upper rim of the Sun disappears below the horizon in the evening.
Some places may not have a sunset every day; if the Sun never sets on a specific day, then the INT `1440` is returned. If it never rises on a specific day, then the INT `0` is returned.
- `dawn([x])` – Similar to `sunrise()`, but returns the time when the center of the Sun is 6 degrees below the horizon in the morning. This is known as civil dawn.
- `dusk([x])` – Similar to `sunset()`, but returns the time when the center of the Sun is 6 degrees below the horizon in the evening. This is known as civil dusk.
- `ndawn([x])` – Similar to `sunrise()`, but returns the time when the center of the Sun is 12 degrees below the horizon in the morning. This is known as nautical dawn.
- `ndusk([x])` – Similar to `sunset()`, but returns the time when the center of the Sun is 12 degrees below the horizon in the evening. This is known as nautical dusk.
- `adawn([x])` – Similar to `sunrise()`, but returns the time when the center of the Sun is 18 degrees below the horizon in the morning. This is known as astronomical dawn.
- `adusk([x])` – Similar to `sunset()`, but returns the time when the center of the Sun is 18 degrees below the horizon in the evening. This is known as astronomical dusk.

Here are examples of how you can use the sun-related astronomical functions:

```
REM NOQUEUE AT [sunrise()] MSG Sunrise %! %2.
REM NOQUEUE AT [sunset()] MSG Sunset %! %2.
# Similar patterns may be used for the other Sun-related functions
```

11.3 Moon-Related Functions

The period of the lunar phases is called a *synodic month*. This is how long the Moon takes to complete one orbit around the Earth¹ and averages about 29 days, 12 hours, 44 minutes and 3 seconds. However, because both the Moon's and the Earth's orbits are elliptical, a synodic month can vary by about seven hours from this mean.

It turns out that 235 mean synodic months is almost exactly 19 years long (it's off by just over two hours) and several *lunisolar calendars* align their months to the moon phase but their years to the seasons by incorporating 7 leap months in a cycle of 19 years. This cycle of 235 mean synodic months is called the *Metonic cycle*.

Remind recognizes four Moon phases: The *New Moon* is when the Moon is completely dark (from Earth's point of view), with the Moon, Earth and Sun aligned so the dark side faces Earth. The *Full Moon* is the opposite: The Moon is as illuminated as it can be, from Earth's perspective, with the Moon, Earth and Sun aligned so the light side directly faces Earth.

The *First Quarter* is halfway between a New Moon and a Full Moon, when half of the Moon appears illuminated and it is heading towards a Full Moon. The *Last Quarter* is halfway between a Full Moon and a New Moon, when half of the Moon appears illuminated and it is heading towards a New Moon.

As an aside, we don't get a lunar eclipse every Full Moon or a solar eclipse every New Moon because the Moon's orbit is tilted by about 5 degrees with respect to Earth's orbit around the Sun, and so eclipses can only happen when the two orbits are suitably aligned.

Remind has several functions related to phases and rising/setting of the Moon. They are as follows:

11.3.1 Moon Phases

- `moonphase([date [, time]])` or `moonphase(datetime)` – This function returns the phase of the Moon on the given date and time, which default to `today()` and `now()`, respectively. The return value is an integer from 0 to 359, where 0 is a new Moon, 90 is the first quarter, 180 is a full Moon and 270 is the last quarter. You may supply the date and time as two separate arguments of type DATE and TIME, or one DATETIME object.
- `moondate(phase [, date [, time]])` or `moondate(phase, datetime)` – given an integer *phase* from 0 to 3 where 0 is new Moon, 1 is first quarter, 2 is full Moon and 3 is last quarter, returns

¹Technically, both the Moon and the Earth orbit around their common center of gravity, However, since this point is some 1707 km below the surface of the Earth, effectively the Moon orbits the Earth.

the a DATE object holding the date on or after the given *date* and *time* (or *datetime*) on which the Moon is at the given phase. If *date* is omitted, it defaults to `today()`. If *time* is omitted, it defaults to `00:00`.

- `moontime(phase [, date [, time]])` or `moontime(phase, datetime)` – takes the same argument as `moondate()` but returns a TIME object giving the exact time of day when the specific Moon phase is reached.
- `moondatetime(phase [, date [, time]])` or `moondatetime(phase, datetime)` – takes the same argument as `moondate()` but returns a DATETIME object containing the date and time of day when the specific Moon phase is reached.

11.3.2 Moonrise and Moonset

The following functions supply information about moonrise and moonset.

- `moonrise([date])` – This function returns a DATETIME giving the date and time of the first moonrise on or after `00:00` on *date*. If *date* is not supplied, it defaults to `today()`.

Note that it is not uncommon for a day to have no moonrise, so the date part of the return value may not be the same as the *date* argument. If you want a calendar of moonrise times, you could use something like this:

```
SET mr moonrise()
IF datepart(mr) == today()
    REM NOQUEUE [mr] MSG Moon rises at %3.
ELSE
    REM MSG No moonrise today
ENDIF
```

- `moonset([date])` – This function returns a DATETIME giving the date and time of the first moonset on or after `00:00` on *date*. If *date* is not supplied, it defaults to `today()`.

As with `moonrise`, there may not be a moonset on a given date, so the date part of the return value may differ from *date*.

- `moonrisedir([date])` – This function takes the same optional argument as `moonrise()` and returns an INT giving the compass direction of the first moonrise on or after `00:00` on *date*. The return value can range from 0 to 359, where 0 is North, 90 is East, 180 is South and 270 is West.
- `moonsetdir([date])` – This function takes the same optional argument as `moonset()` and returns an INT giving the compass direction of the first moonset on or after `00:00` on *date*. The return value is interpreted as for `moonrisedir()`.

11.3.3 Blue Moons

One of the definitions of a Blue Moon is that it is the second full moon in a calendar month. Here's how you can use Remind to figure out when a blue moon is:

```
REM SATISFY [$T == moondate(2, $T) && \
            monnum($T) == monnum(moondate(2, $T-31))] \
MSG Blue Moon!
```

The SATISFY condition is true if (1) today is a full moon, and (2) today's calendar month is the same as the calendar month of the previous full moon.

In Ottawa, Ontario and local Ottawa time, there is a blue moon on 2026-05-31. The one after that is 2028-12-31. And the one after that is 2031-09-30.

11.4 Solstices and Equinoxes

A single function returns information about solstices and equinoxes:

- `soleq(which [, start])` – The integer argument *which* specifies which event we are interested in: 0 specifies the March Equinox, 1 the June Solstice, 2 the September Equinox and 3 the December Solstice.

The optional *start* parameter can be an INT, in which case it specifies the year containing the event we're interested in, or a DATE or DATETIME, in which case we're looking for the first event on or after midnight on the specified date (or date part of the DATETIME.)

The return value of `soleq()` is a DATETIME object giving the date and time of the specified solstice or equinox.

Here's how you might use `soleq`:

```
REM NOQUEUE [soleq(0)] March Equinox %! %3.
REM NOQUEUE [soleq(1)] June Solstice %! %3.
REM NOQUEUE [soleq(2)] September Equinox %! %3.
REM NOQUEUE [soleq(3)] December Solstice %! %3.
```

11.5 Easter

Although the date of Easter does not really correspond to an astronomical event², this seems like a good place to talk about it. Remind has two functions for calculating the date of Easter:

²Easter used to be defined as the Sunday following the first Full Moon on or after the March equinox, but the rules for calculating Easter have since diverged from this definition

- `easterdate([date])` or `easterdate(year)` – If given an INT argument *year*, then returns a DATE object that is the date of Easter Sunday for the specified year. If given a DATE *date*, then returns the date of the next Easter Sunday on or after *date*. If no argument is supplied, then it defaults to `today()`.

Note that `easterdate()` calculates the date of the Western Easter using an algorithm due to Donald Knuth.

Here's how you can use `easterdate()`:

```
REM [easterdate($Uy) - 2] MSG Good Friday
REM [easterdate($Uy)]      MSG Easter Sunday
REM [easterdate($Uy) + 1] MSG Easter Monday
```

Note that I used the INT-argument form of `easterdate()`. (The system variable `$Uy` is defined in Section [7.3 on page 60](#).) That is because the last reminder (Easter Monday) adds 1 to a calculated date and that runs the risk of breaking the Remind algorithm; if I supplied `today()` as the argument to `easterdate()`, then on Easter Monday the return value of `easterdate()` would be the *following* Easter Sunday, about a year into the future, and the reminder would never trigger.

- `orthodoxeaster([date])` or `orthodoxeaster(year)` – Takes the same arguments as `easterdate` and returns a DATE object containing the date of Easter Sunday, but computed according to the Eastern Orthodox rules rather than the Western rules.

Chapter 12

Producing Calendars

We've already discussed Calendar Mode in [Section 2.2 on page 9](#). This chapter delves more deeply into Calendar Mode and also describes helper programs that let you produce PDF or HTML calendars.

12.1 Calendars in the Terminal

We've seen that you can produce a calendar in your terminal by running one of the following commands:

```
$ remind -c filename [date]
$ remind -cu filename [date]
```

The first command uses ASCII characters +, - and | to draw the calendar boxes, whereas the second uses Unicode box-drawing characters.

The full syntax of the -c command-line option is actually:

```
-c[flags][n]
```

Note that the square brackets indicate *optional* components and should not be used literally. The optional *flags* are:

- a – Causes Remind to display reminders on the days they actually trigger *as well as* any preceding days specified by the reminder's delta.
- u – Use Unicode line-drawing characters to draw the calendar boxes.
- + – Generate a weekly calendar instead of a monthly one.

The last part of the option, *n*, is a positive decimal number. It instructs Remind to produce *n* months' (or weeks') worth of calendars.

So, for example, to produce 4 weeks' worth of calendars using Unicode line-drawing characters:

```
$ remind -cu+4 filename
```

12.1.1 If Your Terminal Calendar Displays are Messed Up

If you run `remind -c` and the columns are all out of alignment and funny characters appear, it is likely that your terminal emulator does not support Unicode *left-to-right marks*. In a UTF-8 environment, Remind emits these characters (that should be invisible) to ensure that the calendar printing direction is correct for column alignment, even if your reminders contain some right-to-left text.

If your terminal does not properly support left-to-right marks, you can suppress them by setting this system variable at the top of your Remind script:

```
SET $SuppressLRM 1
```

Alternatively, you can invoke Remind as follows:

```
$ remind -c '-i$SuppressLRM=1' filename [date]
```

12.2 Additional Calendar-Related Command-Line Options

12.2.1 First Day of the Week

Remind normally generates a calendar with Sunday in the leftmost column and Saturday in the rightmost column. Some countries standardize having Monday at the left and Sunday at the right; the `-m` command-line option makes Monday the first day of the week:

```
$ remind -cu -m filename
```

12.2.2 Size of the Calendar

Normally, if standard output is connected to a terminal, Remind adjusts the width of the calendar to approximately fill the terminal. The `-w` option lets you control the width of the calendar as well as some other dimensions. The full syntax of the `-w` option is:

```
-wcol[,pad[,spc[,spc2]]]
```

(Again, the square brackets denote optional arguments. The commas, however are literal.)

In the above syntax, *col* is a number specifying the width of the calendar in columns. Alternatively, it can be specified as the letter *t* which causes Remind to compute the width according to the size of `/dev/tty`.

pad specifies how many lines to use to pad empty calendar boxes, and it defaults to 5. If the calendar is too big to fit comfortably on the terminal, you can use a smaller *pad*.

spc specifies how many blank lines to leave between the day number and the first reminder entry. It defaults to 1; you can set it to 0 to make the calendar more compact.

spc2 can be specified either as 1 (the default) or 0. If specified as 0, then no blank lines are left between reminders in a given calendar box. If specified as 1, then one blank line is left between such reminders.

Any of *col*, *pad*, *spc* or *spc2* can be omitted. Here are some examples:

```
# Use the default number of columns and padding, but
# leave no blank lines between the day number and the reminder
$ remind -cu -w,,0 pathname

# Make the calendar 80 columns wide; keep boxes as small as
# possible with no blank lines between the day number and the reminder
# or between reminders themselves.
$ remind -cu -w80,0,0,0 pathname
```

12.2.3 Time Format

In Calendar Mode, Remind normally precedes reminders with the time of day. The *-b* option controls how times are formatted:

- *-b0* causes times to be displayed in 12-hour (AM/PM) format, and is the default.
- *-b1* causes times to be displayed in 24-hour format (00:00 to 23:59.)
- *-b2* suppresses the time that normally precedes the body of timed reminders.

12.3 Machine-Readable Calendar Output

Remind's *-p* option produces machine-readable output that helper programs can use to generate calendars. The *-p* options has three variants:

- *-p* produces machine-readable output, but in an ad-hoc format. This format is relatively inflexible, but is the oldest format.
- *-pp* produces a slightly more flexible format using JSON to represent calendar entries.
- *-ppp* produces pure JSON output and is the most flexible format.

All of the *-p*, *-pp* and *-ppp* formats are described in the **rem2ps(1)** man page.

Any of the *-p...* options may be followed by a plus sign, *+*, in which case data for a weekly calendar is output rather than a monthly calendar. The *+* sign also automatically forces the output format to pure JSON (the same as *-ppp*).

Finally, the option may be followed by a number *n* (with no intervening whitespace.) This produces calendar data for *n* months (or weeks if *+* was used.)

12.4 PDF Calendars

Remind can't create PDF calendars directly, but a helper program called `rem2pdf` can take the output of `remind -ppp` and convert it into a PDF calendar.

You can create a PDF calendar for January 2027 something like this:

```
$ remind -ppp pathname January 2027 | rem2pdf > jan.pdf
```

Rem2pdf has *many* command-line options; I will not document them all here. See the **rem2pdf(1)** man page for details, or run `rem2pdf --help` for a synopsis. However, below are some of the more frequently-used command-line options:

- `-l` or `--landscape` – generate a calendar in landscape orientation rather than the default portrait orientation.
- `-e` or `--fill-page` – make the monthly calendar fill the page. Normally, `rem2ps` makes each row slightly smaller than necessary to fill the page, in case one row contains a lot of reminders. This option makes all the rows the same size.
- `-cn` or `--small-calendars=n` – controls the placement of the small previous-month and next-month calendars. The value of *n* can range from 0 to 3, with the following meanings:
 - 0 – do not draw any small calendars. This is the default.
 - 1 – Place the small calendars together at the bottom right if there is room; otherwise, place them at the top left.
 - 2 – Place the small calendars together at the top left if there is room; otherwise, place them at the bottom right.
 - 3 – Place the previous month's calendar at the top left and the next month's at the bottom right if there is room. Otherwise, use the same rules as *n*=1.
- `-y` or `--wrap` – make a monthly calendar fit in at most 5 rows by “wrapping around” the day boxes for the 30th and 31st to the top left if there's not enough room on the bottom right.
- `-x` or `--left-numbers` – put day numbers in the top left corner of each day's calendar box rather than the default top right.
- `-pn` or `--weeks-per-page=n` – if generating weekly calendars, put *n* weeks' worth of data on each page, where *n* can range from 1 to 4.

Here are some examples of generating PDF calendars:

```
# Generate a landscape calendar for January 2027, filling the page
$ remind -ppp ~/.reminders 2027-01-01 | rem2pdf -e -l > /tmp/cal.pdf

# Same as previous example, but with small calendars, day numbers
# on the left, and occupying at most 5 rows
$ remind -ppp ~/.reminders 2027-01-01 | \
  rem2pdf -e -l -c3 -x -y > /tmp/cal.pdf

# A 12-month calendar for 2028 with the first column being Monday
# Note that the -m option is for remind, not rem2pdf. Remind
# communicates to back-end programs that they should start the week
# on Monday.
$ remind -m -ppp12 ~/.reminders 2028-01-01 | \
  rem2pdf -e -l -c3 -x -y > /tmp/cal.pdf

# A calendar for the next 8 weeks, with 4 weeks per page
$ remind -p+8 ~/.reminders | rem2pdf -l -p4 > /tmp/cal.pdf
```

12.4.1 Generating PostScript and SVG Instead of PDF

Although the program is called `rem2pdf`, it can actually generate calendars in other formats:

- `remind options pathname | rem2pdf --ps` will generate PostScript output.
- `remind options pathname | rem2pdf --svg` will generate SVG output.
- `remind options pathname | rem2pdf --eps` will generate encapsulated PostScript output.

Note that for EPS and SVG output, `rem2pdf` will convert only the *first* page of output and will issue a warning if you try to generate a multi-page file. EPS and SVG don't directly support multi-page files.

12.5 HTML Calendars

A program called `rem2html` converts the output of `remind -ppp ...` to HTML, just as `rem2pdf` converts it to PDF.

`Rem2html` accepts a number of command-line options; the most commonly-used ones are:

- `--utf8` – Assume that Remind produces UTF-8 output and write UTF-8-encoded HTML.

- `--stylesheet url.css` – Supply the URL for a CSS stylesheet. If this option is not given, then a default inline stylesheet is used.
- `--nostyle` – Do not use CSS at all to style the output; produce a very basic HTML-only calendar.
- `--tableonly` – Instead of producing a complete HTML document, produce only a `<table>...</table>` block suitable for including into a larger HTML file.

For documentation on the other command-line options, see the **rem2html(1)** man page.

Here are some examples of generating HTML calendars:

```
# Generate calendar for January 2027
$ remind -ppp ~/.reminders 2027-01-01 | rem2html > /tmp/cal.html

# A calendar for the next 8 weeks, with Monday leftmost.
$ remind -p+8 -m ~/.reminders | rem2html > /tmp/cal.html
```

12.6 Restricting Reminders to Calendar Mode or Agenda Mode

Sometimes, you might want a reminder that appears only in Calendar Mode and not in Agenda Mode. Conversely, you might want one in Agenda Mode but not Calendar Mode.

To make a reminder appear in Calendar Mode but not Agenda Mode, simply use the **CAL** keyword instead of **MSG** in the **REM** command:

```
REM Wednesday CAL Only in Calendar Mode!
```

To make a reminder appear in Agenda Mode but not Calendar Mode, use the **MSG** keyword as normal, but use a pair of `%"%"` substitution filter sequences next to one another to make the part that would normally appear in the calendar empty:

```
REM Wednesday MSG %%"Only in Agenda Mode!
```

12.7 Special Data for Calendars

Remind has a facility for passing special types of data to back-end calendar-producing programs. This data can be used for various purposes:

1. To shade a calendar box a certain color.
2. To display a moon-phase icon.

3. To annotate the calendar box with the week number.
4. To display colored reminders.

The way you pass this information to the back-end is with a **SPECIAL** reminder, which looks like this:

```
REM trigger... SPECIAL type body
```

The *trigger* is any normal Remind trigger specification. The *type* specifies the type of special data to send to the back-end, and the *body* is the body of data to send.

Remind does not care what *type* or *body* you supply; it will happily send whatever you put there. Back-ends can define what types of data they accept and the syntax that the body should follow. Any back-end *must* ignore a **SPECIAL** that it doesn't understand.

SPECIAL reminders are ignored in Agenda Mode (with one exception as discussed in [Section 12.7.4 on page 93](#)); they are triggered only in Calendar Mode.

All of the back-ends that ship with Remind respect four standard types of special data: **SHADE**, **MOON**, **WEEK** and **COLOR**. These will be described in the following sections.

12.7.1 SHADE

The **SHADE** special reminder has a body that consists either of a single integer from 0 to 255, or three space-separated integers from 0 to 255.

If the **SHADE** keyword is followed by a single number, then the calendar cell where it triggers is shaded grey, where 0 is completely black and 255 is completely white. If it is followed by three numbers, then they are interpreted as red, green and blue values from 0 to 255, respectively.

Here is a concrete example. Suppose you want to shade a calendar box light green if it is somebody's birthday, and all your birthday reminders are in a file called `birthdays.rem`. You could use this:

```
# The system variable $NumTrig contains the number of REM
# commands that have been triggered so far. Here we save
# the current value
SET n $NumTrig

DO birthdays.rem

# If anything triggered, shade the calendar box
IF $NumTrig > n
    REM SPECIAL SHADE 224 255 224
ENDIF
```

For example, if Jane’s birthday is on February 11, then `rem2pdf` might produce something like Figure 12.1

February 2026						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
1	2	3	4	5	6	7
8	9	10	11 Jane's Birthday	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28

Figure 12.1: A Calendar with a Shaded Box

12.7.2 MOON

The MOON special reminder looks like this:

```
REM trigger SPECIAL MOON phase moonsize fontsize msg
```

In the body of the MOON special reminder, the parts have the following meanings:

- *phase* is an integer from 0 to 3, where 0 represents a new moon, 1 the first quarter, 2 a full moon, and 3 the last quarter.
- *moonsize* is the diameter in 1/72 of an inch of the moon icon. If omitted or supplied as -1, the back-end will pick an appropriate size. Note that *moonsize* is merely a suggestion and some back-ends ignore it.
- *fontsize* is the height in 1/72 of an inch of the font used to draw the message *msg*. If omitted or supplied as -1, the back-end will pick an appropriate size. Note that *fontsize* is merely a suggestion and some back-ends ignore it.
- *msg* is additional text that is placed near the moon glyph.

For example, if I take the following Remind script:

```
REM [moondate(0)] SPECIAL MOON 0 -1 -1 [moontime(0)]
REM [moondate(1)] SPECIAL MOON 1 -1 -1 [moontime(1)]
REM [moondate(2)] SPECIAL MOON 2 -1 -1 [moontime(2)]
REM [moondate(3)] SPECIAL MOON 3 -1 -1 [moontime(3)]
```


And I run the following command with my location and time zone set to Ottawa, Ontario, Canada:

```
$ remind -ppp moon.rem 2027-06-01 | rem2pdf -l > out.pdf
```

Then the result looks like Figure 12.2

June 2027						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
		1	2	3	● 15:41 4	5
6	7	8	9	10	● 06:57 11	12
13	14	15	16	17	○ 20:45 18	19
20	21	22	23	24	25	26
● 00:56 27	28	29	30			

Figure 12.2: A Calendar with Moon Phase Icons

12.7.3 WEEK

The WEEK special looks like this:

```
REM trigger SPECIAL WEEK text
```

The *text* can be anything you like and it is used to annotate the calendar cell in which it appears. For example, if you want every Monday annotated with the week number, you could use this:

```
REM Monday SPECIAL WEEK (W[weekno()])
```

which annotates every Monday with the text (*Wn*) where *n* is the week number.

12.7.4 COLOR

The COLOR¹ special lets you create colored reminders. COLOR is *extra-special* in that COLOR reminders are treated like MSG reminders in that they are *also* issued in Agenda Mode.

A COLOR special reminder looks like this:

¹COLOR can also be spelled COLOUR; the two are treated identically

```
REM trigger SPECIAL COLOR R G B text
```

The *R*, *G* and *B* components are integers ranging from 0 to 255, representing the intensity of the red, green and blue components, respectively, of a color. The text *text* is rendered in the given color.

Consider this reminder script:

```
REM 31 October SPECIAL COLOR 128 82 0 Happy Halloween!!
```

Producing a PDF calendar by running:

```
$ remind -ppp halloween.rem 1 oct 2030 | rem2pdf -l > halloween.pdf
```

will yield the calendar shown in Figure 12.3.

October 2030						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31 Happy Halloween!!		

Figure 12.3: A Colored Reminder

Colored Reminders and Shading in the Terminal

By default, Remind does *not* attempt to color reminders or shade calendar cells in the terminal (that is, if you run Remind with the `-c` option.)

However, you can tell Remind to color reminders (assuming your terminal supports the necessary escape sequences) with the `-@` option. This option works in both Agenda Mode and Calendar Mode, and looks like this:

```
-@[n][,m][,b]
```

The optional parameters *n*, *m* and *b* can take the following values. (You can omit any or all of the parameters; as long as you have the right number of commas, Remind will know if you've omitted a parameter that precedes a non-omitted parameter.)

- *n* can be 0, 1 or 2, with the default being 0. A value of 0 means Remind approximates colors with a very coarse 16-color palette supported by the original VT100 terminal. A value of 1 make Remind approximate the colors with a larger 256-color palettes supported by many terminals. And a value of 2 makes Remind use true 24-bit color escape sequences; this should be supported by most modern terminals.
- *m* can be 0, meaning your terminal background is dark, or 1, meaning it is light. In either case, Remind will adjust the colors in the REM command to try to ensure they are readable. A value of 2 means that Remind will not do any color adjustments. Finally, you can use a value of *t* (which is the default). In this case, Remind attempts to obtain the background color of your terminal with a special escape sequence.
- *b* can be 0 (the default) meaning that Remind will ignore SHADE specials in the terminal, or 1, meaning it will attempt to shade calendar cells. If you enable shading, then you must use either -@1 or -@2 (the 256-color or the 24-bit true color modes.)

Putting everything together, consider this reminder script:

```
# Shade weekends blue
REM Sat Sun SPECIAL SHADE 192 192 255

# Show moon phases
REM [moondate(0)] SPECIAL MOON 0 -1 -1 [moontime(0)]
REM [moondate(1)] SPECIAL MOON 1 -1 -1 [moontime(1)]
REM [moondate(2)] SPECIAL MOON 2 -1 -1 [moontime(2)]
REM [moondate(3)] SPECIAL MOON 3 -1 -1 [moontime(3)]

# Show week number on Mondays
REM Monday SPECIAL WEEK (W[weekno()])

# A happy red reminder on Fridays
REM Friday SPECIAL COLOR 192 64 64 Happy Friday!!
```

If we generate a calendar in the terminal with the command:

```
$ remind -cu -@2,,1 -w90,2,0 everything.rem 1 apr 2030
```

then we get output similar to Figure [12.4 on the next page](#):²

²The moon phase icons only work if your terminal supports UTF-8 and the font it uses has the moon phase icons.

April 2030						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
	1 (W14)	2 ● 18:04	3	4	5 Happy Friday!!	6
7	8 (W15)	9	10 ● 22:58	11	12 Happy Friday!!	13
14	15 (W16)	16	17 ● 23:21	18	19 Happy Friday!!	20
21	22 (W17)	23	24 ● 14:40	25	26 Happy Friday!!	27
28	29 (W18)	30				

Figure 12.4: Example of SPECIALs in the Terminal

On the other hand, if we run that script through `rem2pdf` to produce a PDF calendar like this:

```
$ remind -ppp everything.rem 1 apr 2030 | rem2pdf -l > everything.pdf
```

we get something like Figure 12.5:

April 2030						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
	1 ● 18:04 (W14)	2	3	4	5 Happy Friday!!	6
7	8 (W15)	9 ● 22:58	10	11	12 Happy Friday!!	13
14	15 (W16)	16 ○ 23:21	17	18	19 Happy Friday!!	20
21	22 (W17)	23 ● 14:40	24	25	26 Happy Friday!!	27
28	29 (W18)	30				

Figure 12.5: Example of SPECIALs rendered as PDF

And finally, if we run it through `rem2html` like this:

```
$ remind -ppp everything.rem 1 apr 2030 | rem2html > everything.html
```

the result is something like Figure 12.6:

April 2030						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
March S M T W T F S 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31	1 (W14)	● 18:04	2	3	4 Happy Friday!!	5
	8 (W15)	● 22:58	10	11 Happy Friday!!	12	13
	15 (W16)	○ 23:21	17	18 Happy Friday!!	19	20
	22 (W17)	● 14:40	24	25 Happy Friday!!	26	27
	29 (W18)	30				May S M T W T F S 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

Figure 12.6: Example of SPECIALs rendered as HTML

You might notice that the shading in Figures 12.5 on the facing page and 12.6 is a bit off... the weekends are shaded too darkly and the red Friday reminders are a bit too light. That's because I keep my terminal background dark, so colors that work well on a dark background don't necessarily work well on a light background.

We can fix this by testing the `$PsCal`³ special variable. This variable is set to 1 if the `-p` command-line option (or its variant) was supplied, or 0 otherwise. So we change the script as follows (changes are highlighted):

```
IF $PsCal
```

```
  SET blue "224 224 255"
```

```
  SET red "64 0 0"
```

```
ELSE
```

```
  SET blue "32 32 64"
```

```
  SET red "192 64 64"
```

```
ENDIF
```

```
# Shade weekends blue
```

```
REM Sat Sun SPECIAL SHADE [blue]
```

```
# Show moon phases
```

```
REM [moondate(0)] SPECIAL MOON 0 -1 -1 [moontime(0)]
```

³`$PsCal` is so named as a relic from the days when the only back-end shipped with Remind produced a PostScript calendar

```

REM [moondate(1)] SPECIAL MOON 1 -1 -1 [moontime(1)]
REM [moondate(2)] SPECIAL MOON 2 -1 -1 [moontime(2)]
REM [moondate(3)] SPECIAL MOON 3 -1 -1 [moontime(3)]

# Show week number on Mondays
REM Monday SPECIAL WEEK (W[weekno()])

# A happy red reminder on Fridays
REM Friday SPECIAL COLOR [red] Happy Friday!!

```

With this script, the terminal output is unchanged while the PDF output looks a lot more reasonable (Figure 12.7):

April 2030

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
	1 (W14)	2 ● 18:04	3	4	5 Happy Friday!!	6
7	8 (W15)	9 ● 22:58	10	11	12 Happy Friday!!	13
14	15 (W16)	16 ○ 23:21	17	18	19 Happy Friday!!	20
21	22 (W17)	23 ● 14:40	24	25	26 Happy Friday!!	27
28	29 (W18)	30				

Figure 12.7: Improving the Colors in a PDF Calendar

12.7.5 Coloring Reminders by Default

Remind has a system variable called `$DefaultColor` that lets you color MSG-type reminders by default. `$DefaultColor` should be set to a STRING taking the form of three space-separated integers ranging from 0 to 255 representing the red, green and blue components of the color. It can also be set to "-1 -1 -1" which is a special value that turns off coloring of MSG-type reminders (and is the default value.)

Here's how you might use `$DefaultColor`. Suppose you have work reminders and birthdays in separate files, and you want to color birthdays blue and work reminders red. You could use something like this:

```

# Save $DefaultColor
PUSH-VARS $DefaultColor

```

```
# Birthdays are blue
SET $DefaultColor "0 0 128"
DO birthdays.rem

# Work reminders are red
SET $DefaultColor "128 0 0"
DO work.rem

# Restore old value of $DefaultColor
POP-VARS
```

As the example above shows, PUSH-VARS and POP-VARS work with writable system variables as well as regular variables.

12.8 Back-End-Specific SPECIALs

While the SPECIAL types discussed earlier are universally supported by all back-ends, there are some back-end-specific SPECIALs.

12.8.1 Rem2PDF SPECIAL: PANGO

rem2pdf supports a special called PANGO. It looks like this:

```
REM trigger PANGO body
```

The *body* is text formatted in the Pango markup language, somewhat similar to HTML markup, described at https://docs.gtk.org/Pango/pango_markup.html.

If the PANGO special starts with @x, y where x and y are floating-point numbers, then the marked-up text is positioned absolutely within the calendar box. x and y are measured in units of 1/72 of an inch.

A positive x positions the left edge of the text x units to the right of the left side of the calendar box while a negative x positions the right edge of the text to the left of the left side of the calendar box.

Similarly, a positive y positions the top edge of the text y units below the top of the calendar box while a negative y positions the bottom edge of the text above the bottom of the calendar box.

Consider the following script *pango.rem*:

```
REM 15 Jan SPECIAL PANGO \
    <span size="xx-large" face="Zapf Chancery"><b>Fancy!</b></span>
REM Wed SPECIAL PANGO @1.5,0.5 TL
REM Wed SPECIAL PANGO @1.5,-3.0 BL
REM Wed SPECIAL PANGO @-1.5,0.5 TR
REM Wed SPECIAL PANGO @-1.5,-3.0 BR
```

If we run this command:

```
$ remind -ppp pango.rem 1 Jan 2028 | rem2pdf -l > fancy.pdf
```

the result looks like Figure 12.8:

January 2028						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

Figure 12.8: The PANGO SPECIAL

You might have to experiment a bit with absolutely-positioned text to get the x and y values just right for your use-case. Also note that `rem2pdf` doesn't take any precautions to prevent absolutely-positioned text from overlapping with day numbers or normal reminders, so use it with care.

12.8.2 Rem2HTML SPECIALs

`rem2html` supports two different SPECIAL types:

- **HTML** – Adds an HTML reminder to the calendar. The body is whatever HTML you want to insert in the calendar box.
- **HTMLCLASS** – Adds a CSS class to the calendar cell.

Here are examples of how you might use them:

```
REM Wednesday AT 11:00 SPECIAL HTML <b>Very important</b> meeting.
REM [realtoday()] SPECIAL HTMLCLASS rem-today
```

The second example adds the CSS class `rem-today` to the calendar box corresponding to the actual system date. The standard CSS that `rem2html` uses defines a `rem-today` class that highlights a calendar cell in red. So the REM command has the effect of highlighting today's date in red.

Chapter 13

The Hebrew Calendar

In addition to support for the Gregorian calendar, Remind has built-in functions to support events based on the Hebrew calendar.

13.1 A Description of the Hebrew Calendar

The Hebrew calendar is a lunisolar calendar (Section [11.3 on page 81](#).) That means that its months are approximately aligned with phases of the Moon, while its years are approximately aligned with the solar year.

A normal Hebrew year has 12 months, alternately 30 and 29 days long. The Hebrew months are:

- Tishrey (תשרי), also spelled Tishri or Tishrei. 30 days long.
- Heshvan (חשוון), also spelled Cheshvan or Kheshvan. 29 (or 30) days long.
- Kislev (כסליו). 30 (or 29) days long.
- Tevet (טבת). 29 days long.
- Shvat (שבט), also spelled Shevat. 30 days long.
- Adar (אדר). 29 days long.
- Nisan (ניסן). 30 days long.
- Iyar (אייר), also spelled Iyyar. 29 days long.
- Sivan (סיון). 30 days long.
- Tamuz (תמוז), also spelled Tammuz. 29 days long.

- Av (אב). 30 days long.
- Elul (אלול). 29 days long.

(These months are given in the order used by the *civil year*. In Biblical times, Nisan was considered the first month and Tishrey was the 7th. This is the *ecclesiastical year*.)

In a cycle of 19 years, there are 7 leap years, being years 3, 6, 8, 11, 14, 17 and 19 of the cycle. In a leap year, an extra 30-day month is added before Adar. This extra 30-day month is called Adar A and the original 29-day Adar is called Adar B. This pattern ensures that a 19-year cycle has 235 months, in keeping with the Metonic Cycle.

Remind lets you use any of the following to name the two Adars in leap years:

- Adar A, Adar 1, Adar I, אָדָר א', אָדָר 1 or אָדָר A
- Adar A, Adar 2, Adar II, אָדָר ב', אָדָר 2 or אָדָר B

The beginning of the year is 1 Tishrey and is referred to as *Rosh Hashanah* or ראש השנה (literally, “head of the year.”)

For certain religious reasons, Rosh Hashanah cannot occur on a Sunday, Wednesday or Friday. To adjust for this, a day is taken off Kislev or added to Heshvan. Thus, a regular year can have from 353 to 355 days, and a leap year can have from 383 to 385 days.

Note: Technically, the Hebrew date changes at sunset. However, Remind changes the date at midnight, not sunset. So in the period between sunset and midnight, Remind’s calculation of the Hebrew date will be a day earlier than the actual Hebrew date. This should not be much of a problem in practice, but you should keep it in mind and adjust if necessary.

13.2 Gregorian-to-Hebrew Conversion

The following functions simply convert a given Gregorian date to Hebrew date components. All of them take a single argument of type DATE, representing the Gregorian date. (Once again, if you are converting a date that is after sunset, you need to add 1 to the date argument to account for the fact that Hebrew dates change at sunset and not midnight.)

- `hebday(date)` – Returns an INT representing the day of the Hebrew month associated with the *date*.
- `hebmon(date)` – Returns a STRING containing the name of the Hebrew month associated with the *date*. This name is the transliterated name in ASCII characters (such as “Nisan” or “Kislev”) If a month has multiple accepted transliterations on input, `hebmon` always returns the “canonical” transliteration, which is the first one.

- `ivritmon(date)`¹ – Similar to `hebmon`, but returns the name of the month in UTF-8 Hebrew characters (such as “ניסן” or “כסלו”). For Adar A and Adar B, `ivritmon` returns ’אדר א’ and ’אדר ב’, respectively.
- `hebyear(date)` – Returns an INT representing the Hebrew year corresponding to *date*. Depending on the time of year, the Hebrew year is greater than the Gregorian year by either 3760 or 3761, with the increment taking place at Rosh Hashanah, which is in September or October.

Here is an example script:

```
REM MSG Today is [hebday($T)] [hebmon($T)] [hebyear($T)]
REM MSG Today is [hebday($T)] [ivritmon($T)] [hebyear($T)]
```

If we run that script on Tuesday, January 20th, 2026, the result is:

```
Today is 2 Shvat 5786
Today is 2 5786 שבט
```

The second example is a bit messed up because it really needs a left-to-right mark to reset the printing direction. You can fix it as follows:

```
# Unicode character 0x200e is the LEFT-TO-RIGHT MARK
SET LRM mbchar(0x200e)
REM MSG Today is [hebday($T)] [ivritmon($T)][LRM] [hebyear($T)]
```

which yields:

```
Today is 2 שבט 5786
```

The Unicode LEFT-TO-RIGHT MARK fixes up the formatting.

13.3 Hebrew-to-Gregorian Conversion

Hebrew-to-Gregorian conversion is handled by a single function called `hebdate`. This function may be called in a number of ways; I’ll document the most common ways here, but consult the **remind(1)** man page for a full description of how you can call `hebdate`.

¹“Ivrit” is the Hebrew word for Hebrew. (Technically עברית is the Hebrew word for Hebrew, but Ivrit is its transliteration.)

13.3.1 Conversion of a Full Date

You can convert a full Hebrew date by calling `hebdate` with 3 parameters:

- `hebdate(day, mon, year)` – In this method of calling `hebdate`, *day* is an INT from 1 to 30 representing the day of the Hebrew month. *mon* is a STRING containing the name of the Hebrew month—either the transliterated version like “Nisan” or the UTF-8-encoded Hebrew version like “ניסן”. And *year* is an INT representing the *Hebrew* year (so a number greater than or equal to 5750).

Here are some examples:

```
SET a hebdate(2, "Shevat", 5786)           Sets a to '2026-01-20'
SET a hebdate(1, "Tishrey", 5787)          Sets a to '2026-09-12'
SET a hebdate(30, "Elul", 5790)            Error: Invalid Hebrew date
```

The final example raises an error because Elul only has 29 days.

13.3.2 Conversion of Partial Dates

Another way to call `hebdate` is as follows:

- `hebdate(day, mon [, start])` – Here, *day* is an INT from 1 to 30 and *mon* is a STRING containing the name of a Hebrew month. *start* is a DATE (and if *start* is omitted, it defaults to `today()`.)

When `hebdate` is called this way, it returns (as a DATE) the Gregorian date corresponding to the first Hebrew date on or after *start* that matches *day* and *mon*.

Here are some examples of using `hebdate` to convert partial dates:

```
REM [hebdate(1, "Tishrey")] ++7 MSG "%Rosh Hashanah%" is %b.
REM [hebdate(14, "Adar A")] ++7 MSG "%Purim Katan%" is %b.
REM [hebdate(25, "Kislev")] ++7 MSG "%Chanukah 1%" is %b.
```

If, on 2026-01-21, we run the above script through `remind -n`, the output is:

```
2026/09/12 Rosh Hashanah is in 234 days' time.
2027/02/21 Purim Katan is in 396 days' time.
2026/12/05 Chanukah 1 is in 318 days' time.
```

Note that Jewish holidays begin at sunset on the day *before* the Gregorian dates returned by `hebdate()`. Thus (for example) Rosh Hashanah in 2026 actually begins at sunset on 2026-09-11.

The `hebdate()` function has other optional parameters that control its behavior in leap years and in years when 30 Heshvan or 30 Kislev do not exist, but a partial date of 30 Heshvan or 30 Kislev was supplied. For details, see the **remind(1)** man page.

13.4 A Silly Example

From the point of view of the Gregorian calendar, the date of Chanukah jumps around all over the place, while Christmas is fixed. On the other hand, from the point of view of the Hebrew calendar, Chanukah is fixed while Christmas jumps around. Do the two holidays ever coincide? We can find out with this script:

```
REM 25 December SATISFY [hebday($T) == 25 && hebmon($T) == "Kislev"] \
MSG Chanukah
```

Running this in next mode (remind -n) in February 2026, yields:

```
2027/12/25 Chanukah
```

Technically, Chanukah begins at sundown on 2027-12-24, but the first day of Chanukah is still considered to be 2027-12-25.

The holidays don't coincide very often. The next time it happens after 2027 is on 2073-12-25.

If we want to find out when the first day of Passover and Easter Sunday coincide, we can use this script:

```
SET $MaxSatIter 100000
REM SATISFY \
  [$T == easterdate($T) && hebday($T) == 15 && hebmon($T) == "Nisan"] \
MSG Passover/Easter
```

Note that because it is *extremely* rare for Easter and Passover to coincide, I had to increase \$MaxSatIter to 100,000. The next time the holidays coincide is 2123-04-11.

Chapter 14

The TkRemind Graphical Front-End

Although Remind is a command-line tool and all the examples so far have shown its use on the command-line, Remind also ships with a program called `tkremind`. This is a graphical front-end for Remind; it displays your calendar as a window and it lets you create events graphically.

If you are going to use `tkremind`, I *strongly* recommend that you create a directory called `.reminders` in your home directory and store your reminders in `*.rem` files in that directory. That is the best setup for using `tkremind` and I'll assume you've gone and done that. If you don't want to do this, see the **`tkremind(1)`** man page for other ways to run `tkremind`.

To start TkRemind, simply type the following command:

```
$ tkremind
```

The main TkRemind window will appear (Figure [14.1 on the next page](#)):

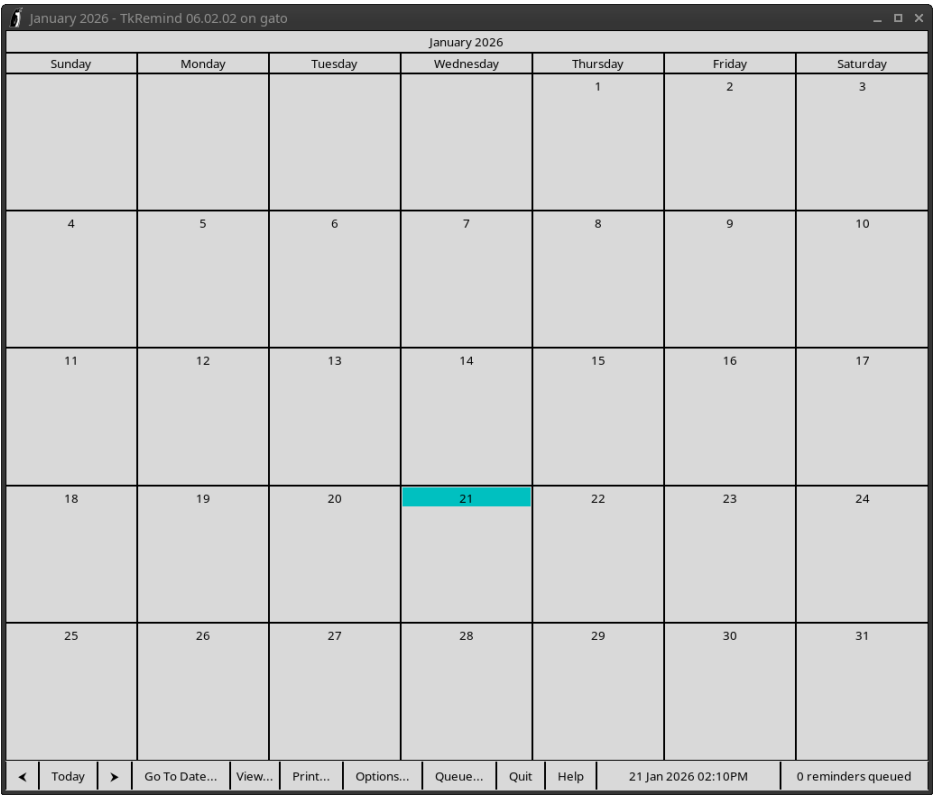


Figure 14.1: The Main TkRemind Window

In addition, another window with today’s reminders in Agenda Mode will pop up (Figure 14.2):



Figure 14.2: The Agenda Mode Window

14.1 Main Window

The main window displays a calendar. The default view is one month, but this can be changed to display one week, two weeks or four weeks instead via the “View...” menu.

The “<” and “>” buttons move the view backwards or forwards, respectively. If the view is set to one month, then the buttons move the view by one month backwards or forwards. If the view is set to any of the weekly views, then the view is moved by *one week* each time the button is pressed.

The “Today” button changes the view to the current month (if the view is set to one month) or positions the current date in the top row of the view if it is set to one of the weekly views.

The “Go To Date...” button pops up a small dialog (Figure 14.3) that lets you enter a month and year. The view is then positioned to that month and year. (In one of the weekly views, the view is positioned such that the first day of the specified month appears in the top row.)

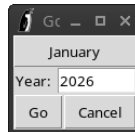


Figure 14.3: The Go To Date Dialog

14.1.1 Keyboard Shortcuts

The following keyboard shortcuts are available in the main window:

- **Left Arrow, Page Up** – Move the view backwards.
- **Right Arrow, Page Down** – Move the view forwards.
- **Home** – Move the view to the current date.
- **m** – Set the view to one month.
- **1** – Set the view to one week.
- **2** – Set the view to two weeks.
- **4** – Set the view to four weeks.
- **h, ?, F1** – Open Help window.
- **g** – Go to a specific date
- **o** – Open the “Options...” dialog.
- **p, Control-p** – Open the “Print...” dialog.

- **t** – Open an “Agenda Mode” window with today’s reminders.
- **Control-q** – Quit.

14.2 Queued Reminders

In addition to showing a calendar, TkRemind also runs a Remind process in a special mode called *Server Mode*. This handles queued reminders and allows TkRemind to pop up a window whenever a queued reminder is triggered.

For details about Server Mode, please see the **tkremind(1)** man page.

14.3 Printing The Calendar

The “Print...” button brings up the Print Dialog (Figure 14.4).

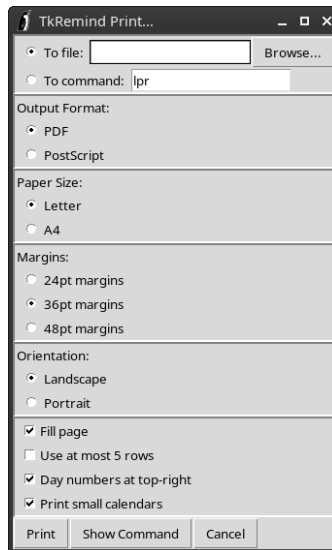


Figure 14.4: The Print Dialog

Note that TkRemind will print whatever matches the main window view; if you are viewing a monthly calendar, that’s what will be printed. If you are viewing a 4-week calendar, then a 4-week calendar will be printed.

The Print Dialog has six sections, as follows:

1. In the top section, choose to print either to a file or to a command. The first option lets you save the generated PDF or PostScript, and the second option lets you send it to a printer.

2. The Output Format section lets you choose whether to generate PDF or PostScript output.
3. The Paper Size section lets you select Letter size or A4 size paper.
4. The Margins section lets you choose how wide the margins are around the calendar.
5. The Orientation section lets you pick Landscape or Portrait orientation.
6. The final Options section has the following options:
 - Fill page – if you enable this, then the calendar will fill the page and all rows will be the same height. If you do not enable it, then some rows may be smaller than others, to leave room for calendar boxes with many reminders.
 - Use at most 5 rows – if you enable this, then a monthly calendar that might normally use 6 rows will have the last few days “wrapped around” to the top left so only 5 rows are used.
 - Day numbers at top-right – if you enable this, day numbers will be placed at the top right of each calendar cell. Otherwise, they will be placed at the top left
 - Print small calendars – if you enable this, then small calendars for the previous and next month will be included in a monthly calendar. This option has no effect for weekly calendars.

Once you’ve filled in the Print Dialog, you can click “Print” to print the calendar or “Cancel” to cancel. The “Show Command” option gives you a preview of the command that will be executed to produce the calendar. You can edit the command (to enable command-line options not settable via the GUI, for example) and then run the modified command.

14.4 Options

The “Options...” button pops up the Options Dialog ([Figure 14.5 on the next page](#)).

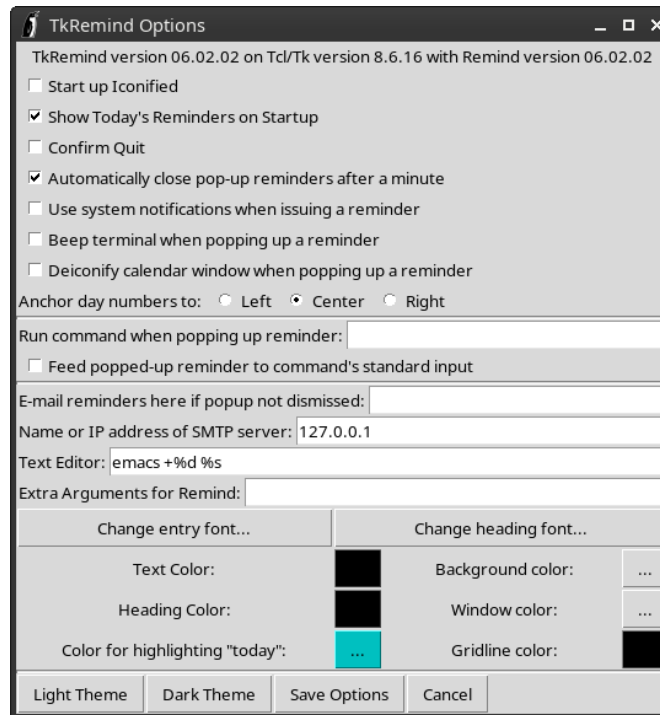


Figure 14.5: The Options Dialog

14.4.1 Basic Options

- If the “Start up Iconified” option is enabled, then TkRemind will start up in an iconified state rather than showing the main window upon startup.
- If “Show Today’s Reminders on Startup” is enabled, then TkRemind will open up an Agenda Mode window with today’s reminders upon startup. If not, then the Agenda Mode window will not automatically open on startup.
- If “Confirm Quit” is enabled, then if you click “Quit” or press `Control-q`, you will be asked to confirm. If not, then TkRemind will simply immediately quit.
- If “Automatically close pop-up reminders after a minute” is enabled, then TkRemind will automatically close the pop-up reminder windows that it creates for queued reminders, if they are not manually closed.
- If “Use system notifications when issuing a reminder” is enabled, then TkRemind will try to use your desktop environment’s system notification facility when queued reminders are issued. For this to work, you must either be running Tcl/Tk 9.0.0 or later, or you must have the `notify-send` program installed and on your `$PATH`.

- If “Beep terminal when popping up a reminder” is enabled, then TkRemind will attempt to make the computer beep when popping up a reminder. Note that modern computer hardware typically lacks the little speaker needed for the beep, so this option is unlikely to be very useful.
- If “Deiconify calendar window when popping up a reminder” is enabled, then TkRemind will open and show the main window whenever a queued reminder is popped up.
- You can choose the alignment of the day numbers in the calendar boxes with the “Anchor day numbers to:” option.

14.4.2 Running a Command When Popping Up a Queued Reminder

If you enter a command in the “Run command when popping up reminder:” box, then that command will be executed when TkRemind pops up a queued reminder. If, in addition, you enable “Feed popped-up reminder to command’s standard input”, then TkRemind will send the queued reminder to the standard input of the command, in the format:

time: *body*

Where *time* is the AT time of the reminder, and *body* is the body.

14.4.3 Miscellaneous Options

If you would like a reminder emailed to you if you don’t dismiss the popup reminder, enter the email address in the “E-mail reminders here if popup not dismissed” box. You’ll also need to enter a hostname or IP address of an SMTP server that must accept the email without authentication.

You can enter your favorite text editor command in the “Text Editor:” box. We’ll see later that TkRemind invokes the text editor in response to certain actions. In the command, put %d where you want to have a line number and %s where the name of the file being edited goes. That way, the text editor is opened on the correct file and positioned to a specific line number.

If your editor is Emacs (which it should be!) a good entry for the text editor is:

`emacs +%d %s`

If you use vim, then a good choice is:

`vim +%d %s`

For gedit, use:

`gedit +%d %s`

The “Extra Arguments for Remind” is a catch-all for any extra command-line options you want to pass when TkRemind invokes Remind. I do not recommend putting anything in this box apart from possibly an `-ivar=value` command to initialize a variable; some command-line options will interfere with the correct operation of TkRemind.

14.4.4 Appearance

The final set of options let you change the font for calendar entries, the font used for day number headings, and various colors. Click on a color swatch to open a color picker. As a convenience, the “Light Theme” button presets all colors to the default set of colors, and “Dark Theme” presets them all to a dark theme. Press “Save Options” to save the options or “Cancel” to cancel.

Figure [14.6 on the facing page](#) is a screenshot of the TkRemind main window in 4-week view mode using the dark theme, a few font changes, some realistic reminders, and the MOON and WEEK specials.

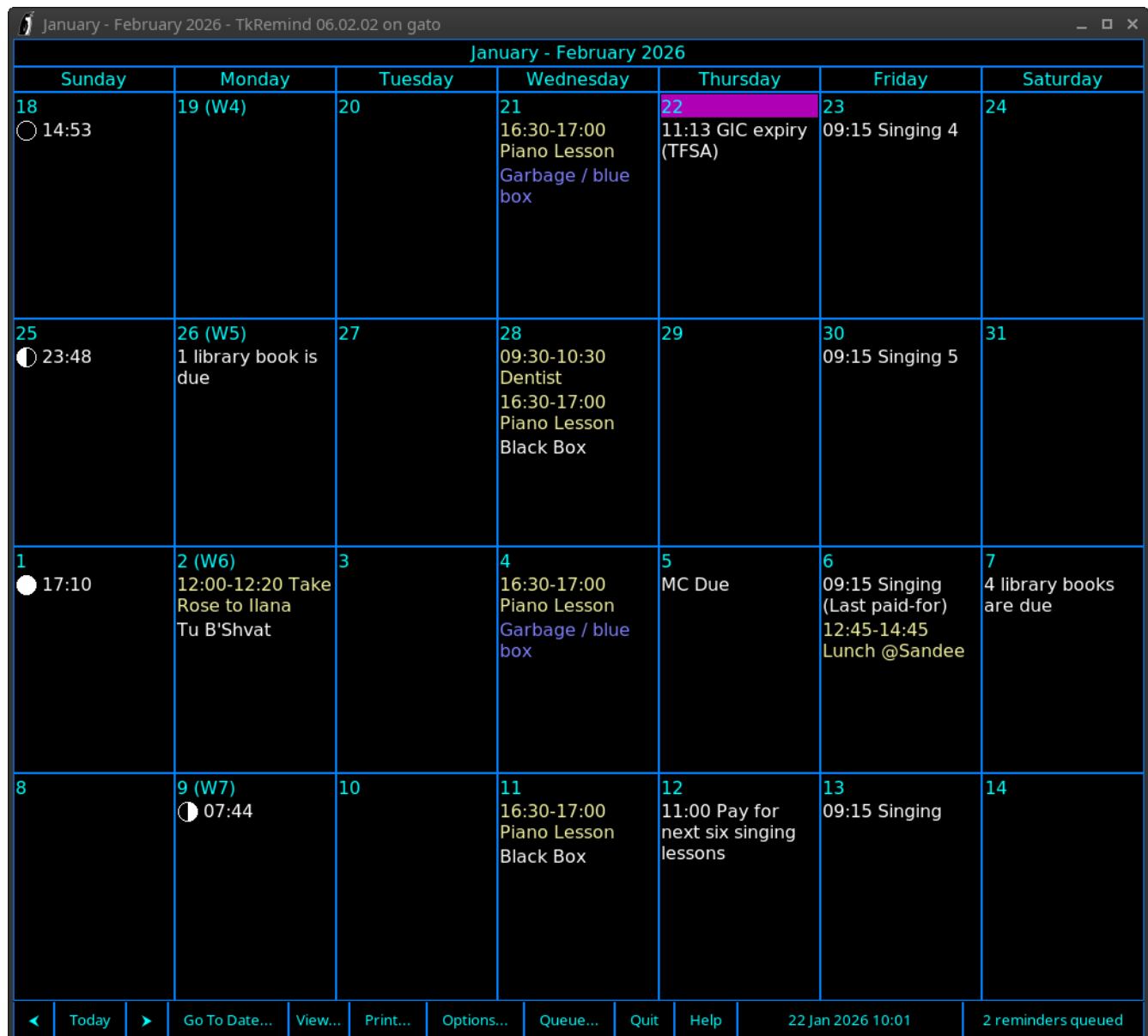


Figure 14.6: Example of the Dark Theme

14.5 Adding Reminders

To add a reminder using TkRemind, click on the day number of the calendar box to which the reminder should be added (or any one of the boxes if it is a recurring reminder.) The Add Reminder dialog (Figure 14.7 on the next page) appears:

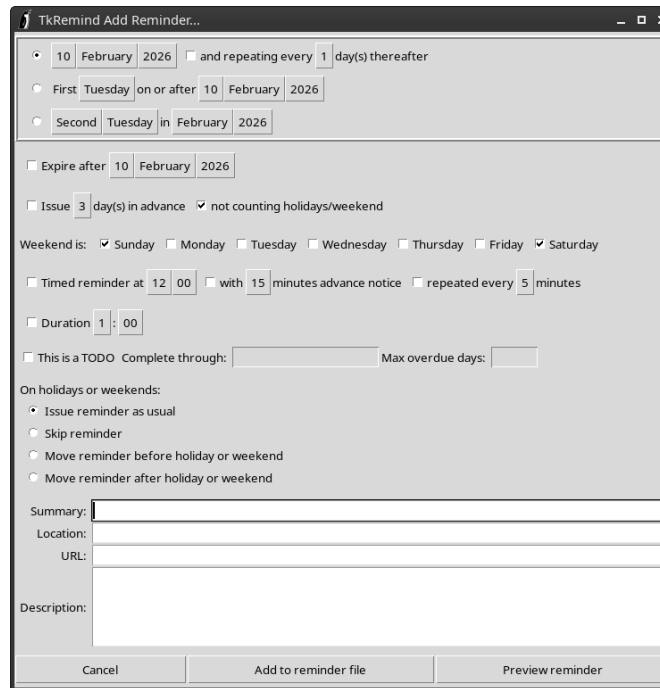


Figure 14.7: The Add Reminder Dialog

14.5.1 Basic Recurrences

The top section of the dialog lets you pick one of three basic types of reminders. Pick the one that is the most appropriate for your situation. The raised buttons pop up menus that let you pick specific choices for the parts of the trigger.

14.5.2 Options

- If you want the reminder to expire after a certain date, enable the “Expire after” checkbox and set the expiry date appropriately.
- If you want advance warning, enable the “Issue ... day(s) in advance” checkbox and set the amount of warning that you want. You can choose whether or not holidays and weekends are counted in the amount of advance warning.
- Set the days that constitute the weekend by checking the appropriate checkboxes to the right of “Weekend is:”. The default of Saturday and Sunday is probably correct.
- If you want to create a timed reminder, enable the “Timed reminder at” checkbox and set the trigger time. You can also enable advance warning with the next two checkboxes.

- If you want your reminder to have a duration, enable the “Duration” checkbox and set the duration appropriately.
- If you are creating a TODO-style reminder, enable the “This is a TODO” checkbox. Optionally fill in the “Complete through:” with a date of the form YYYY-MM-DD and optionally set “Max overdue days:” to a positive integer.
- The “On holidays or weekends:” let you pick what happens if a reminder falls on a holiday or weekend. The options correspond to issuing the reminder as usual, using the SKIP keyword, the BEFORE keyword or the AFTER keyword.

14.5.3 The Reminder Body

Fill the body of the reminder into the “Summary:” box. The corresponds to what comes after the MSG keyword in the REM command.

The “Location:”, “URL:” and “Description” fields are optional, but let you assign a location, a link and a longer description to the reminder. We’ll see in Chapter 15 how this additional information is stored in the REM command.

To add the reminder, click “Add to reminder file”. If you are running TkRemind with the default options and have created a `$HOME/.reminders` directory, then the new REM command is added to the file `$HOME/.reminders/100-tkremind.rem`.

You can also preview the REM command that will be added by clicking “Preview Reminder”.

14.6 Editing Reminders

If you add a reminder using the TkRemind graphical reminder dialog (Figure 14.7 on the facing page), then when you hover over the reminder in the main window, it will turn red. Click the left mouse button on the reminder to open the Edit Reminder dialog. This looks exactly like the Add Reminder dialog, except it has an additional option, “Delete Reminder”, that lets you delete the reminder.

You can adjust the dialog to edit the reminder and click “Replace Reminder” to update it.

If there are hand-crafted reminders, you cannot edit them in the TkRemind Edit Reminder dialog. Instead, when you hover over them, they will be underlined. Click the left mouse button to open up a text editor positioned to the relevant REM command. You can also use the right mouse button to edit a reminder with a text editor—even a reminder created with TkRemind’s Add Reminder dialog.

NOTE: If you use the “Preview Reminder” option in the Add Reminder dialog and then you *change* the REM command before saving it, you will *no longer* be able to edit it with TkRemind’s Edit Reminder dialog. Instead, you have to edit it by hand in a text editor.

14.7 URLs, Location and Description

If you have associated a URL with a reminder, then middle-clicking over the reminder in the calendar opens the URL in a Web browser. If you've associated a location or description with a reminder, then hovering over the reminder pops up a box containing the location and description information.

14.8 The Queue

Clicking on "Queue..." opens a dialog showing the reminders that are queued for today. You can left-click on any queued reminder to open its corresponding REM command in a text editor.

14.9 Errors

If there are errors in your reminder file, then the "Queue..." button will turn red and change to "Errors..." Click on "Errors..." to open up a dialog showing the errors. Click on a specific error message to open a text editor positioned to the offending command.

14.10 Automatically Adjusting to Changes

If you are running Remind on a Linux system with the `inotify` facility enabled, then TkRemind monitors the `$HOME/.reminders` directory for changes and automatically refreshes the calendar view (and updates the queued reminders, if necessary) when you update a file in that directory.

14.11 Variables Set by TkRemind

When TkRemind invokes Remind, it sets the variable `tkremind` to 1. When it invokes Remind to generate a printed calendar, it sets both `tkremind` and `tkprint` to 1. Therefore, in your Remind script, you can detect that it's being invoked from TkRemind as follows:

```
IF catch(tkremind, 0)
    IF catch(tkprint, 0)
        # Calendar is being printed from TkRemind
    ELSE
        # Calendar is being drawn in TkRemind window
    ENDIF
ELSE
    # Not in TkRemind
ENDIF
```

14.12 Getting Help

The “Help” button will pop up a window displaying the **tkremind(1)** man page.

Chapter 15

Tags and Ancillary Information

Remind has two keywords for associating additional information with a reminder. The `TAG` keyword is used to tag a `REM` statement with a label. This label can be used by back-end programs to identify a specific `REM` command. The `TkRemind` program, for example, tags all of the reminders it creates with a tag of the form `TKTAG $n$` , where n is a number. This lets it find the correct command to edit when you click on a reminder to edit it.

The `INFO` keyword, on the other hand, lets you associate arbitrary key/value pairs with a `REM` command. `INFO` is used to associate a URL, a location, a long description, or whatever else you like with a `REM` command.

15.1 TAG

To add a tag to a reminder, simply use the `TAG` keyword followed by a tag name. A tag can be any sequence of non-space characters that does not contain a comma. You can use multiple `TAG` keywords in a `REM` command. The syntax is as follows:

```
REM trigger TAG tag MSG body
```

When you run `Remind` with one of the `-p` options, all of the tags are passed along to the back-end program which can do what it wants with them. `Remind` itself assigns no special meaning to tags.

15.2 INFO

The `INFO` keyword lets you add key/value pairs to a reminder. The syntax looks like this:

```
REM trigger INFO "Hdr: value" MSG body
```

The parameter after the `INFO` keyword is a double-quoted string consisting of a header *Hdr*, a colon, and a value *value*.

The header can be any sequence of non-blank characters (other than a colon) and the value can be anything you like. You can also have as many `INFO` clauses as you want in a `REM` statement, but the headers cannot be duplicated. Header names are not case-sensitive.

Note: To pass along `INFO` headers to a back-end, you must use the `-pp` or `-ppp` `Remind` command-line option. The plain `-p` option won't work.

Here is an example of how you might use `INFO`:

```
REM 12 Feb 2026 AT 15:00 \
    INFO "Url: https://example.com/videoconference/?id=123954" \
    INFO "Description: Discuss upcoming budget" \
    INFO "Location: Online" MSG Meeting with Dan
```

The previous example simply puts “Meeting with Dan” in the calendar, but the `TkRemind`, `rem2pdf` and `rem2html` back-ends also turn it into a clickable link. And `TkRemind` and `rem2html` will pop up the Description and Location information if you hover over the reminder.

You can use whatever keywords you like for the header, but the back-ends that ship with `Remind` recognize “Url:”, “Description:” and “Location:” specially.

15.2.1 INFO-related Substitution

In the body of a reminder, the substitution sequence `%<foo>` is replaced with the `INFO` string with header *foo*: if it exists, or the empty value if not. Here's an example:

```
REM 12 Mar 2026 AT 9:00 \
    INFO "Location: 235 Main Street" \
    MSG Meeting at %<Location>
```

On 12 March 2026, the output will be:

```
Meeting at 235 Main Street
```

15.2.2 URLs in the Terminal

If a reminder has a “Url:” `INFO` header, then in Agenda Mode or in Calendar Mode in the terminal, `Remind` will use the [OSC-8 specification](https://github.com/Alhadis/OSC8-Adoption/?tab=readme-ov-file) to turn the body of the reminder into a hyperlink. Not all terminals support this; a partial list of terminals that do support it is at <https://github.com/Alhadis/OSC8-Adoption/?tab=readme-ov-file>.

If the display of reminders with “Url:” headers is messed up in your terminal, you can disable the OSC-8 escape sequences by setting this system variable in your reminder script:

```
SET $TerminalHyperlinks 0
```

By default, `$TerminalHyperlinks` is set to 1, which enables the terminal-mode hyperlinking.

Chapter 16

Time Zones

So far, all the reminders and date-manipulation functions you've seen have assumed that the date and time is in the *local time zone*—that is, the time zone that is configured as your computer's time zone.

Remind has facilities for converting dates and times between different time zones and also for specifying reminders in different time zones. Note that all time zone conversions are done by your system's C library, so the accuracy of conversions depends on how good your C library's time zone implementation is.

16.1 Reminders in a Specific Time Zone

If you want to issue a reminder based on a date a time in a time zone other than your computer's default time zone, use the TZ keyword in the REM command. The syntax looks like this:

```
REM trigger TZ timezone MSG body
```

The format of *timezone* is system-specific, but on most UNIX systems, it will be something like America/Toronto or Australia/Sydney. Note that time zone names *are* case-sensitive.

On most UNIX-like systems, time zone data is stored under the directory /usr/share/zoneinfo and the name of a time zone is simply the relative path to the time zone information under that directory.

If your system contains the file /usr/share/zoneinfo/zonenow.tab, then you can get a list of some of the valid time zone names supported by your system with this shell command:

```
$ awk 'print $3' /usr/share/zoneinfo/zonenow.tab | grep / | sort
```

Note: If you use the TZ keyword, then you *must also* use the AT keyword to specify a time. Non-timed reminders do not permit a time zone specification because it's impossible to convert just a date from one time zone to another; you need a time as well.

As an example, suppose I have a friend in Sydney, Australia, whom I want to call at 22:00 Sydney time every Thursday. Consider this reminder script:

```
REM Thursday AT 22:00 TZ Australia/Sydney MSG Call Sue
```

My time zone is America/Toronto, so if I run the above script as follows:

```
$ remind -s2 aus.rem 1 mar 2026
```

The output is:

```
2026/03/05 * * * 360 6:00am Call Sue
2026/03/12 * * * 420 7:00am Call Sue
2026/03/19 * * * 420 7:00am Call Sue
2026/03/26 * * * 420 7:00am Call Sue
2026/04/02 * * * 420 7:00am Call Sue
2026/04/09 * * * 480 8:00am Call Sue
2026/04/16 * * * 480 8:00am Call Sue
2026/04/23 * * * 480 8:00am Call Sue
2026/04/30 * * * 480 8:00am Call Sue
```

A few things to note:

- On 2026-03-05, Sydney is 16 hours ahead of America/Toronto and the reminder is issued for 06:00 in my time zone.
- The reminders for 2026-03-12 through 2026-04-02 take place when America/Toronto has started Daylight Saving Time, and Sydney is also still on Daylight Saving Time. In this situation, Sydney is 15 hours ahead of America/Toronto and so the reminder is issued at 07:00.
- The reminders for 2026-04-09 through 2026-04-30 take place after Sydney has switched back to standard time. It is therefore 14 hours ahead of America/Toronto and the reminder is issued at 08:00.

16.1.1 How TZ works

When Remind evaluates a REM command with a TZ clause, it effectively performs the following actions:

- It changes the system time zone to the specified time zone. It also adjusts the effective date and time to be in the new time zone.
- It evaluates the REM command as usual. Note that any SATISFY expressions are evaluated in the time zone specified by TZ
- Once a trigger date and time have been calculated, they are converted back to the local date and time and the system time zone is switched back to the local time zone.

An example is probably best to illustrate this. Suppose I want a reminder at 01:00 every Friday the 13th in the Universal Time Zone. I could use this:

```
REM 13 TZ Universal AT 1:00 SATISFY [$Tw == 5] MSG Booo!
```

I live in the America/Toronto time zone, which as of February 2026 is 5 hours behind Universal Time. If I run the above script as follows:

```
$ remind -n boo.rem 1 Feb 2026
```

The output is:

```
2026/02/12 8:00pm Booo!
```

Note that the final calculated trigger date and time is 2026-02-12 at 20:00 (in the America/Toronto time zone.) This is a Thursday evening. But in the Universal time zone, it works out to 2026-03-13 at 01:00, just what we wanted. When the SATISFY expression is evaluated, it sees the date *in the Universal time zone*. Conversion back to the local time zone is done only *after* the final trigger date/time have been calculated.

16.2 Time Zone Conversion Functions

Remind has a few built-in functions for converting between time zones:

- `utctolocal(dt)` – Given a DATETIME object *dt* interpreted in UTC (Coordinated Universal Time), return a DATETIME object representing the same time, but in the local time zone.
- `localtoutc(dt)` – Given a DATETIME object *dt* interpreted in the local time zone, return a DATETIME object representing the same time, but in UTC.
- `tzconvert(dt, srczone [, dstzone])` – Given a DATETIME object *dt* and a STRING *srczone*, convert *dt* from the time zone named by *srczone* to the time zone named by the STRING *dstzone*. If *srczone* is the empty string, the local time zone is used. If *dstzone* is omitted or is the empty string, the local time zone is used.

Here are some examples of the functions in action. The following examples assume that the local time zone is “America/Toronto”.

```
SET a localtoutc('2025-01-20@14:44')           (a is set to '2025-01-20@19:44')
SET a utctolocal('2031-01-01@00:00')           (a is set to '2030-12-31@19:00')

# a is set to 2027-03-04@05:00
SET a tzconvert('2027-03-04@09:30', "America/St_Johns", "America/Los_Angeles")
```

As the last example shows, “America/St_Johns” is 4 hours and 30 minutes ahead of “America/Los_Angeles”

16.3 Validating Time Zone Names

Unfortunately, the standard Linux library functions for manipulating time zones have no way to validate that a time zone name is actually correct. If you supply an incorrect time zone name, Remind will give undefined results.

Remind does try to validate time zone names by looking at files under `/usr/share/zoneinfo` and will issue a warning if the specified time zone doesn't appear to exist

However, there are ways to specify time zones other than by using file names under `/usr/share/zoneinfo`. For example, under Linux you can use any of the formats documented in [the glibc manual](#).

If you happen to use a time zone name that you know is correct, but Remind issues a warning, simply prefix the time zone name with a `!` to suppress the warning.

For example: Under Linux, you can specify time zones that are n hours West of UTC as `UTC+ $n$` and ones that are n hours East of UTC as `UTC- $n$` .¹

If you do something like this:

```
# a is set to 2026-06-01@17:00 but with warnings
set a tzconvert('2026-06-01@10:00', "UTC+6", "UTC-1")
```

Remind will issue warnings about the time zone names. To suppress the warnings, use:

```
# a is set to 2026-06-01@17:00
set a tzconvert('2026-06-01@10:00', " !UTC+6", " !UTC-1")
```

¹I know those signs relative to UTC look backwards, but that's the specification.

Chapter 17

Remind and Shell Commands

In addition to MSG- and CAL- type REM commands, Remind also supports a RUN-type command that looks like this:

```
REM trigger RUN body
```

In this case, rather than printing out the *body* of the reminder, Remind executes it as a shell command, after running it through the substitution filter. For example, if you have a script called `switch-light` that takes an argument of `on` or `off` to turn on or off a smart light, you can have the light come on an hour after sunset and go off 30 minutes before sunrise with this script:

```
REM AT [sunrise()-30] RUN switch-light off
REM AT [sunset()+60]  RUN switch-light on
```

Note that Remind passes a RUN-type reminder's body through the substitution filter (Section [2.6 on page 15](#)) before executing the result as a shell command.

17.1 Daemon Mode

To have the light-switch script work properly, Remind should be started on the computer at boot time and be left running indefinitely. To make this happen, use the `-z` command-line option, which puts Remind in *Daemon Mode*.

In Daemon Mode, Remind does not issue timed reminders when it starts up. Instead, it merely queues them and triggers them at the trigger time. It also periodically checks to see if the script has changed and re-executes itself if so. On Linux, it immediately detects changes to the script using `inotify`. It also re-executes itself when the date changes. Finally, Remind does not fork and put itself in the background to handle queued reminders if you supply the `-z` flag; instead, it stays in the foreground.

As an example, suppose your user-ID is `user` and you have placed the light-switch script in `/home/user/home-automation.rem`. You could use the following `systemd` unit to start it on boot:

```
[Unit]
Description=Remind script for home automation
After=network.target

[Service]
WorkingDirectory=/home/user
User=user
Type=simple
ExecStart=/usr/local/bin/remind -z /home/user/home-automation.rem
TimeoutStartSec=0
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

If your computer does not use `systemd`, then you simply need to come up with a startup script that runs `Remind` as shown in the `ExecStart` line. You might need to append an `&` to put the script into the background, and potentially redirect standard output and standard error somewhere useful.

17.2 The shell function

`Remind` has a built-in function that executes a shell command and returns whatever was printed to standard output:

- `shell(cmd [, maxlen])` – given a `STRING cmd`, run it as a shell command and return the first `maxlen` characters that it emits to standard output. Any whitespace in the output is converted to a space. The trailing newline (if any) is removed.

If the `INT maxlen` is omitted, it defaults to 511. If it is supplied as a negative number, then it defaults to the system variable `$MaxStringLen`, which is the maximum allowable length of a string (see the **remind(1)** man page.)

Here are examples of ways to use `shell`:

```
# hostname is set to the full name of the machine
SET hostname shell("hostname -f")

# me is set to the logged-in user-ID
SET me shell("logname")
```

17.3 Safety

The ability to run shell commands is very powerful, but it also means you need to be wary of running Remind scripts you receive from third parties. Remind has a number of safety features to mitigate the risk:

- The `-r` command-line option unconditionally disables the ability to run shell commands. If you invoke Remind with this option, then RUN-type reminders, the `shell()` function, and the `INCLUDECMD` command all fail with the error “RUN disabled.”
- If Remind reads a script that is not owned by the user invoking Remind, then RUN will be disabled during the execution of that script.
- Remind will flat-out refuse to run a script that is world-writable.
- If Remind is run as `root`, which is *emphatically not recommended*, it will not run any scripts that are not owned by `root`.
- You can explicitly disable RUN with the command `RUN OFF` and re-enable it with `RUN ON`. However, `RUN ON` works *only* at the top-level script and not within an included script. So the following could be used to handle a third-party script:

```
# This is the top-level file. We don't quite trust thirdparty.rem
RUN OFF
DO thirdparty.rem
RUN ON
```

If `thirdparty.rem` tries to enable RUN with `RUN ON`, it will fail because it is not the top-level script.

- If a function is defined while RUN is disabled, then RUN will also be disabled during the evaluation of the function. For example, the following script will fail with a “RUN disabled” error:

```
RUN OFF
FSET s(x) shell(x)
RUN ON
REM MSG [s("echo hello")]
```

Even though RUN is enabled when `s()` is evaluated, because it was disabled when `s()` was *defined*, Remind refuses to execute `shell()`.

- If RUN was disabled at the point when a reminder is queued, it will remain disabled when the reminder is issued, even if it is re-enabled before the end of the reminder script.

Chapter 18

Interoperability

While Remind is a great calendar for a Linux or UNIX desktop, most people have mobile devices with their own calendar software. Synchronizing calendar data between Remind and mobile devices is therefore quite desirable.

18.1 iCalendar and CalDAV

Most calendar tools exchange calendar data in a format called *iCalendar*, specified in RFC 5545 available at <https://www.rfc-editor.org/rfc/rfc5545>. The iCalendar format is a pretty flexible format for specifying events and tasks, and can handle many types of recurrences. iCalendar data is often stored in files whose names end with `.ics`. The term “iCalendar” is often shortened to “iCal” even though the official name of the specification is “iCalendar”.

CalDAV is a standard for exchanging iCalendar data over HTTP or HTTPS. It is used both to download events into a client that then displays the events on a calendar, and to upload events to synchronize any modifications made on the client with the data stored on the calendar server.

Remind itself does not include tools to synchronize calendar data, but there are several third-party tools to make this happen. They rely on converting Remind events to iCalendar data or vice-versa.

A good set of tools for converting from iCalendar to Remind and vice-versa is the Python `remind` library at <https://pypi.org/project/remind/> maintained by Jochen Sprickerhof. This includes Python libraries for doing the conversion as well as `rem2ics` and `ics2rem` command-line tools. In the following section, I will describe how to use the Python tools. On my Debian 13 machine, I installed them as root with the following command:

```
# pip3 install --break-system-packages remind
```

Despite the ominous `--break-system-packages`, no system packages were harmed.

18.2 Importing Calendars into Remind

To import a Google or Apple calendar into Remind, you first have to export it as an iCalendar file. You can get a link to your calendar by going to “Settings” for the specific calendar and copying the “Secret address in iCal format” link, which will look something like this:

```
https://calendar.google.com/calendar/ical/myaddr%40gmail.com/private-9039db38b8983b891b8923b89821b092/basic.ics
```

This will give you a URL that you can fetch automatically with `wget` or `curl`. Be sure not to give out the secret link; anyone who has it can see your calendar!

If you use other calendar systems, such as Apple’s calendar, you need to research to figure out how to get the calendar in iCalendar format; there should be a way.

Once you have the iCalendar file, convert it to Remind by running:

```
$ ics2rem icalfile.ics import.rem
```

The file *import.rem* will contain your calendar in Remind format. You can automate this by creating a script to do the conversion and running it periodically from `cron`.

You can get help about additional options for `ics2rem` by running:

```
$ ics2rem --help
```

18.3 Exporting Calendars From Remind

To export your Remind calendar to iCalendar format, use the `rem2ics` command:

```
$ rem2ics file.rem file.ics
```

Again, you can get help by running:

```
$ rem2ics --help
```

Because the types of recurrences that Remind can support are far richer than those that can be expressed by the iCalendar format, `rem2ics` directly converts only reminders with simple recurrences, such as weekly recurrences, to an `RRULE` property. For other reminders, it essentially runs `remind -ppp` and specifies a list of dates with an `RDATE` property attached to the event.

Once you have your Remind events in iCalendar format, you should be able to import them to your Google or Apple calendar. I recommend keeping a separate calendar dedicated to imported Remind events.

Jochen Sprickerhof also maintains a `remind-caldav` tool at <https://github.com/jspricke/remind-caldav> that lets you synchronize Remind calendars to and from CalDAV servers. This lets you automatically synchronize your Google and Remind calendars. I don’t recommend two-way synchronization. Instead, keep a separate reminder file for downloading your main Google calendar, and a separate Google calendar for uploading whatever reminder files you want to synchronize to Google.

18.4 Running Radicale

Rather than using a cloud provider such as Google, Apple or Microsoft to store your calendar, you can run your own CalDAV server. A good choice is Radicale, found at <https://radicale.org/>.

Radicale is an easy-to-configure CalDAV server written in Python. It stores calendar data in individual iCalendar files under a specific directory, and serves them to clients using the CalDAV protocol. You can also update the files using CalDAV if you have sufficient permission.

remind-caldav works with any CalDAV server. On my Web site, I describe how I set up my own Radicale server and use it to synchronize my mobile devices with each other and with my Remind installation on my workstation. The article is at <https://dianne.skoll.ca/writings/de-googling-my-life/>.

18.5 Radicale Remind Storage

Jochen Sprickerhof has a project called Radicale Remind Storage at <https://github.com/jspricke/radicale-remind> that offers an alternate way to keep your Remind calendars synchronized with mobile devices.

radicale-remind is a plugin for the Radicale CalDAV server. Rather than storing events as individual iCalendar files, radicale-remind offers a storage back-end that stores events as Remind scripts. That is, if you create an event using CalDAV, it ends up creating a REM command in a Remind script.

The Remind scripts created by radicale-remind may be used directly by Remind, and serve as the single “source of truth” for your calendar events. Modifying the Remind scripts automatically modifies the events served by Radicale using CalDAV, and modifying a calendar entry via CalDAV automatically updates the Remind script.

Chapter 19

Localization

Not everybody speaks English or wants calendars with English month and day names. Remind has a localization mechanism called the *translation table* that lets you translate the normal English words and phrases that it outputs into another language.

Note that localization is applied only to *output*. If you localize your version of Remind for the Dutch language, for example, it will happily output “maandag” instead of “Monday”, but in your reminder scripts, you still have to specify day names and month names with their English names; Remind will *not* recognize “maandag” in *input*.

19.1 The TRANSLATE Command

To add a string to the translation table, use the TRANSLATE command. The syntax is:

```
TRANSLATE "English-phrase" "Translated-phrase"
```

For example, if you want to translate all of the day names from English to Dutch, you could use:

```
TRANSLATE "Sunday" "zondag"  
TRANSLATE "Monday" "maandag"  
TRANSLATE "Tuesday" "dinsdag"  
TRANSLATE "Wednesday" "woensdag"  
TRANSLATE "Thursday" "donderdag"  
TRANSLATE "Friday" "vrijdag"  
TRANSLATE "Saturday" "zaterdag"
```

If we execute the above commands in a script followed by:

```
REM MSG Vandaag is %w.
```

and we run the script on Monday, 2026-02-02, we get:

```
Vandaag is maandag.
```

19.2 Functions and Filter Sequences

Remind has a single built-in function for doing a translation lookup as well as a special substitution sequence that also does translation lookups. The function is `_`, a single underscore that takes one STRING argument, and the substitution sequence is `%(string)`.

Examples are probably the best way to illustrate how they work:

```

TRANSLATE "Sunset" "Zonsondergang"
TRANSLATE "Monday" "maandag"

REM MSG [_("Monday")]           Outputs "maandag"
REM MSG %(Monday)              Outputs "maandag"

SET a _("Tuesday")            Sets a to "Tuesday"
REM MSG %(Monday) %(Tue)       Outputs "maandag Tue"

SET a _("Sunset")              Sets a to "Zonsondergang"
SET a _("sunset")              Sets a to "zonsondergang"

```

Note a few things:

- If a string is not in the translation table, then both the function `_("string")` and the substitution sequence `%(string)` simply return *string* unmodified.
- If a lower-case string such as `sunset` does not exist in the translation table, then Remind tries the mixed-case version `Sunset`. If that exists, then the result is converted to lower case and returned, as in the last example above. Note, however, that this *only* works if the first letter is ASCII.
- If a lower-case string such as `sunset` exists in the translation table and you ask for a translation of `Sunset`, then Remind tries the lower-case version `sunset`. If that exists, then the result is converted to mixed case and returned. Again, this *only* works if the first letter is ASCII. If you need translations for case variants of non-ASCII words, you should explicitly add `TRANSLATE` commands for each variant.

19.3 Localizing the Substitution Filter

The simple TRANSLATE mechanism is not good enough to localize substitution sequences such as %b. Recall that %b is replaced by “today”, “tomorrow”, or “in *n* days’ time” depending on the relationship between today and the trigger date.

To handle substitution filter localization, Remind lets you define *callback functions* that override the normal operation of the filter sequence. These are simply user-defined functions named subst_*x*, where *x* is the substitution sequence you wish to override.

For example, let’s say we want to localize %b to Dutch. Consider this example:

```
FSET subst_b(alt, date, time) \
    iif(date == $U, "vandaag", \
        date == $U+1, "morgen", \
        "over " + (date-$U) + " dagen")

REM 26 Feb 2026 +10 MSG Tandarts afspraak %b
```

The results of running the above file are:

```
# On 2026-02-23
Tandarts afspraak over 3 dagen

# On 2026-02-24
Tandarts afspraak over 2 dagen

# On 2026-02-25
Tandarts afspraak morgen

# On 2026-02-26
Tandarts afspraak vandaag
```

The substitution callback subst_*n* is passed three arguments in the following order:

- *alt* is an INT which is 0 if the substitution sequence was %*n* or 1 if it was %**n*. The alternate forms of substitution slightly change the output; see the **remind**(1) man page for details.
- *date* is a DATE which is the reminder’s trigger date.
- *time* is a TIME. If the reminder has an AT clause, then *time* is the trigger time. Otherwise, it is set to now().

The return value of the substitution function replaces the original substitution sequence.

19.3.1 Simplified Callbacks for Today/Tomorrow/Something Else

Many callbacks such as %b return “today”, “tomorrow”, or something else, depending on if the trigger date is today, tomorrow, or further in the future. You can simplify your localization as follows:

```
TRANSLATE "today" "vandaag"
TRANSLATE "tomorrow" "morgen"

FSET subst_bx(alt, date, time) "over " + (date-$U) + " dagen"

REM 26 Feb 2026 +10 MSG Tandarts afspraak %b
```

This produces exactly the same output as the previous version. But note that the subst_bx function is much simpler than the earlier subst_b function. That is because it is called *only* for cases that don’t match “today” or “tomorrow”.

The substitution sequences that output “today”, “tomorrow” or something else and that meaningfully support the subst_nx variants are: %a, %b, %c, %e, %f, %g, %h, %i, %j, %k, %l, %u, and %v. See the **remind**(1) man page for details about those substitution sequences.

The substitution sequences %:, %!, %?, %@, and %# contain characters that are illegal in a function name. They may be overridden with functions names subst_colon, subst_bang, subst_question, subst_at, and subst_hash, respectively, and also support variants with the letter x appended.

19.3.2 Custom Substitution Callbacks

In addition to overriding built-in substitution sequences, you can create new ones of your own. Suppose you have a reminder every Wednesday, but if the Wednesday happens to fall on the 18th of a month, you want to suffix it with “ - WOOT!”. Here’s how you could do it. Suppose we create a file called `wedwoot.rem` with the following contents:

```
FSET subst_wedwoot(alt, date, time) iif(day(date)==18, " - WOOT!", "")
REM Wednesday MSG Meeting%{wedwoot}
```

Then running the command:

```
$ remind -s wedwoot.rem 2026-02-01
```

gives us:

```
2026/02/04 * * * * Meeting
2026/02/11 * * * * Meeting
2026/02/18 * * * * Meeting - WOOT!
2026/02/25 * * * * Meeting
```


The substitution sequence `%{anything}` will attempt to call a function `subst_anything` with the usual `alt`, `date` and `time` arguments, and its return value will replace the entire sequence.

19.4 Overriding `ord()`

As we saw in Section 6.9.1 on page 49, the `ord(n)` function returns a string that is the ordinal number *n* (for example, "1st", "13th", "423rd", etc. For non-English languages, you can override the behavior of `ord` by defining your own `ordx` function. If this function is defined, then calling `ord` ends up calling `ordx` instead.

Some languages have quite complicated rules for generating ordinal numbers; here is an excerpt from the Finnish localization script that ships with Remind:

```
FSET ord_helper(d) iif(d==1, ":senä", d==2, ":sena", \
                      (d%10)==2 || (d%10)==3 || (d%10)==6 || (d%10)==8, ":ntena", \
                      ":ntenä")
FSET ordx(d) d + ord_helper(d)
```

19.5 How To Localize Remind

Remind ships with a number of localization packages; they are installed in the directory `$SysInclude/lang` and are named something like `lc.rem`, where *lc* is the lower-case two-letter [ISO 639-1 language code](#). You can examine those files to get a feel for how to localize Remind.

To generate a template for making your own localization file, simply run this shell command:

```
$ echo 'TRANSLATE GENERATE' | remind -h - > template.rem
```

(The `-h` command-line option suppresses the `No reminders.` message that Remind normally prints if there are no reminders.)

The generated template has several parts for you to adjust:

19.5.1 LANGID

The special string `LANGID` should always be translated to the ISO 639-1 language code of your translation file. For example, if you are creating a translation file for Swedish, then you should put this at the top of the file:

```
TRANSLATE "LANGID" "se"
```

`LANGID` is extra-special in that if you don't translate it, or you clear out its translation, it reverts to "en". A Remind script can query the language by calling `_("LANGID")`.

19.5.2 The Banner

Next, comes a translation for the banner. Replace the default:

```
BANNER Reminders for %w, %d%s %m, %y%o:
```

with a localized version for your language.

19.5.3 System Variables

Next come a whole lot of system variables holding translations of day names, month names, and various other phrases. These are leftovers from when the only way to localize Remind was to set various system variables rather than using the TRANSLATE command. The template uses system variables for consistency with existing language packs. However, the following pairs of commands are *completely equivalent*:

```
SET $Monday "lundi"  
TRANSLATE "Monday" "lundi"
```

```
SET $June "Junio"  
TRANSLATE "June" "Junio"
```

Setting the system variable will also create a TRANSLATE entry, and translating the English default value of the system variable will also update the system variable.

19.5.4 Messages

Finally, there is a large section for translating error messages and other messages issued by Remind. Update the translations by editing the second quoted string in each TRANSLATE command.

If the English version of a string has printf-style escape sequences (%x), then the localized version must have the *same* escape sequences in the *same* order as the English version, or the translated version will not be used and Remind will issue a warning. I realize that requiring the same order can be a bit constraining, but that's something translators have to live with.

In addition to the foregoing, a proper localization script will also localize the substitution filter. Check the language files in \$SysInclude/lang/ for examples.

19.6 More on TRANSLATE

In addition to the syntax shown in [Section 19.1 on page 137](#), there are other ways to use TRANSLATE, as shown below:

```
# Translate "string" to "tekenreeks"
TRANSLATE "string" "tekenreeks"

# Remove the translation entry for "string"
TRANSLATE "string"

# Remove all translation table entries
TRANSLATE CLEAR

# Emit TRANSLATE commands to standard output
# that reflect the current translation table
TRANSLATE DUMP

# Generate a translation table template on standard output
TRANSLATE GENERATE
```

19.7 Back-Ends

If you run Remind with the `-p`, `-pp` or `-ppp` option, it transmits the entire translation table to the back-end in JSON format. Back-ends are expected to respect the translations (and all of the back-ends that ship with Remind do so.) Thus, if you localize Remind, you also localize TkRemind, Rem2PDF, and Rem2HTML “for free”.

Chapter 20

Debugging Remind Scripts

Sometimes, you write a Remind script and it just doesn't give the results you want or expect. Remind has debugging facilities to help you better understand why a script runs the way it does.

20.1 Debug Flags

The `-d` command-line option sets one or more *debug flags*. The syntax of the option is as follows: `-dflags`, where *flags* is a sequence of one or more of the following letters:

- `t` – Display all trigger-date computations.
- `x` – Trace all expression evaluation.
- `f` – Trace how Remind opens files (i.e., `INCLUDE` and similar statements).
- `e` – Echo all input lines as they are read.
- `l` – If an error occurs, echo the line that caused it.

There are many more debugging flags (documented in the **remind(1)** man page), but most of them are a bit esoteric or only of interest if you are modifying the source code of Remind itself; the above five flags are the most useful.

All debugging output is sent to standard error.

Debugging output can be *very* voluminous, so Remind has a `DEBUG` command that lets you turn on or of debug flags. For example, if you want to trace trigger computation just for one specific `REM` command, you can put this in your script:

```
# Enable the "t" debug flag
DEBUG +t

REM tricky-trigger-spec MSG Whatever...

# Disable the "t" debug flag
DEBUG -t
```

Use `DEBUG +flag...` to enable the corresponding flag or flags and `DEBUG -flag...` to disable them.

20.1.1 Sample of t Debugging Output

Consider the following script, `tdebug.rem` that contains our old friend, the Friday the 13th reminder:

```
REM 13 SATISFY [$Tw == 5] MSG Friday the 13th!
```

If we run the following command:

```
$ remind -dt tdebug.rem 1 December 2025
```

Then the output is:

```
tdebug.rem(1): Trig = Saturday, 13 December, 2025
tdebug.rem(1): Trig = Tuesday, 13 January, 2026
tdebug.rem(1): Trig = Friday, 13 February, 2026
tdebug.rem(1): Trig(satisfied) = Friday, 13 February, 2026
```

You can see that the first possible trigger date was 2025-12-13. That didn't satisfy the expression, so Remind kept trying until it found one that satisfied the `SATISFY` expression (as indicated by "Trig(satisfied)"). The final trigger date is 2026-02-13.

20.1.2 Sample of x Debugging Output

Suppose we have a reminder file `xdebug.rem` that contains the following

```
FSET sumsq(x, y) x*x + y*y

SET a max(sumsq(1+2, 2+3), iif(today() > '1990-01-01', 14*2, 97*3))
```

If we run:

```
$ remind -dx xdebug.rem 1 December 2025
```

Then the result is as follows; I have added the “#” annotation lines.

```
# First calculate the arguments to sumsq
1 + 2 => 3
2 + 3 => 5

# Call sumsq
Entering UserFN sumsq(3, 5)
x => 3
x => 3
3 * 3 => 9
y => 5
y => 5
5 * 5 => 25
9 + 25 => 34
Leaving UserFN sumsq(3, 5) => 34

# Evaluate the first argument of iif
today() => 2025-12-01
2025-12-01 > 1990-01-01 => 1

# First argument of iif is true, so evaluate second argument
14 * 2 => 28
iif(1, 28, ?) => 28

# Finally evaluate the max of the sumsq return value and the iif return value
max(34, 28) => 34
```

Note how the `iif` line shows (via a `?`) that the third argument was *not* evaluated. Because the `iif` condition is true, Remind never bothered evaluating the third argument `97*3`.

20.2 Other Debugging Commands

Remind has a few other commands useful for debugging your scripts:

```
# Dump the OMIT context to standard error  
OMIT DUMP
```

```
# Dump specific variables to standard error  
DUMP a b c xyz $Latitude
```

```
# Dump all normal variables to standard error  
DUMP
```

```
# Dump all system variables to standard error  
DUMP $
```


Chapter 21

Going Further

You now have a thorough understanding of Remind. But this book still hasn't covered all of the features of Remind such as purge mode, daemon mode, server mode, and a whole lot of other system variables and built-in functions. As always, I refer you to the **remind(1)** man page.

Remind is an open-source program released under the terms of the GNU General Public License, Version 2. As such, I encourage you to read its source code, found under the `src/` directory in the source package. Some of the code is old and convoluted, such as the trigger calculation algorithms in `trigger.c` and `dorem.c`, and I'm not too proud of it.

But some of the code is newer and better, such as the expression-evaluation engine in `expr.c` and I'm very proud of that.

If you like Remind, I encourage you to use it. And if you really like it, why not write programs that interoperate with it, such as `rem2pdf`, `tkremind` or the ICS-to-Remind and Remind-to-ICS tools? Perhaps Remind will spark your imagination and get you started on your own programming projects.

21.1 Example Project

Here's an example of a project I made recently that uses Remind. I wanted a digital clock in my bedroom, but I wanted one that automatically adjusted for Daylight Saving Time, and I also wanted one that would synchronize to NTP servers on the Internet so it always displayed an accurate time.

As it happens, I had a spare Raspberry Pi Zero W lying around. I bought a 32x8 LED matrix display and wrote a program to display the time on the LED matrix. Once a minute, this program gets Remind to run a tiny Remind script to determine if the Sun is above or below the horizon. If it's after sunset, but before sunrise, the clock program dims the LED display. Otherwise, it brightens it.

Now, while I could have used a light sensor to adjust the display brightness according to the ambient light, I didn't have a light sensor handy, but I did have Remind handy.

21.2 Wrapping Up

This book, like Remind, is free. However, should you wish to contribute financially to the Remind project, donations are gratefully accepted at Liberapay. The donation link is <https://dianne.skoll.ca/donate.php?rbook>

Appendix A

Getting and Installing Remind

A.1 Prerequisites

To run Remind, you need a UNIX-like operating system. Remind runs on Linux, FreeBSD, OpenBSD, NetBSD, Solaris, Mac OS X, and pretty much any reasonable UNIX-like system. It does not run on Windows, though you can run it in a Linux virtual machine under Windows.

A.2 Obtaining Remind

Many Linux distributions pre-package Remind. Therefore, you can install it with your normal package manager commands (for example, `apt` on Debian-derived systems; `dnf` on RedHat-derived systems, and so on.) However, the version packaged with your operating system may be out of date, so I encourage you to install Remind from source. Not only is this good practice for installing from source in general, but it also ensures you have the most up-to-date version of Remind.

A.2.1 Downloading the Source Code

Remind's home page is dianne.skoll.ca/projects/remind/ and that page has links to download the source code. The source code filename will be something like `remind-xx.yy.zz.tar.gz` where `xx`, `yy` and `zz` are two-digit numbers specifying the primary, secondary, and minor release numbers.

A.2.2 Prerequisites for Building Remind

Remind itself has very few prerequisites—all it needs as a working C build environment. The specifics for obtaining this vary across UNIX systems; consult your system's documentation. Here's what you need for a couple of popular Linux distributions:

- For Debian and Debian-derived systems:

```
# apt install build-essential
```

- For recent Red Hat-like systems:

```
# dnf install @c-development @development-tools
```

If you want readline support so that **remind** - supports interactive command editing, you will need the following:

- For Debian and Debian-derived systems:

```
# apt install libreadline-dev
```

- For recent Red Hat-like systems:

```
# dnf install readline-devel readline
```

If you want to build **rem2pdf**, the tool that produces PDF calendars, you additionally need a Perl environment and the Perl modules **Cairo**, **Encode**, **ExtUtils::MakeMaker**, **Getopt::Long**, **JSON::MaybexS** and **Pango**. These prerequisites also suffice for installing **rem2html**, a program that produces HTML calendars.

Consult your system's documentation to figure out how to install the prerequisites. Again, for a couple of popular Linux distributions:

- For Debian and Debian-derived systems:

```
# apt install libcairo-perl libjson-maybexs-perl libpango-perl perl
```

- For recent Red Hat-like systems:

```
# dnf install perl-Cairo perl-JSON-MaybexS perl-Pango \
perl-base perl-lib perl-libs
```

If you want to install **tkremind**, the graphical front-end for Remind, you need **Tcl/Tk**, the **Noto** fonts, and the **tcllib** library.

- For Debian and Debian-derived systems:

```
# apt install fonts-noto-core fonts-noto-color-emoji \
fonts-noto-extra fonts-noto-ui-core fonts-noto-ui-extra tk tcllib
```

- For recent Red Hat-like systems:

```
# dnf install google-noto-sans-fonts google-noto-serif-fonts \
google-noto-emoji-fonts tk tcllib
```

A.3 Building Remind

The following steps will build Remind. You should *not* run any of the steps below as *root*; instead, run them as an ordinary user.

- Unpack the downloaded tar file:

```
$ tar xvf remind-xx.yy.zz.tar.gz
```

- Change into the unpacked directory:

```
$ cd remind-xx.yy.zz
```

- Configure Remind:

```
$ ./configure
```

By default, Remind will be installed under `/usr/local`. If you prefer it to be installed under `/usr`, run the following configuration command instead:

```
$ ./configure --prefix=/usr
```

- Build Remind:

```
$ make
```

- (Optional) Run the test suite:

```
$ make test
```

A.4 Installing Remind

Installing Remind requires you to run the following command as *root* from within the directory where you just built Remind:

```
# make install
```

A.5 Uninstalling Remind

If you think you will ever want to uninstall Remind, then immediately after installing it, you can type:

```
$ make uninstall-script > ../uninstall-remind.sh
```

(You can and should run the above command as an ordinary user rather than as *root*.)

This creates a shell script called `../uninstall-remind.sh` that will, if run as *root*, uninstall the version of Remind that was just installed.

Appendix B

Calendar Systems

This Appendix is a brief discussion of calendar systems. You don't need to read it to use Remind, but I hope you find it interesting anyway!

Ultimately, our calendars are derived from the motion of objects in the Solar System. The most obvious consequence of planetary motion is our cycle of day and night, caused by the rotation of the Earth around its axis. The time it takes for one full rotation of the Earth with respect to the Sun is called a *solar day* and is the time between two consecutive appearances of the Sun at its highest point in the sky. A solar day does not have a fixed length because the Earth moves in an eccentric orbit around the Sun and spins on an inclined axis. What we think of as a 24-hour period is a *mean solar day* which is the average length of a solar day over a long period of time.

The next most obvious effect of astronomical body motion that we observe is the phase of the Moon. The average time between new moons is called a *synodic month* and was described in detail in [Section 11.3 on page 81](#).

The final cyclical astronomical phenomenon that affects the calendar is the time between two summer solstices (or winter solstices). This is called a *tropical year*.

Now, it would be nice if a solar day, a synodic month, and a tropical year were all related by integer multiples of each other, or at least by simple fractional multiples, but the odds against this happening are... well... astronomical. It turns out that a mean synodic month is approximately 29.530589 mean solar days, and a tropical year is approximately 365.24217 mean solar days or 12.3683 mean synodic months.

B.1 Reconciling Days, Months and Years

It would be very annoying to have months that are 29.530589 days long and years that are 365.24217 days long, since the month and the year would change partway through the day and at varying times of the day. So most calendar systems have months and years that comprise an integer number of days, and they make integer-sized adjustments to try to compensate for the irrational ratios between years, months

and days.

There are three basic approaches most calendar systems take:

1. Try to make months closely correspond to synodic months, and don't care about aligning with the tropical year.

This is the approach taken by the Islamic calendar, whose average year length is about 354.367 days. As such, the calendar regresses through the seasons every 33 tropical years or so.

2. Try to make months closely correspond to synodic months, and years to closely correspond with the tropical year. This is the approach taken by the Hebrew and Chinese calendars; they are called *lunisolar calendars* and are based on the Metonic cycle described in Section 11.3 on page 81.

Unfortunately, the Metonic cycle of 235 synodic months isn't *exactly* equal to 19 tropical years, so lunisolar calendars also show a slight drift with respect to the tropical year. For example, the Hebrew calendar drifts by about one day every 216 years.

3. Try to make years correspond closely to the tropical year and don't worry about lining up months with synodic months. This was the approach taken by the Julian Calendar, and is the approach used by the Gregorian Calendar, which is the common civil calendar in most parts of the world and is what most people simply think of as "the calendar".

The Gregorian year is normally 365 days long. However, every so often, a day is added to the year (February 29th) making it 366 days long; such a year is called a *leap year*. The Gregorian rules for determining a leap year are as follows:

- If the year is a multiple of 400, then it is a leap year.
- Otherwise, if the year is a multiple of 100, then it is *not* a leap year.
- Otherwise, if the year is a multiple of 4, then it is a leap year.
- Otherwise, it is not a leap year.

These rules result in a 400-year cycle of 146 097 days for an average year length of 365.2425 days. This is pretty close to the tropical year length of 365.24217 days... but it's not exact. Even the Gregorian calendar drifts by about one day every 3030 years or so.

To make matters worse, the rotation of the Earth is slowing down because some of the Earth's angular momentum is being transferred to the Moon via tidal interactions. The rate of deceleration is tiny (currently about 1.8 milliseconds per century) but over very long periods of time, it will make the day meaningfully longer and thus the tropical year will contain fewer mean solar days.

B.2 Year Numbering

The year number in a calendar (for example, as I write this, it is 2026 according to the Gregorian calendar and 5786 according to the Hebrew calendar) has to be related to some event in the past. The Gregorian year is supposed to be related to the birth of Jesus Christ, which to the eternal annoyance of computer scientists is denoted by the year 1 rather than the year 0.

The numbering of Hebrew years is supposed to start with the creation of the Universe, also presumably in year 1. This is quite likely to be inaccurate to the tune of 13.8 billion years or so.

But really, it doesn't matter what the reference point is for year numbering, as long as everyone who uses a given calendar system agrees on the year number.

B.3 Calendar Systems supported by Remind

Remind supports the Gregorian calendar as its main calendar system, and the Hebrew calendar as a secondary system. Remind's algorithms should be good enough for the next few hundred years, but if you are reading this book in the far distant future relative to 2026, feel free to submit patches if the calendar has been reformed.

Appendix C

Bibliography

- Nachum Dershowitz and Edward M. Reingold, “Calendrical Calculations: The Ultimate Edition”, Cambridge University Press, 2018-03-31. ISBN 978-1107683167.

This book contains a comprehensive description of many calendar systems as well as algorithms to implement them on a computer.

- Jean Meeus, “Astronomical Algorithms”, Willmann-Bell, 2nd edition, 1998. ISBN 978-0943396613,

This book contains algorithms for computing astronomical events such as sunrise, sunset, moon phases, etc.

- LeRoy E. Doggett, George H. Kaplan, P. Kenneth Seidelmann, “Almanac for Computers for the year 1978”, United States Naval Observatory.

This publication contains astronomical tables and algorithms that form the basis of Remind’s sunrise and sunset functions.

- Bernard Desruisseaux (editor), “Internet Calendaring and Scheduling Core Object Specification (iCalendar)”, September 2009.

This is RFC 5545 and specifies the iCalendar interchange format. It is available online at <https://www.rfc-editor.org/rfc/rfc5545>.

Index

- addition, [45](#)
- ADDOMIT, [29](#)
- advance warning, [14](#)
 - timed reminders, [35](#)
- AFTER keyword, [22](#)
- agenda mode, [9](#), [10](#), [20](#), [34](#), [78](#), [90](#)
 - sorted output, [36](#)
- alternate forms of back and delta, [24](#)
- arbitrary recurrences, [27](#)
- arithmetic functions, [49](#)
- arithmetic negation, [44](#)
- astronomical events, [79](#)
- AT, [33](#)

- back, [17](#)
- back, alternate form, [24](#)
- banner, [16](#), [142](#)
- BEFORE keyword, [22](#)
- binary operators, [43](#)
- blank lines, [8](#)
- blue moon, [83](#)
- built-in functions, [48](#)
 - `_`, [138](#)
 - `abs`, [49](#)
 - `adawn`, [80](#)
 - `adusk`, [80](#)
 - `asc`, [49](#)
 - `catch`, [52](#)
 - `char`, [49](#)
 - `choose`, [51](#)
 - `codepoint`, [49](#)
 - `current`, [51](#)
 - `dawn`, [80](#)
 - `day`, [51](#)
 - `dusk`, [80](#)
 - `easterdate`, [55](#), [84](#)
 - `hebdate`, [103](#)
 - `hebdays`, [102](#)
 - `hebmon`, [102](#)
 - `hebyear`, [103](#)
 - `hour`, [51](#)
 - `iif`, [52](#)
 - `index`, [49](#)
 - `isomitted`, [51](#)
 - `ivritmon`, [103](#)
 - `localtoutc`, [127](#)
 - `max`, [49](#)
 - `mbchar`, [49](#)
 - `mbindex`, [50](#)
 - `mbpad`, [50](#)
 - `mbstrlen`, [50](#)
 - `mbsubstr`, [50](#)
 - `min`, [49](#)
 - `minute`, [51](#)
 - `mon`, [51](#)
 - `monnum`, [51](#)
 - `moondate`, [81](#)
 - `moondatetime`, [82](#)
 - `moonphase`, [81](#)
 - `moonrise`, [82](#)
 - `moonrisedir`, [82](#)
 - `moonset`, [82](#)
 - `moonsetdir`, [82](#)
 - `moontime`, [82](#)
 - `ndawn`, [80](#)
 - `ndusk`, [80](#)
 - `now`, [51](#)
 - `ord`, [50](#)
 - `orthodoxeaster`, [84](#)
 - `pad`, [50](#)

- plural, [50](#)
- realcurrent, [51](#)
- realnow, [51](#)
- realtoday, [51](#)
- sgn, [49](#)
- shell, [130](#)
- soleq, [83](#)
- strlen, [50](#)
- substr, [50](#)
- sunrise, [80](#)
- sunset, [80](#)
- today, [51](#)
- trigdate, [60](#)
- tzconvert, [127](#)
- utctolocal, [127](#)
- weekno, [51](#)
- wkdaynum, [51](#)
- year, [51](#)
- bytes, [40](#)
- CAL, [129](#)
- CalDAV, [133](#)
- calendar mode, [9](#), [10](#), [20](#), [34](#), [90](#)
- calendars
 - exporting, [134](#)
 - first day of week, [86](#)
 - Hebrew, [101](#)
 - HTML, [89](#)
 - importing, [134](#)
 - in the terminal, [85](#)
 - machine-readable, [87](#)
 - PDF, [88](#)
 - PostScript, [89](#)
 - producing, [85](#)
 - size, [86](#)
 - special data, [90](#)
 - SVG, [89](#)
 - time format, [87](#)
- callback function, [5](#), [67](#)
- case-sensitivity, [9](#)
- character encoding, [9](#)
- character set, [9](#)
- characters, [40](#)
- CLEAR-OMIT-CONTEXT command, [24](#)
- clearing omit context, [24](#)
- COLOR, [93](#)
- command-line options, [10](#)
- commands, [7](#)
- comments, [8](#)
- comparison, [47](#)
- COMPLETE-THROUGH, [77](#), [78](#)
- conditional execution, [72](#)
- constants, [41](#)
 - DATE, [42](#)
 - DATETIME, [42](#)
 - INT, [41](#)
 - STRING, [41](#)
 - TIME, [42](#)
- continued lines, [8](#)
- counting backwards, [17](#)
- credit card due date, [62](#)
- current date and time, [19](#)
- daemon mode, [9](#), [129](#)
- data types, [40](#)
 - DATE, [40](#)
 - DATETIME, [41](#)
 - INT, [40](#)
 - STRING, [40](#)
 - TIME, [40](#)
- DATE, [40](#)
- date and time functions, [51](#)
- DATETIME, [41](#)
- debug flags, [145](#)
 - e, [145](#)
 - f, [145](#)
 - l, [145](#)
 - t, [145](#)
 - x, [145](#)
- debugging, [145](#)
- default color, [98](#)
- delta, [15](#)
- delta, alternate form, [24](#)
- design goal, [2](#)
- directory, reading as a script, [74](#)
- division, [45](#)

- DO, [71](#)
- domain-specific language, [2](#)
- DUMP, [148](#)
- dumping omit context, [24](#)
- DURATION, [35](#)
- durations, [35](#)
- Easter, [83](#)
- effective date and time, [19](#)
- ELSE, [73](#)
- end of quarter, [64](#)
- EOF marker, [7](#)
- equality, [47](#)
- equinox, [83](#)
- exceptions, [21](#)
- expiry date, [28](#)
- exporting calendars, [134](#)
- expression debugging, [146](#)
- expression pasting, [55](#)
- expressions, [39](#)
- fifth weekday of a month, [61](#)
- file inclusion, [71](#)
- first quarter, [81](#)
- flow control, [72](#)
- Friday the 13th, [57](#)
- FROM, [28](#)
- FSET, [52](#)
- full moon, [81](#)
- full omit, [21](#)
- functions
 - arithmetic, [49](#)
 - built-in, [48](#)
 - date and time, [51](#)
 - logical, [51](#)
 - string, [49](#)
 - user-defined, [52](#)
- FUNSET, [54](#)
- Gregorian-to-Hebrew conversion, [102](#)
- GUI, [107](#)
- Hebrew calendar, [101](#)
- Hebrew-to-Gregorian conversion, [103](#)
- history of Remind, [2](#)
- holidays, [21](#)
- holidays, movable, [28](#)
 - safe, [30](#)
- HTML, [100](#)
- HTMLCLASS, [100](#)
- iCalendar, [133](#)
- ics2rem, [133](#)
- IF, [73](#)
- IFTRIG, [74](#)
- importing calendars, [134](#)
- INCLUDE, [71](#)
- INCLUDECMD, [75](#)
- inequality, [47](#)
- INFO, [121](#)
 - substitution sequences, [122](#)
- INT, [40](#)
- interoperability, [133](#)
- JSON output, [11](#)
- LANGID, [141](#)
- last day of month, [17](#)
- last quarter, [81](#)
- LASTDAY, [19](#), [25](#)
- LASTWORKDAY, [25](#)
- latitude, [79](#)
- leap year, [156](#)
- line continuation, [8](#)
- local omits, [23](#)
- localization, [137](#)
- location, [79](#)
- logical and, [47](#)
- logical functions, [51](#)
- logical negation, [44](#)
- logical or, [48](#)
- longitude, [79](#)
- lunar eclipse, [81](#)
- lunisolar calendar, [81](#), [156](#)
- MAX-OVERDUE, [77](#)
- MAYBE-UNCOMPUTABLE, [60](#)
- mean solar day, [155](#)

- Metonic cycle, [81](#), [156](#)
- modes of operation, [9](#)
- modulus, [45](#)
- MOON, [92](#)
- moon phases, [81](#)
- moon-related functions, [81](#)
- moonrise, [82](#)
- moonset, [82](#)
- movable holidays, [28](#)
 - safe, [30](#)
- MSG, [129](#)
- multibyte strings, [40](#)
- multiplication, [45](#)

- new moon, [81](#)
- next mode, [11](#)
- NOQUEUE, [34](#)
- now, [20](#)

- observed holidays, [61](#)
- OMIT, [28](#)
- OMIT command, [21](#)
- omit context, [21](#)
- OMITFUNC, [65](#)
- operators, [43](#)
 - addition, [45](#)
 - arithmetic negation, [44](#)
 - binary, [43](#)
 - comparison, [47](#)
 - division, [45](#)
 - equality, [47](#)
 - inequality, [47](#)
 - logical and, [47](#)
 - logical negation, [44](#)
 - logical or, [48](#)
 - modulus, [45](#)
 - multiplication, [45](#)
 - subtraction, [46](#)
 - unary, [43](#)
- order of evaluation, [44](#)
- Orthodox Easter, [84](#)

- PANGO, [99](#)
- partial omit, [21](#)

- PDF calendars, [88](#)
- POP-FUNCS, [54](#)
- POP-OMIT-CONTEXT command, [24](#)
- POP-VARS, [54](#)
- PRIORITY, [36](#)
- priority, [36](#)
- producing calendars, [85](#)
- purge mode, [9](#)
- PUSH-FUNCS, [54](#)
- PUSH-OMIT-CONTEXT command, [24](#)
- PUSH-VARS, [54](#)
- Python Remind library, [133](#)

- quarter, end of, [64](#)
- queuing reminders, [34](#)

- Radicale, [135](#)
- Radicale Remind Storage, [135](#)
- radicale-remind, [135](#)
- reading a directory, [74](#)
- recycling date, [63](#)
- REM, [11](#)
- rem-today CSS class, [100](#)
- rem2html, [89](#)
- rem2ics, [133](#)
- rem2pdf, [88](#)
- remind algorithm, [14](#)
- Remind design goal, [2](#)
- remind-caldav, [135](#)
- repeat, [27](#)
- restoring omit context, [24](#)
- RUN, [129](#)
- running remind, [9](#)

- safe movable holidays, [30](#)
- SATISFY, [57](#)
- SATISFY algorithm, [58](#)
- SATISFY iteration limit, [59](#)
- saving and restoring user-defined functions, [54](#)
- saving and restoring variables, [54](#)
- saving omit context, [24](#)
- SCANFROM, [30](#)
- server mode, [9](#)
- SET, [39](#)

- SHADE, [91](#)
- SHED, [67](#)
- shell commands, [129](#)
 - safety, [131](#)
- simple output, [11](#)
- SKIP keyword, [22](#)
- solar day, [155](#)
- solstice, [83](#)
- sortbanner, [68](#)
- sorted output, [36](#)
- SPECIAL, [91](#)
- SPECIALs
 - COLOR, [93](#)
 - HTML, [100](#)
 - HTMLCLASS, [100](#)
 - MOON, [92](#)
 - PANGO, [99](#)
 - SHADE, [91](#)
 - WEEK, [93](#)
- starting date, [28](#)
- STRING, [40](#)
- string functions, [49](#)
- structure of a script, [7](#)
- substitution filter, [15](#)
 - custom callbacks, [140](#)
 - localization, [139](#)
- substitution sequences, [16](#)
 - time-related, [34](#)
- subtraction, [46](#)
- sunrise, [80](#)
- sunset, [80](#)
- synodic month, [81](#), [155](#)
- syntactic sugar, [18](#), [25](#)
- SYSINCLUDE, [72](#)
- system date, [20](#)
- system time, [20](#)
- system variable, [58](#)
- system variables, [39](#)
 - \$AddBlankLines, [40](#), [68](#)
 - \$DefaultColor, [98](#)
 - \$DontQueue, [40](#)
 - \$Latitude, [79](#)
 - \$Location, [80](#)
 - \$Longitude, [79](#)
 - \$MaxSatIter, [59](#), [105](#)
 - \$MaxStringLen, [40](#)
 - \$NumTrig, [91](#)
 - \$SysInclude, [72](#)
 - \$Td, [58](#), [60](#)
 - \$TerminalHyperlinks, [123](#)
 - \$Tm, [60](#)
 - \$Tw, [58](#), [60](#)
 - \$Ty, [60](#)
 - \$U, [60](#)
 - \$Ud, [60](#)
 - \$Um, [60](#)
 - \$Uw, [60](#)
 - \$Uy, [60](#)
- TAG, [121](#)
- tags, [121](#)
- TIME, [40](#)
- time zone conversion functions, [127](#)
- time zone names, [128](#)
- time zones, [125](#)
- time-related substitution sequences, [34](#)
- timed reminders, [33](#)
- tkremind, [107](#)
 - adding reminders, [115](#)
 - agenda window, [108](#)
 - editing reminders, [117](#)
 - errors, [118](#)
 - help, [119](#)
 - inotify, [118](#)
 - keyboard shortcuts, [109](#)
 - main window, [107](#), [109](#)
 - options, [111](#)
 - printing, [110](#)
 - queued reminders, [110](#)
- today, [20](#)
- TODO, [77](#), [78](#)
- TODOs, [77](#)
 - in calendar mode, [78](#)
- TRANSLATE, [137](#)
- TRANSLATE GENERATE, [141](#)

- trigger date, [12](#)
- trigger debugging, [146](#)
- trigger-calculation algorithm, [14](#)
- trigger-spec, [12](#)
- tropical year, [155](#)
- true and false values, [42](#)
- TZ, [125](#), [126](#)

- unary operators, [43](#)
- UNSET, [54](#)
- unsetting a function, [54](#)
- unsetting a variable, [54](#)
- UNTIL, [28](#)
- URLs in the terminal, [122](#)
- US election day, [62](#)
- user-defined functions, [52](#)
 - arguments, [52](#)
 - recursive, [53](#)
 - redefinition, [53](#)
 - saving and restoring, [54](#)
 - unsetting, [54](#)

- variables, [39](#)
 - saving and restoring, [54](#)
 - system, [58](#)
 - unsetting, [54](#)

- WARN, [66](#)
- WEEK, [93](#)
- weekday omit, [21](#)

- zero value, [42](#)

Colophon

This book was typeset using the `lualatex` implementation of Leslie Lamport’s $\text{\LaTeX}$ computer typesetting program (based, of course, on Donald Knuth’s $\text{\TeX}$ program.) The main body font is Liberation Serif and the monospace font is Liberation Mono. In a few places, the fonts Noto Color Emoji and Noto Sans Symbols 2 make an appearance.

The figures are either screenshots of Remind output, or generated on the fly with `remind` and `rem2pdf` as part of the creation of the final PDF; this is done with a `Makefile`.

The EPUB version was generated from the same $\text{\LaTeX}$ source files using `tex4ebook`. Only a very small number of tweaks had to be made to the $\text{\LaTeX}$ sources to get them to work properly with `tex4ebook`.

The $\text{\LaTeX}$ input files were prepared with GNU Emacs on a Debian 13 Linux system.

No generative AI was used in the production of this book.